📘 **Project Title: Student Success & Career Guidance Portal**

Platform: Salesforce Education Cloud

Author: Swayam Tamrakar

Date: 26/09/2025

🎯 **Problem Statement**

Colleges and universities struggle with tracking **student performance, internships, and placements**. Data is scattered across multiple systems (LMS, placement cells, manual records), making it difficult to create a **360° student profile**.

Students receive **generic career advice** instead of personalized guidance. Institutions lack **data-driven insights** to improve curriculum and placement strategies.

💡 **Proposed Salesforce Solution**

Building a **Salesforce Education Cloud app** that:

- Centralizes student academic, extracurricular, and internship data.
- Integrates with LMS to track real-time progress.
- Leverages **Einstein AI** to recommend personalized career paths, internships, and certifications.
- Provides dashboards for faculty, placement officers, and management to measure student success and employability.

Project Overview

The *Student Success & Career Guidance Portal* is a Salesforce Education Cloud CRM application designed to help colleges and universities track **student performance, skills, internships, and placements** in one centralized system. The CRM provides a **360° view of each student**, integrates learning management features, and supports placement officers with real-time analytics and dashboards. Using **Einstein AI**, it can recommend career paths and internships that match student skills, improving overall employability.

Key features include:

- Centralized **student records** with academic details and skills.
- **Internship & Placement tracking** with company mapping.
- **Job Sync Flow** to fetch job posts automatically into Salesforce.
- Reports & dashboards for placement officers to monitor student readiness.
- Role-based security to ensure data confidentiality and controlled access.

Objectives

The primary goal of this CRM is to **bridge the gap between students, faculty, and placement officers** by providing a single platform for student success tracking.

Specific objectives include:

- **Student Success Monitoring**: Track GPA, skills, and readiness scores for career guidance.
- **Internship & Placement Management**: Streamline the process of managing job offers and company interactions.
- **Automation & Integration**: Use Salesforce Flows to automate repetitive tasks such as job synchronization, skill updates, and career recommendations.
- **Analytics for Decision-Making**: Provide dashboards showing top skills, placement funnel, and average career readiness for better institutional planning.
- **Scalability & Future Growth**: Ensure the CRM can be extended with new modules (e.g., alumni management, employer feedback) to support long-term needs.

By achieving these objectives, the CRM provides **business value** such as:

- Higher employability for students.
- Streamlined placement operations.

- Data-driven decision-making for colleges.
- Increased transparency and efficiency across departments.

Phase 1: Problem Understanding and Industry Analysis

1.1. Problem Statement

Colleges face challenges in:

Tracking student academic performance, co-curricular activities, and skills in one place.

Managing internships and placement drives effectively.

Connecting students with career opportunities based on their skills and interests.

Offering personalized career guidance that improves student employability.

This leads to:

Manual data management.

Missed internship and placement opportunities.

Low student engagement and unclear career paths.

Our project addresses these issues by building a Salesforce Education Cloud App that:

Maintains a complete student profile.

Integrates with LMS.

Tracks skills, certifications, and attendance.

Uses Einstein AI to recommend career paths, internships, and placements.

1.2. Requirement Gathering

Functional Requirements

Student Management: Create student records with personal info, academic history, skills, and certifications.

Company/Internship Management: Track recruiter companies, internship openings, eligibility criteria, and placement status.

Performance Tracking: Map grades, attendance, and event participation.

Career Guidance: Use AI to suggest suitable career paths based on skill gaps and performance.

Notifications: Send alerts for upcoming interviews, deadlines, or recommended internships.

Non-Functional Requirements

Scalability: Must handle thousands of students and company records.

Data Security: Follow Salesforce's security model (profiles, roles, sharing rules).

User Experience: Offer a simple Lightning UI for students, faculty, and placement officers.

Integration: Ensure smooth integration with external LMS and job portals using REST APIs.

1.3. Stakeholder Analysis

Stakeholder	Role	Expectations
College Admin/Placement Officer	Manages student data, tracks placements	Easy dashboard, scheduling tools, analytics
Faculty	Inputs grades, monitors performance	Simple UI to update student progress
Students	View performance, apply for internships	Personalized career guidance, job alerts
Recruiter Companies	Post internships, hire students	Access to eligible candidates, communication
Management	Decision-making based on reports	Complete view of performance and placement stats

1.4. Business Process Mapping

As-Is Process (Current Situation):

Student data maintained manually (Excel and spreadsheets).

Placement drives coordinated via email and phone.

No centralized performance dashboard.

Students depend on generic career advice.

To-Be Process (After Salesforce Implementation):

Student data stored in Salesforce (centralized).

Automatic flow for internships (eligibility check, scheduling, and status tracking).

Real-time dashboards for college management.

1.5. Industry-Specific Use Case Analysis

Education Industry Trends:

Growing use of Education CRM platforms for student lifecycle management.

Need for data-driven career guidance to enhance placement rates.

Demand for personalized learning paths using AI and machine learning.

Salesforce is widely used in education for:

Student information systems.

Engagement and retention.

Career counseling and alumni management.

Our solution fits these industry needs perfectly.

1.6. AppExchange Exploration

In this phase, we explored Salesforce AppExchange apps such as:

TargetX and EDA (Education Data Architecture): For student lifecycle management.

Blackthorn Events: For scheduling placement drives.

Mulesoft Connectors: For integrating external LMS.

We will use Salesforce Education Data Architecture (EDA) as the base model and customize it for our project.

1.7. Use Cases, Student Success & Career Guidance Portal

Student Management

• Create and maintain student records, including personal info, academic history, skills, and certifications.

• Centralize all performance data in one place for faculty, placement officers, and students to access.

• Track attendance, grades, and participation in events or workshops.

Company & Internship Management

• Maintain a database of recruiter companies with details such as industry, open roles, and locations.

• Record internship and job postings with eligibility criteria, stipend, and duration.

• Track student applications and connect them to companies.

Placement & Internship Scheduling

• Schedule interviews and placement drives for eligible students.

• Send SMS and email notifications and reminders to students about upcoming events.

• Track student attendance and results from each placement round.

Career Guidance & Recommendations

• Use Einstein AI to evaluate student performance and skills.

• Recommend career paths, certifications, and internships based on skill gaps.

• Display recommended opportunities on the student portal or dashboard.

Reporting & Analytics

• Provide a dashboard for placement officers that shows:

- Number of students placed
- Internship success rates
- Skills gap analysis across batches

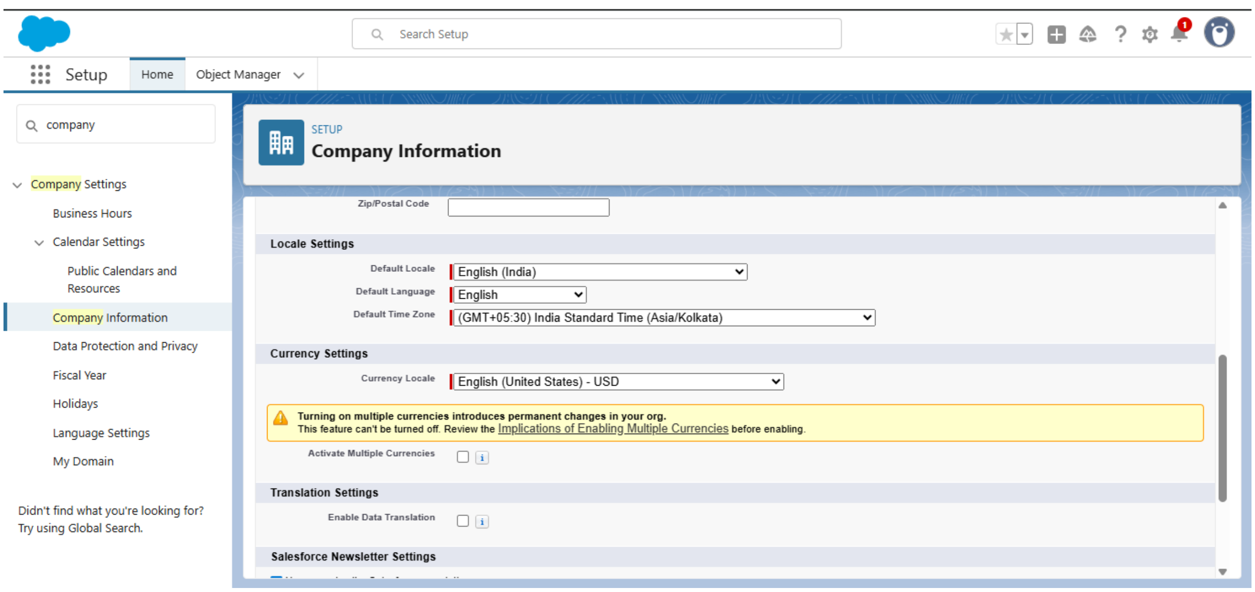
• Track student progress over semesters.

• Offer management real-time insights for decision-making.

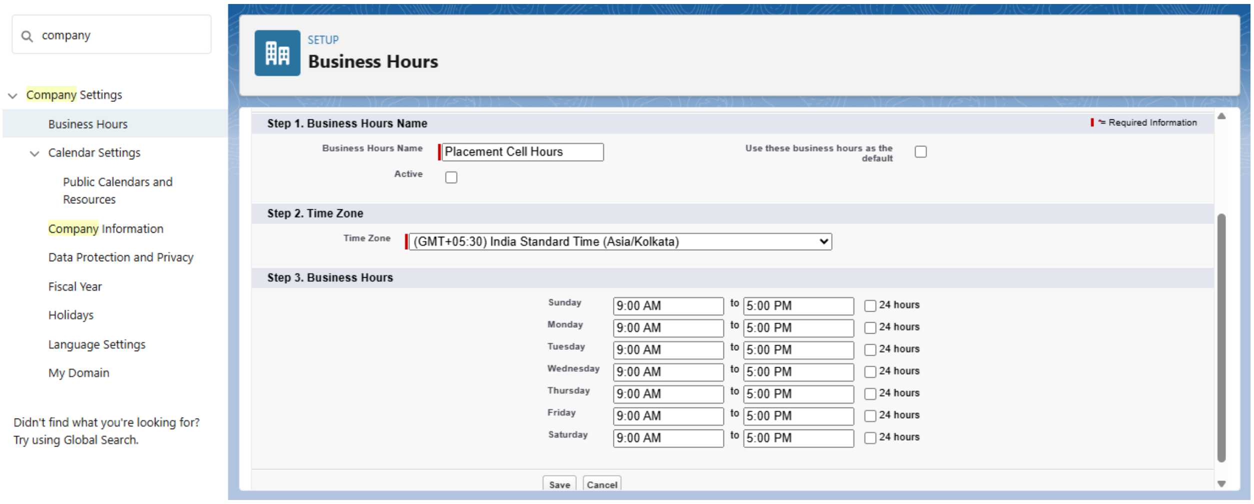
Phase 2 — Org setup & configuration

2.1.Basic org settings

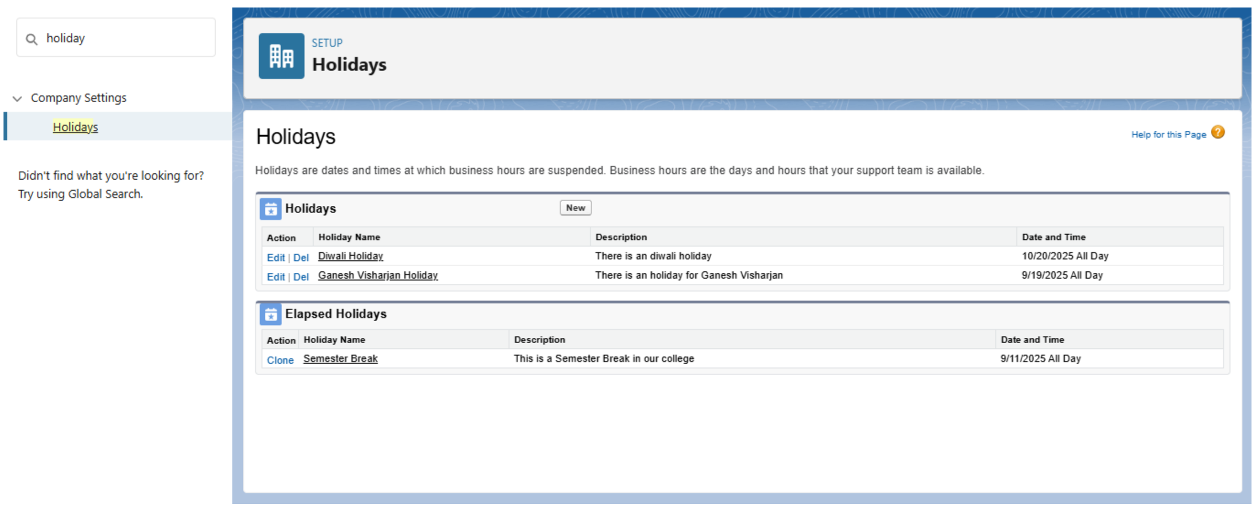
1. **Setup** → *Company Settings* → **Company Information** → update Default Locale, Time Zone, Default Currency.



2. **Setup** → *Business Hours* → **New** → create Placement Cell Hours.



3. **Setup** → *Holidays* → add semester breaks.

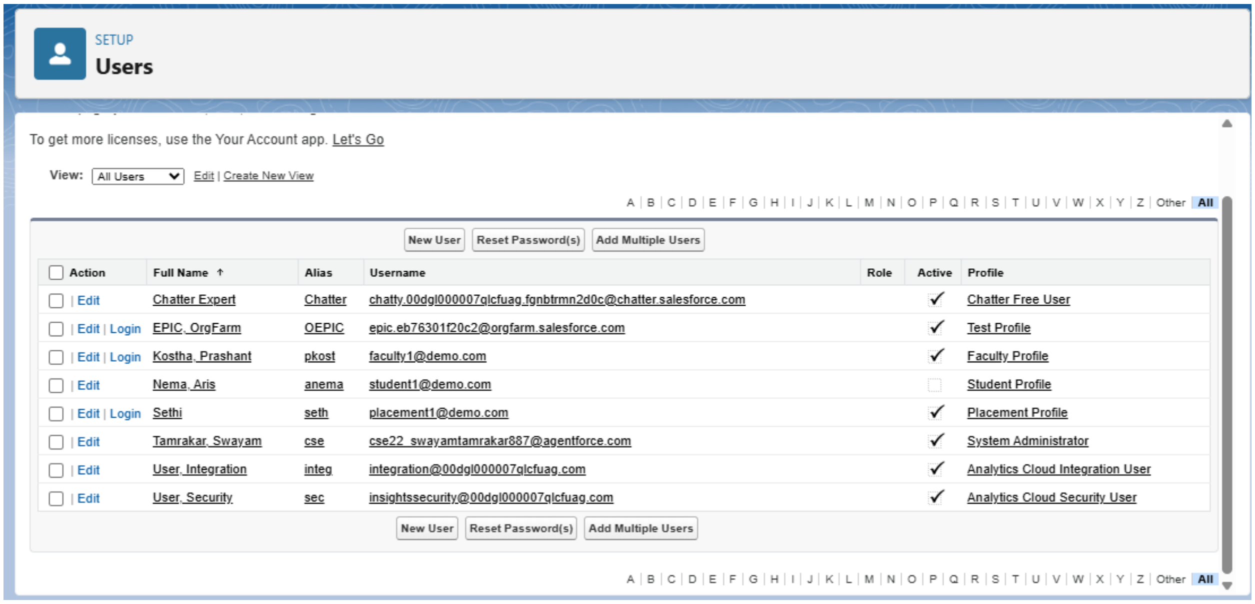


2.2 Enable features

1. **Setup** → *Quick Find* → **Digital Experiences** → enable if you want a student/recruiter portal.

2.3 Users, Profiles, Roles

1. **Setup** → *Users* → **New User** → create sample accounts: student1@, faculty1@, placement1@.



SETUP
Users

To get more licenses, use the Your Account app. Let's Go

View: All Users | Edit | Create New View

A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | Other | All

New User | Reset Password(s) | Add Multiple Users

Action	Full Name ↑	Alias	Username	Role	Active	Profile
Edit	Chatter Expert	Chatter	chatty.00dgl000007qlcfuag.fgnbtrmn2d0c@chatter.salesforce.com		✓	Chatter Free User
Edit | Login	EPIC, OrgFarm	OEPIC	epic.eb76301f20c2@orgfarm.salesforce.com		✓	Test Profile
Edit | Login	Kostha, Prashant	pkost	faculty1@demo.com		✓	Faculty Profile
Edit	Nema, Aris	anema	student1@demo.com			Student Profile
Edit | Login	Sethi	seth	placement1@demo.com		✓	Placement Profile
Edit	Tamrakar, Swayam	cse	cse22_swayamtamrakar887@agentforce.com		✓	System Administrator
Edit	User, Integration	integ	integration@00dgl000007qlcfuag.com		✓	Analytics Cloud Integration User
Edit	User, Security	sec	insightssecurity@00dgl000007qlcfuag.com		✓	Analytics Cloud Security User

New User | Reset Password(s) | Add Multiple Users

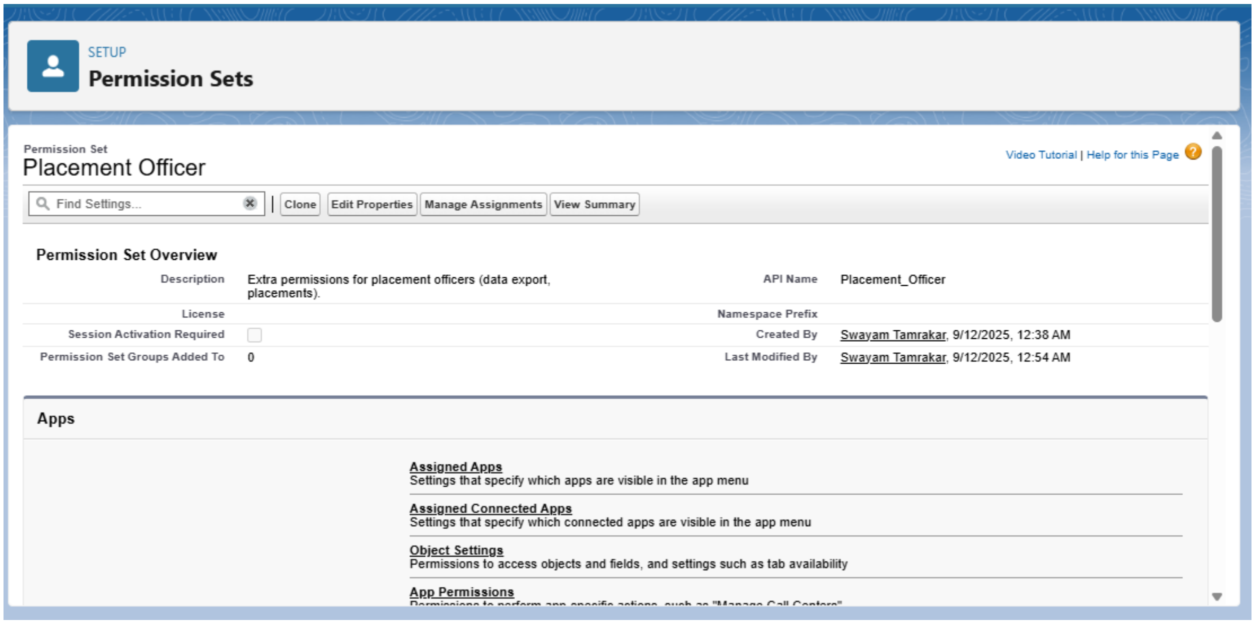
A | B | C | D | E | F | G | H | I | J | K | L | M | N | O | P | Q | R | S | T | U | V | W | X | Y | Z | Other | All

2. **Setup** → *Profiles* → open Standard User → **Clone** → name Student_Profile; repeat for Faculty_Profile, Placement_Profile. Set baseline object permissions (Contacts: read for Students, edit for Faculty).

3. **Setup** → *Roles* → **Set Up Roles** → create hierarchy:
    - Admin
        - Placement Head
            - Placement Officer
        - Faculty
            - Faculty Member

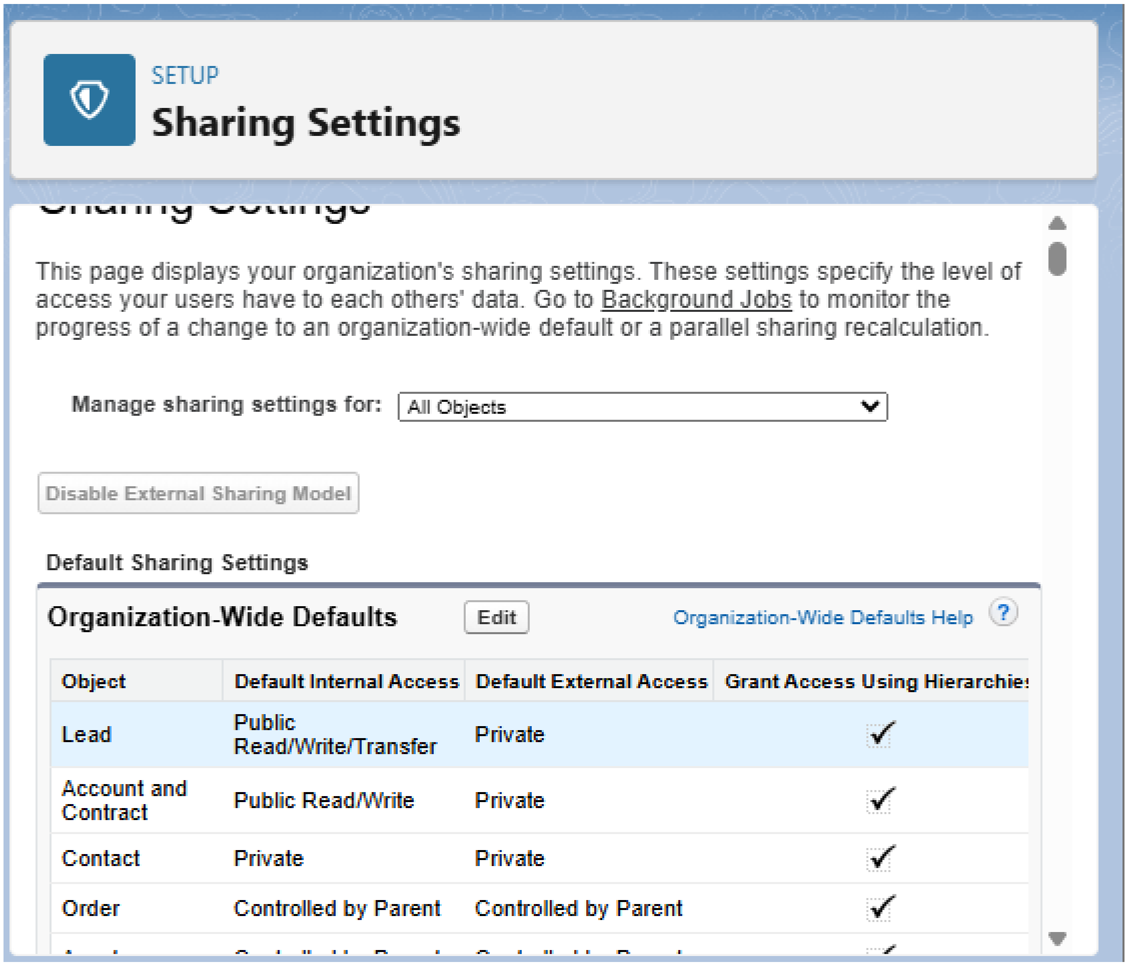
- Student

4. **Setup** → *Permission Sets* → **New** → Placement_Officer_PS — grant object and field-level permissions not in profile (e.g., export data).

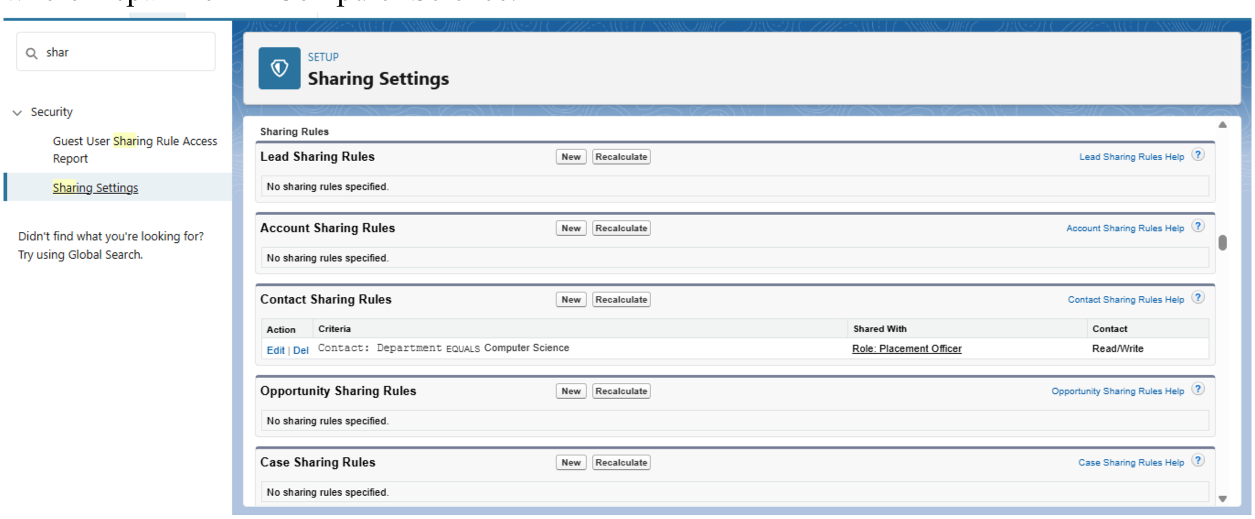


2.4 Security / OWD / Sharing

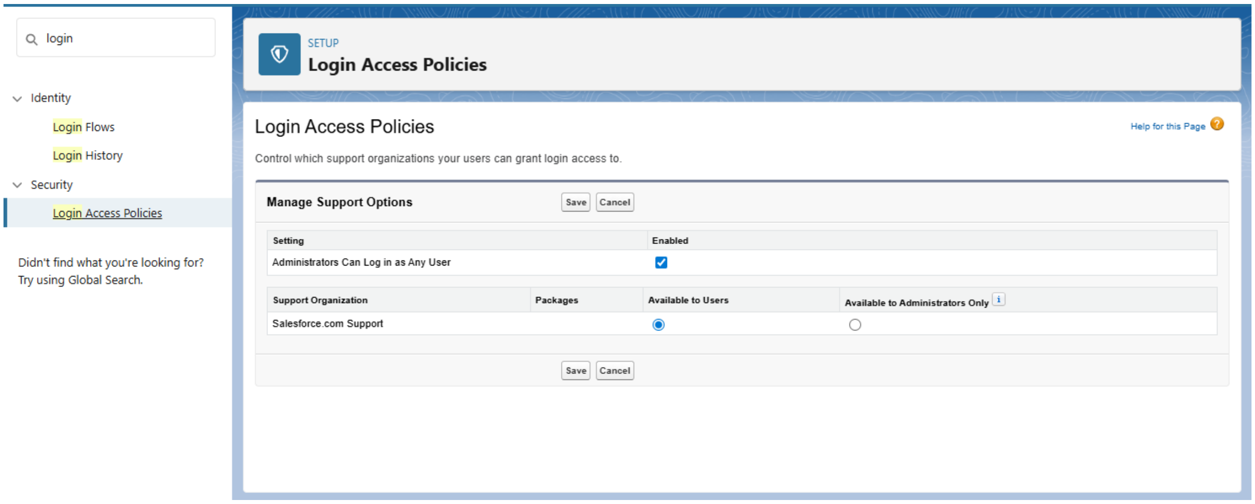
1. **Setup** → *Sharing Settings* → set **Contact** OWD = **Private** (students' personal data protected).



2. Create **Sharing Rules**: e.g., Share Student (Contact) records to Role = Placement Officer where Department = Computer Science.



3. **Setup** → *Login Access Policies* → enable admin access if needed for troubleshooting.



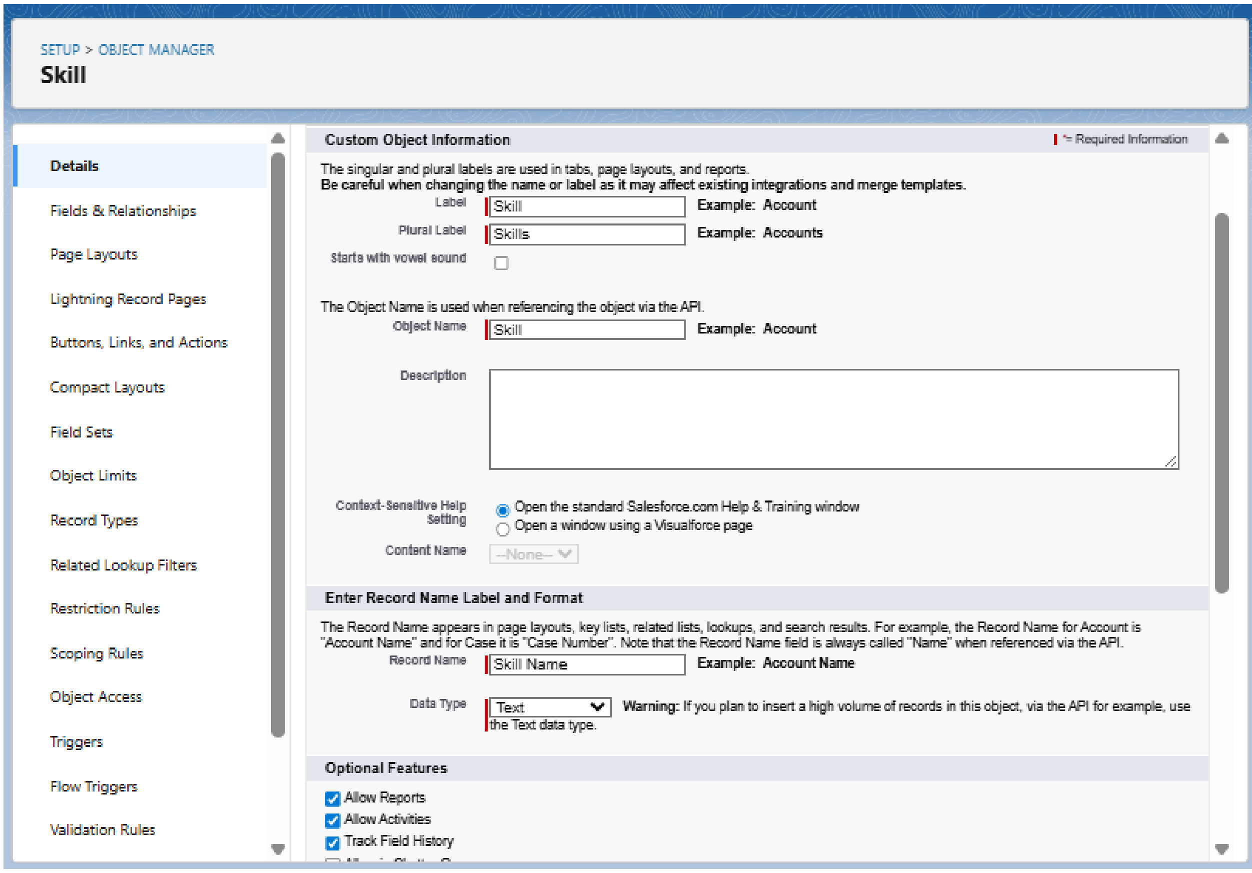
2.5 Dev org & sandbox usage

- Use Developer org for building, Sandbox for UAT. Create a Partial Copy sandbox for more realistic data if available.
- Dev Org Setup
- To implement the following project a Salesforce Developer Edition org was setup.
- Made a GitHub Repository for source control.
- Setting up the VS Code and SFDX for the implementation of the LWC Component for development.

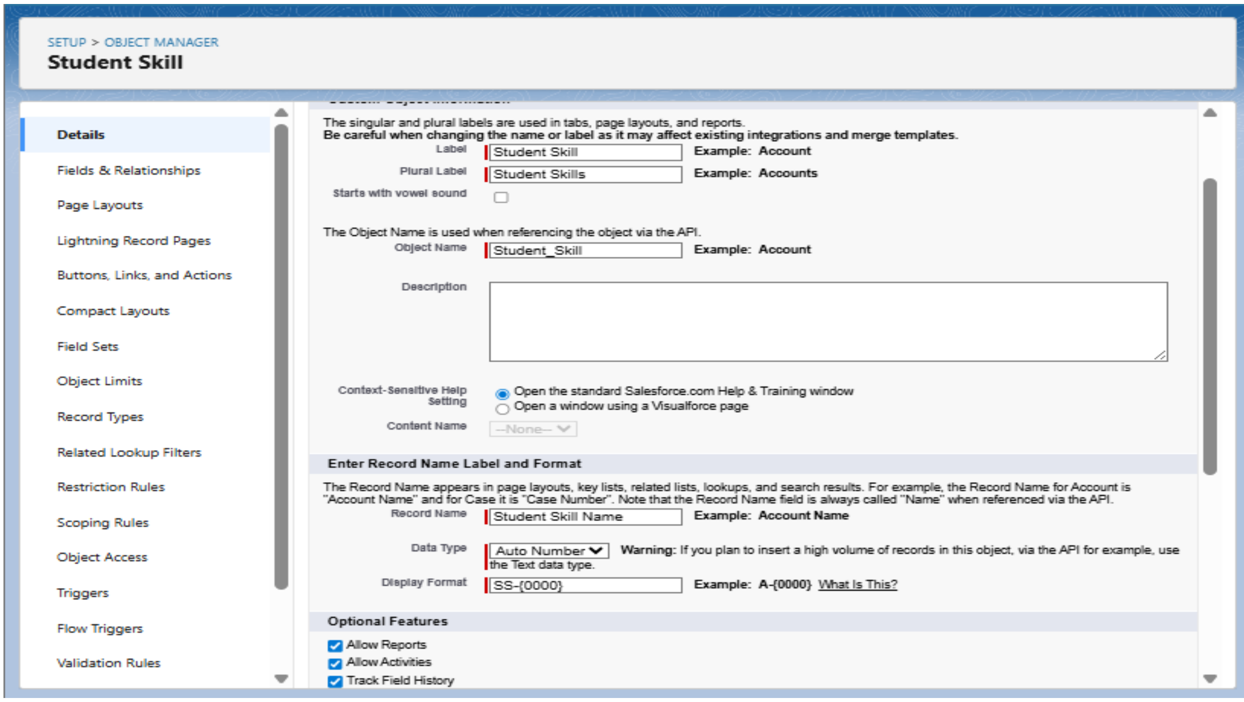
Phase 3 — Data modeling & relationships

3.1 Use standard objects vs custom

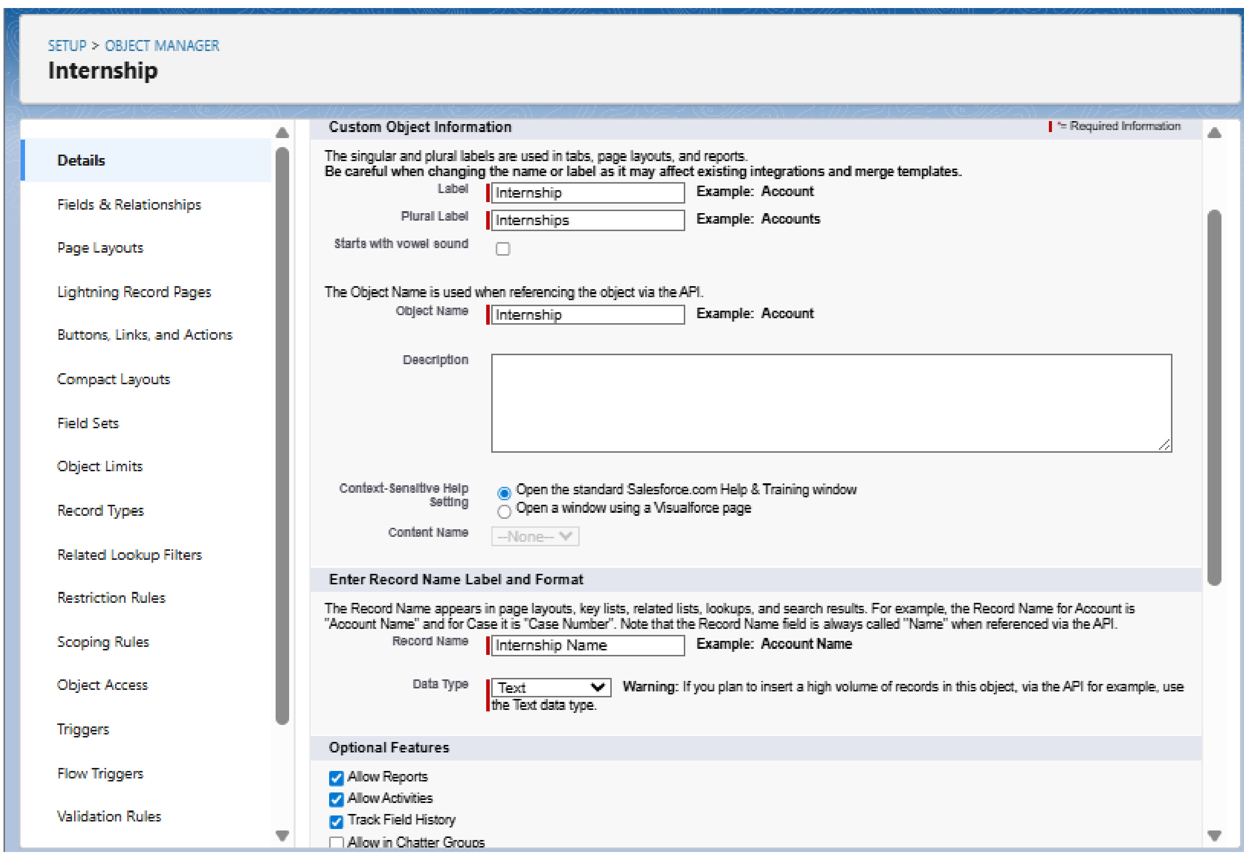
- **Account** → Use for College and Recruiter Company.
- **Contact** → Use for Student (recommended).
- Create custom objects:
  - Skill__c (label: Skill)



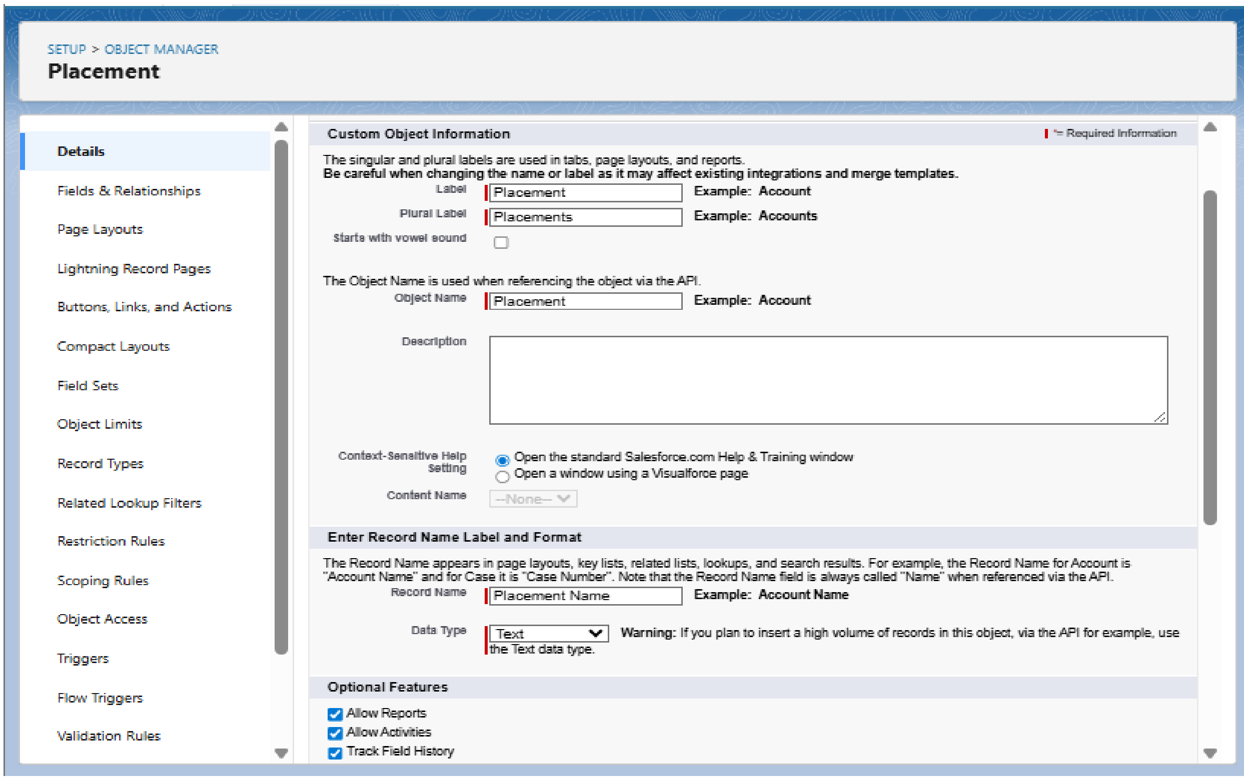
- Student_Skill__c (junction between Contact & Skill)



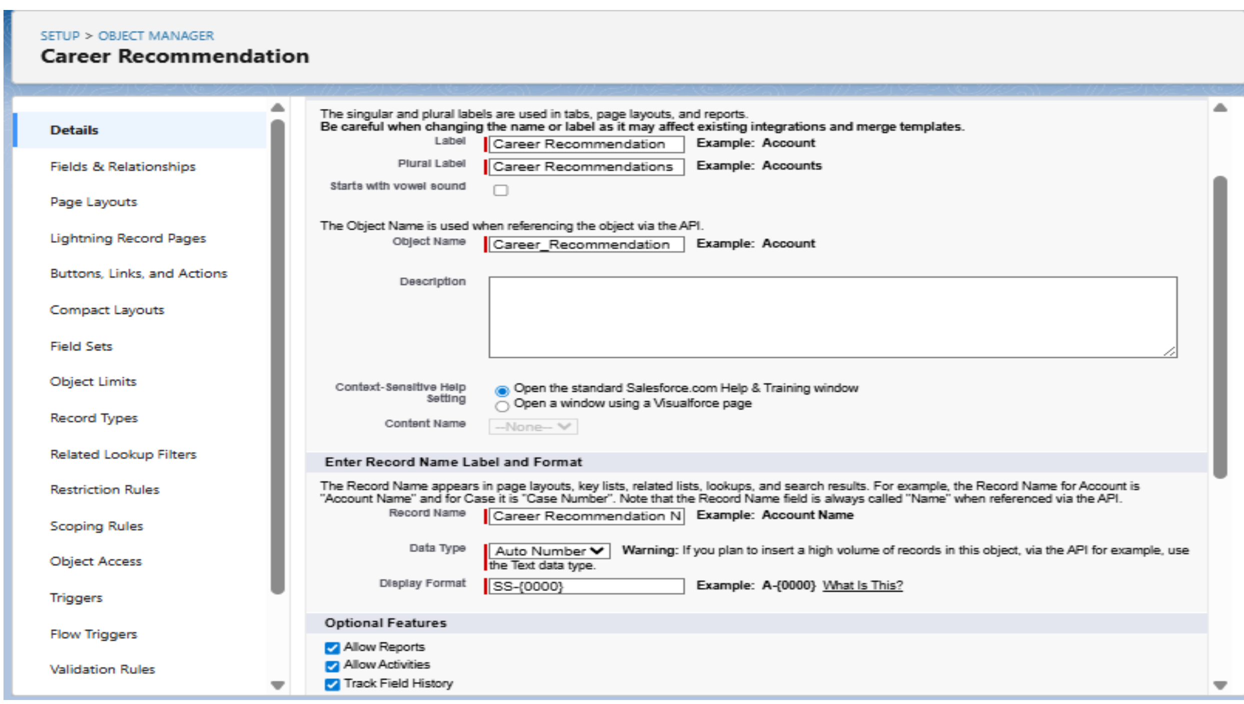
- Internship__c



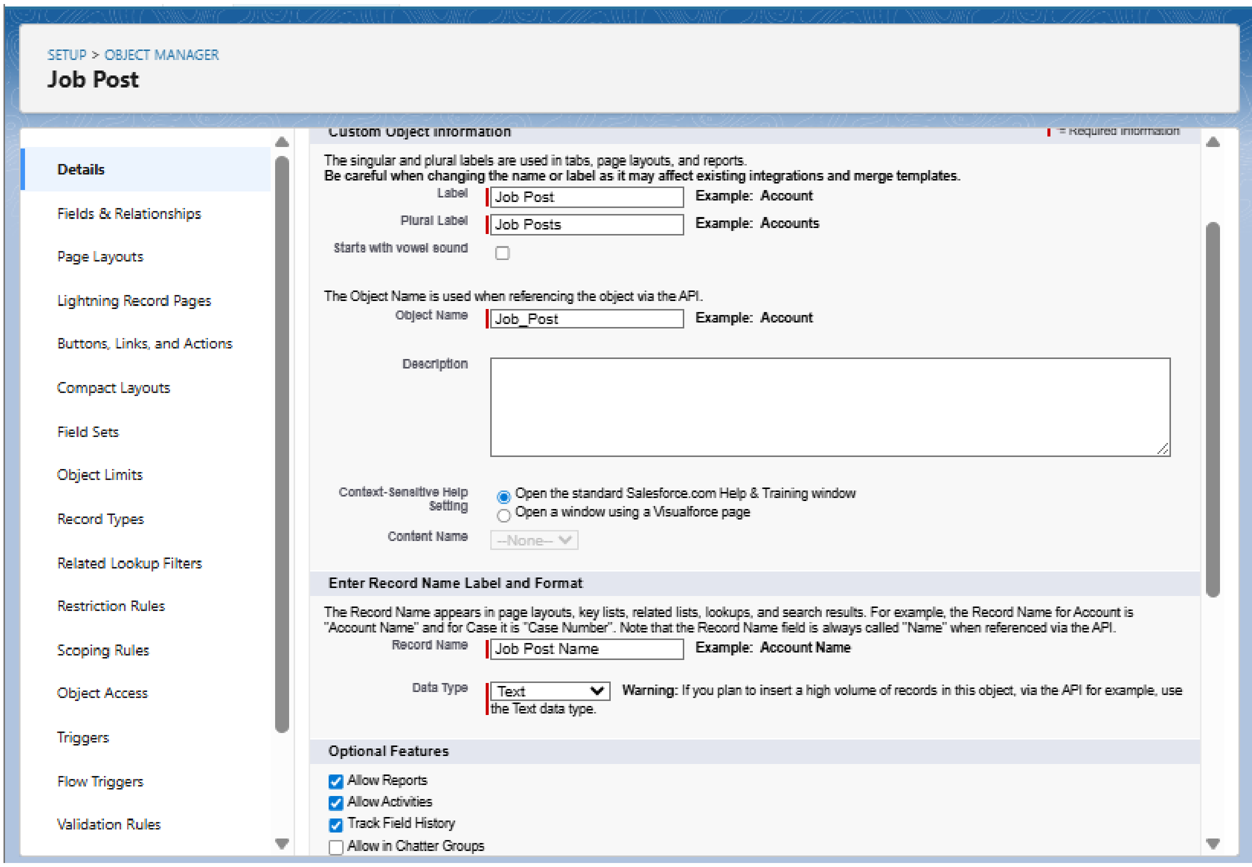
- Placement__c



- Career_Recommendation__c



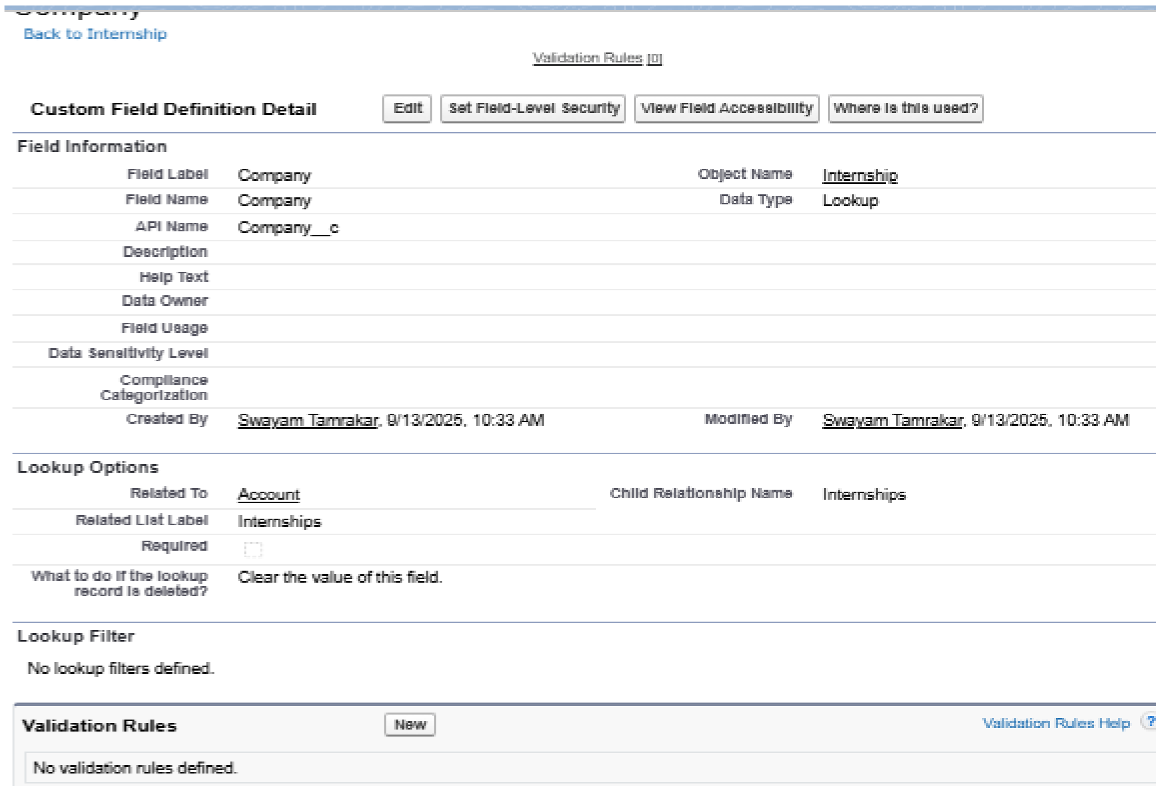
- (Optional) Job_Post__c for external job postings



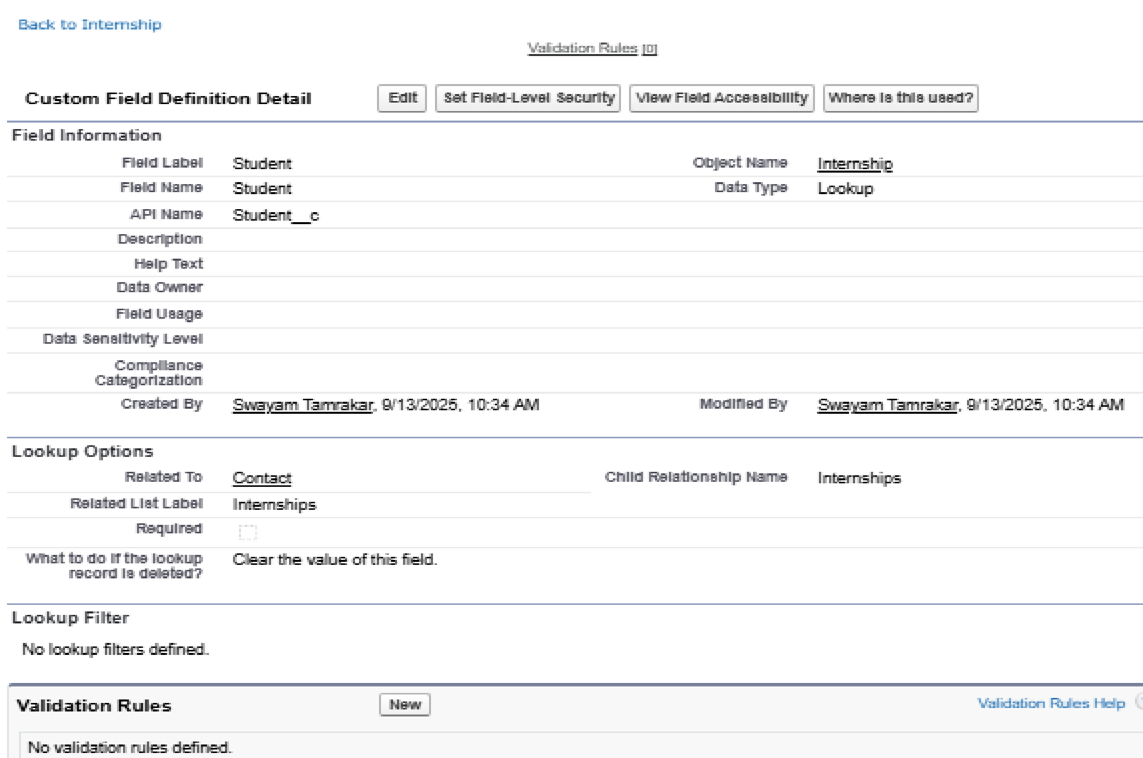
3.2 Create custom objects and fields

For each object:

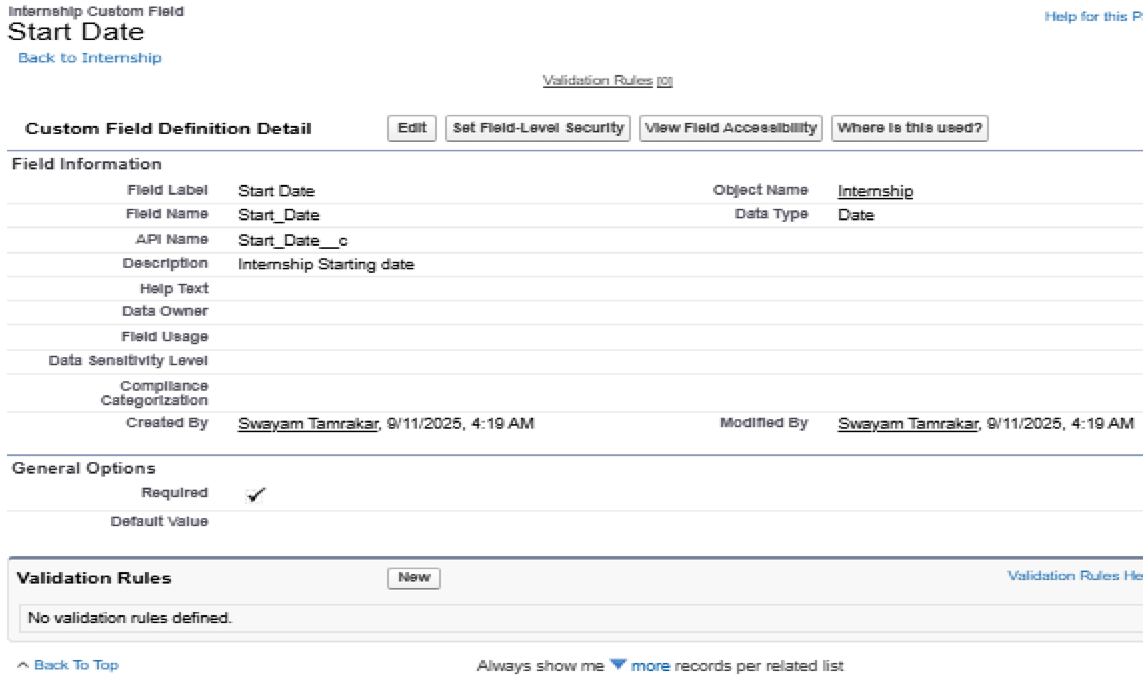
1. **Setup** → *Object Manager* → **Create** → **Custom Object** → fill:
   - Label: Internship → Plural: Internships → Object Name: Internship__c → Check "Allow Reports", "Allow Activities", "Track Field History" as needed → Save.
2. **Fields & Relationships** → **New**:
   - Example Internship__c fields:
     - Company__c — Lookup(Account) → Select *Data Type*: Lookup(Account).



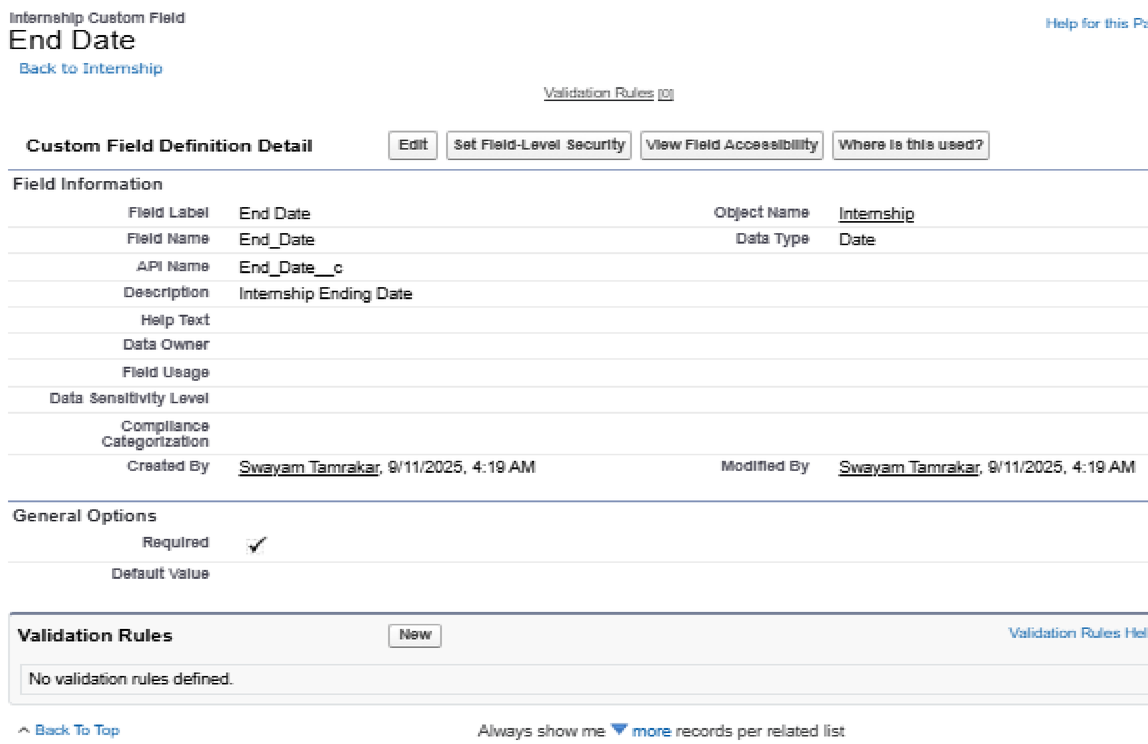
- Student__c — Lookup(Contact).



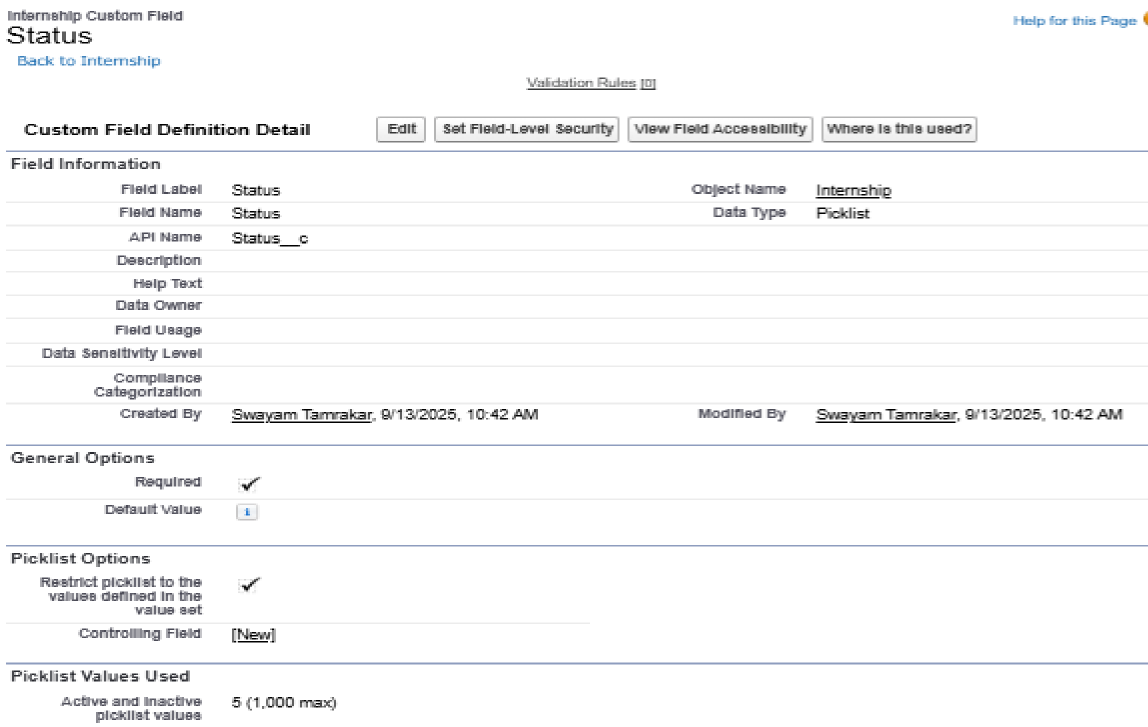
- Start_Date__c — Date



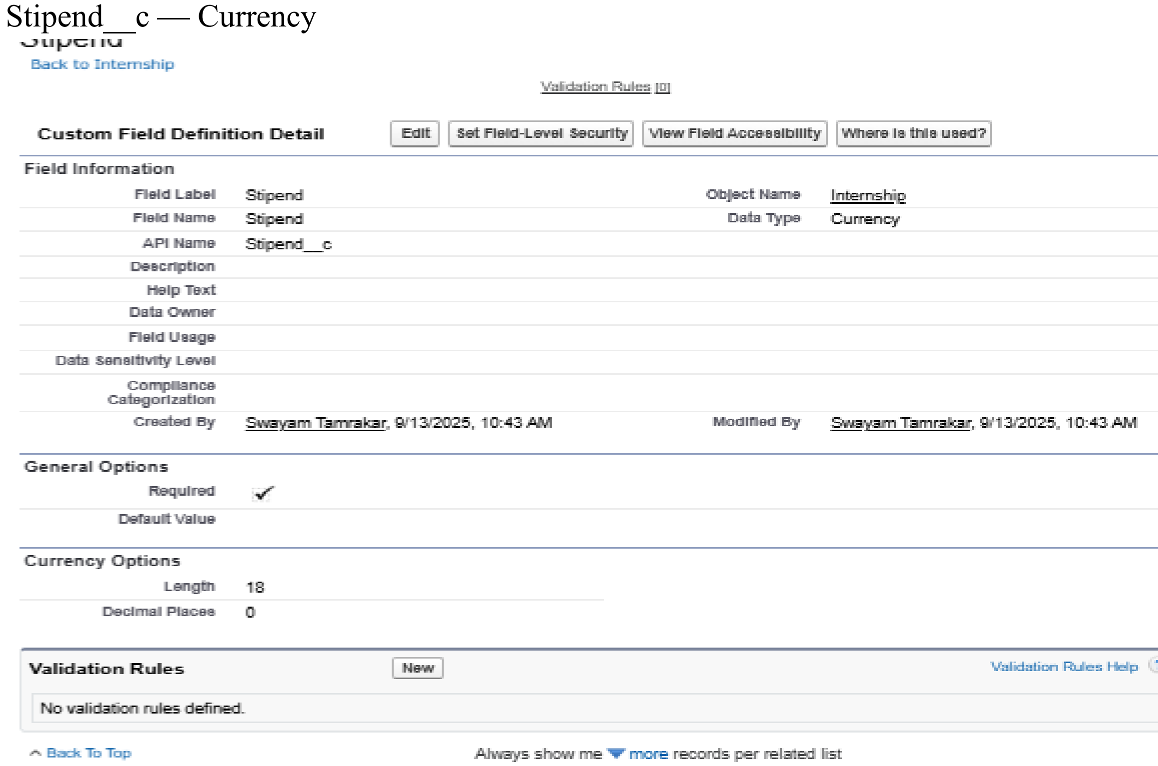
- End_Date__c — Date



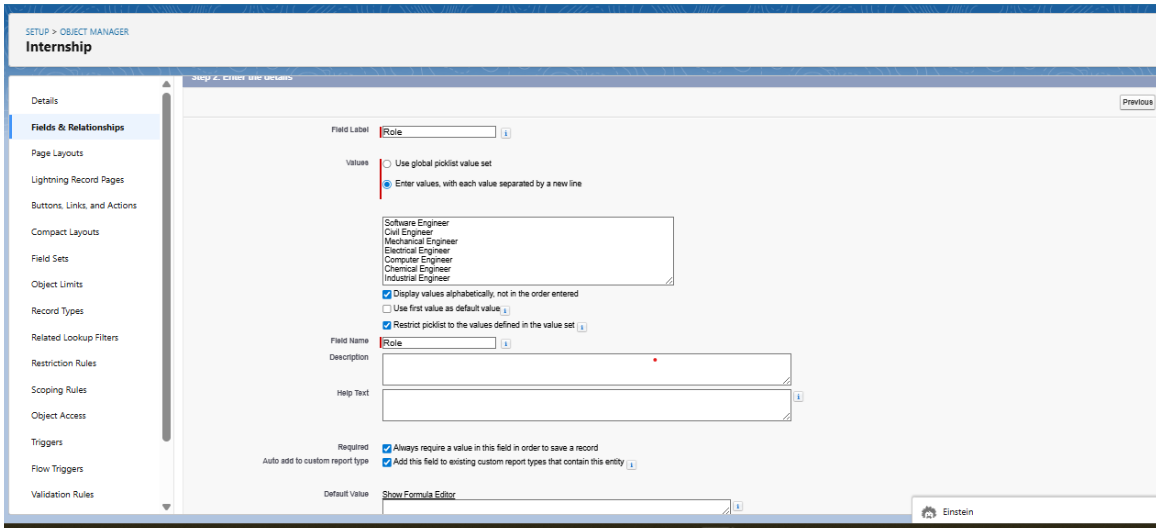
- Status__c — Picklist values: Applied, Interviewing, Selected, Completed, Rejected.



- Stipend__c — Currency

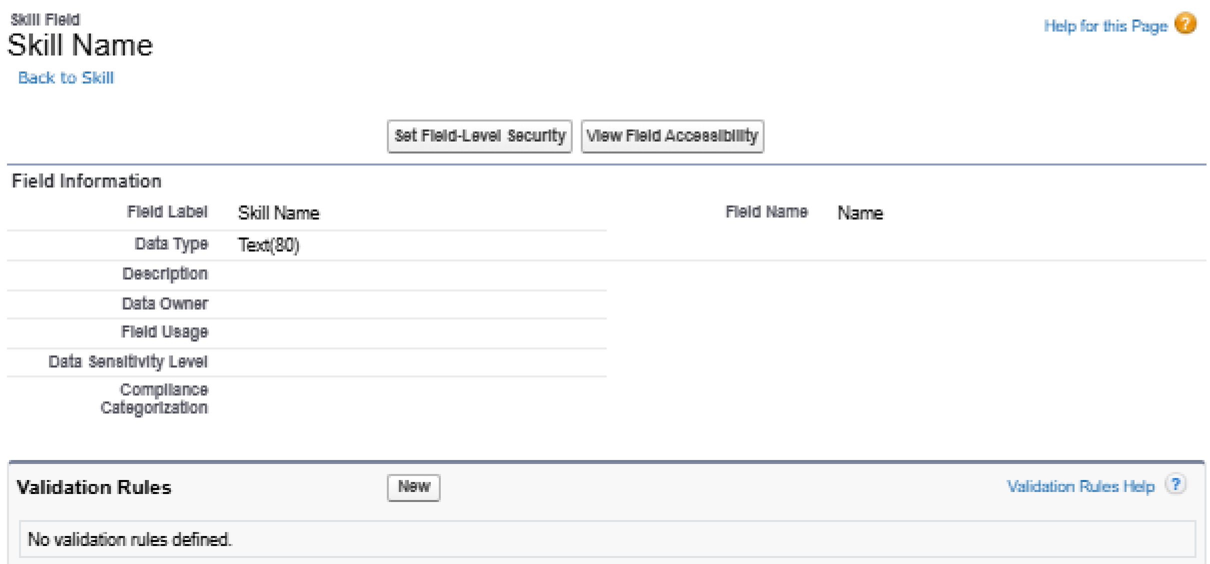


- **Role__c** — *Text* or *Picklist* (role title)

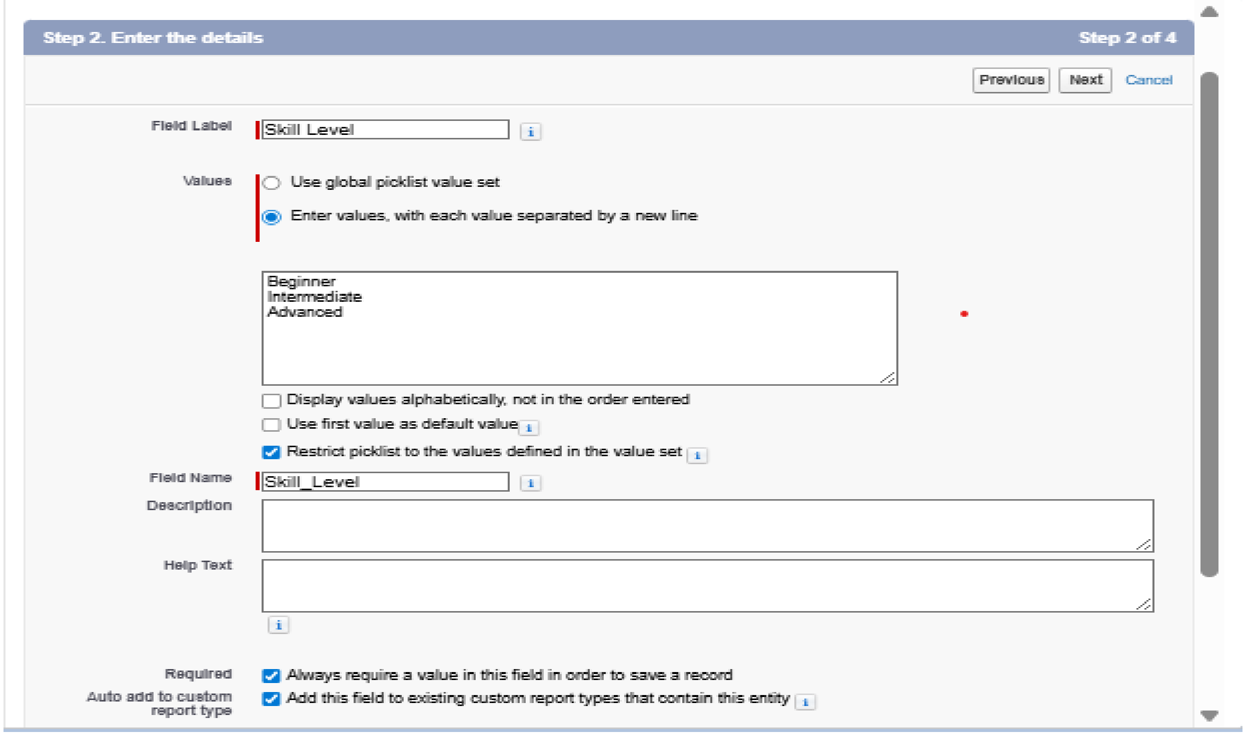


3. Create Skill__c:

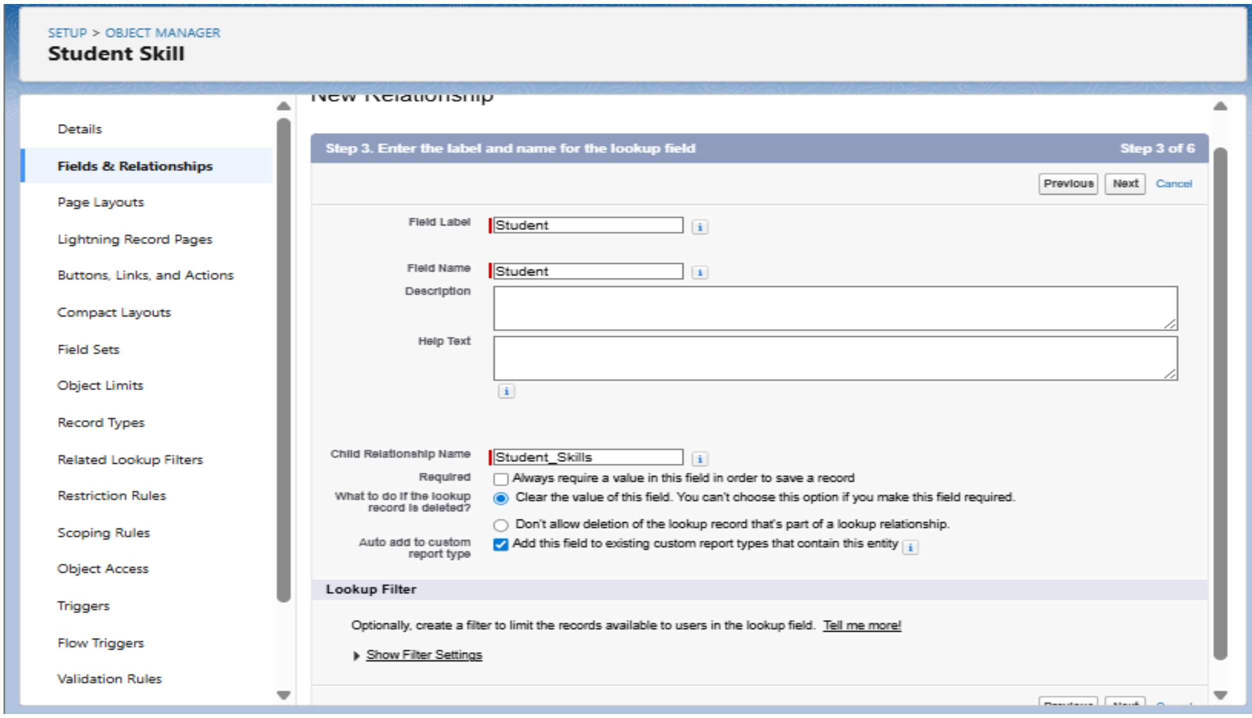
    - Field Name used for skill label (e.g., “Java”).



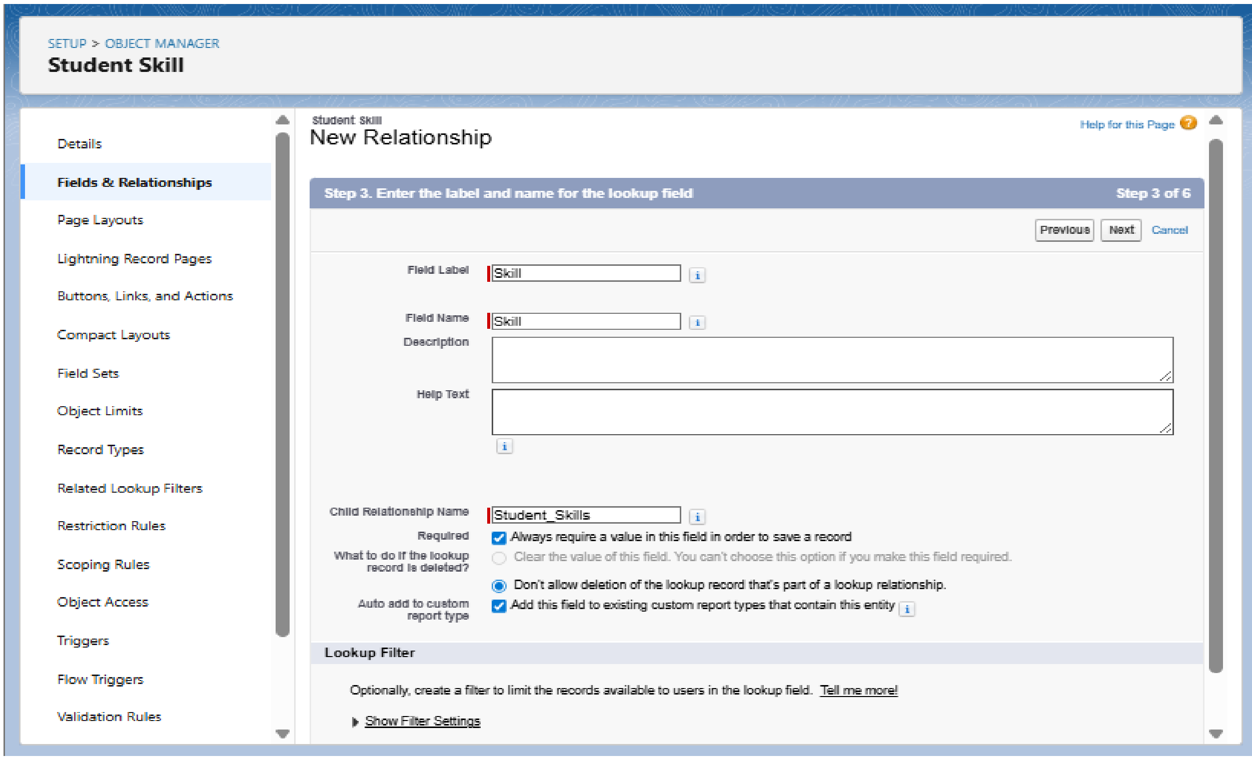
- Skill_Level__c — Picklist: Beginner, Intermediate, Advanced.



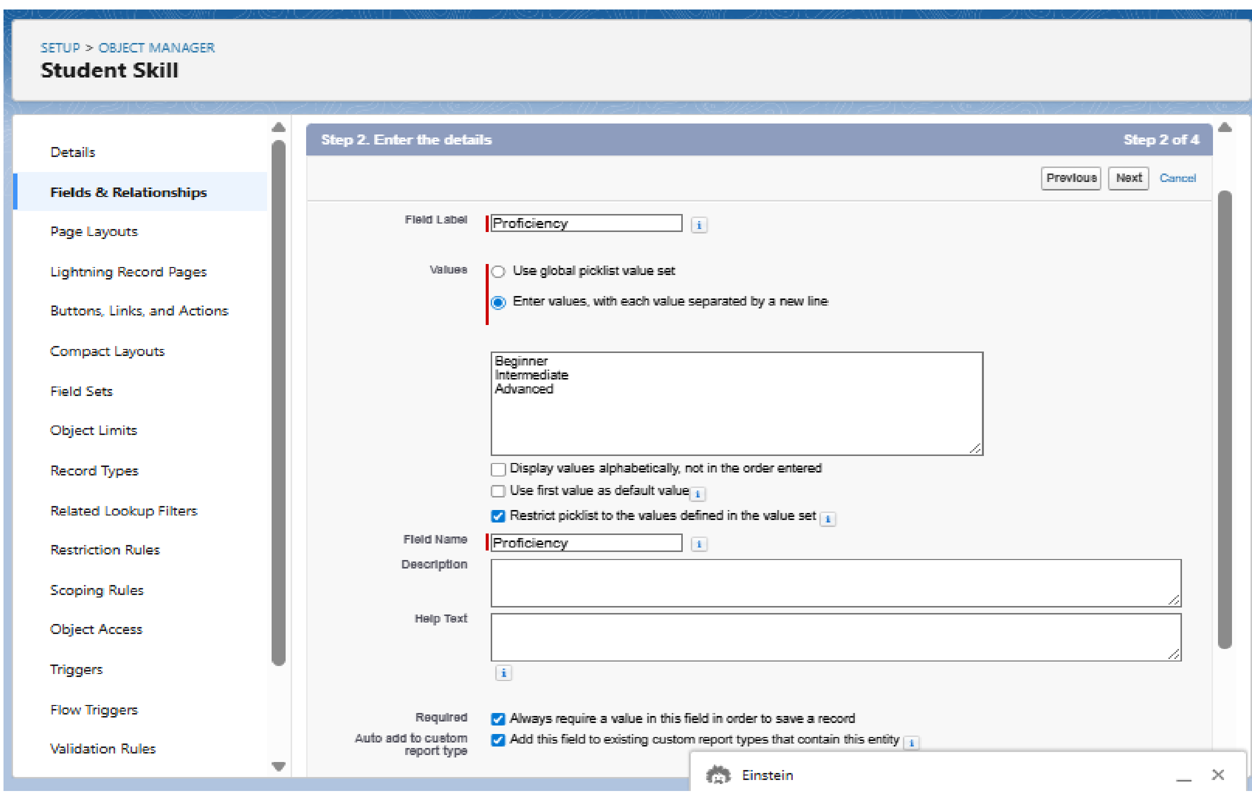
4. Create junction Student_Skill__c:
   - Student__c — Lookup(Contact)



- Skill__c — Lookup(Skill)

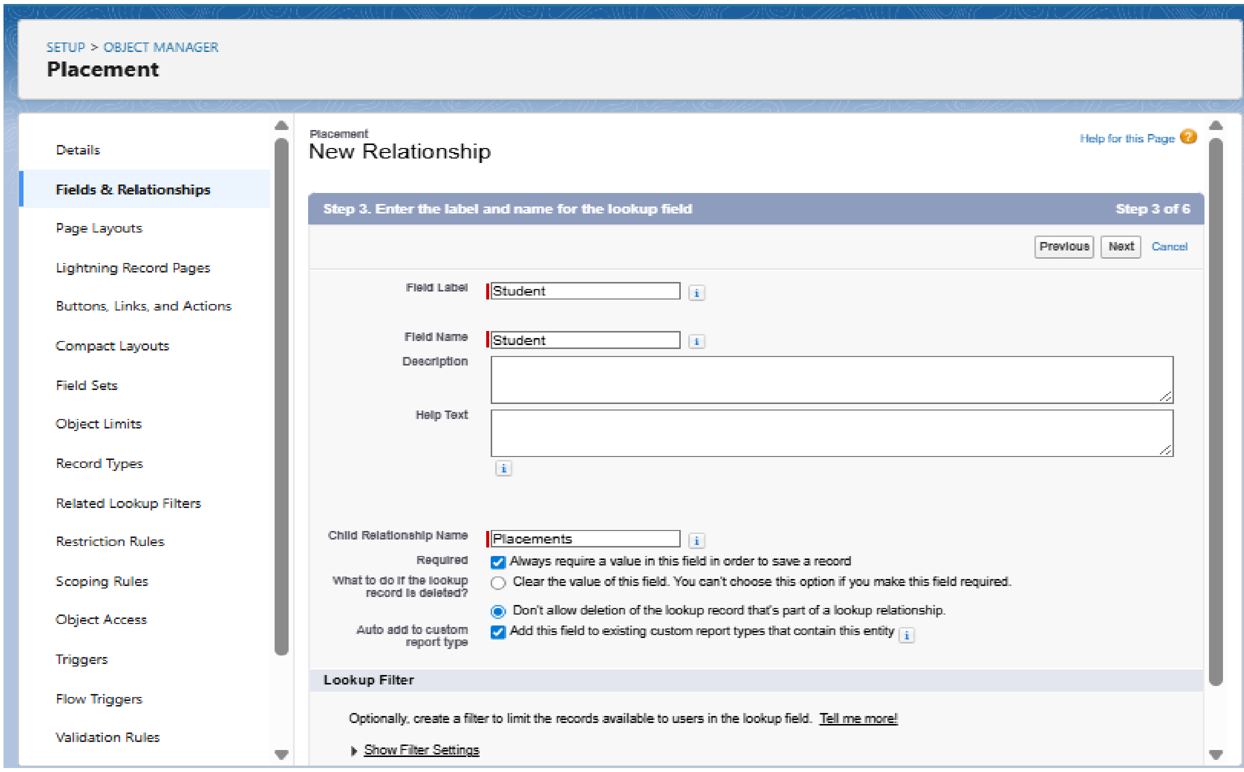


- Proficiency__c — Number or Picklist as needed

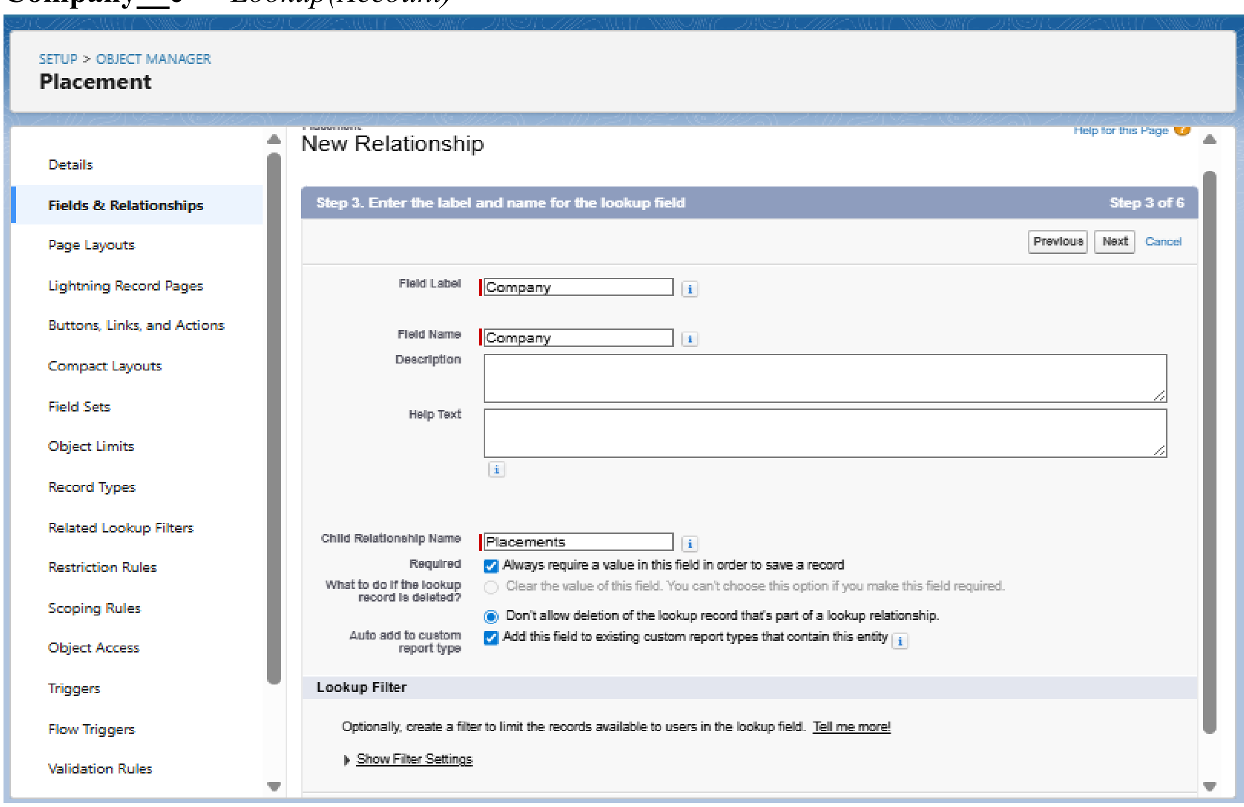


5. Placement__c

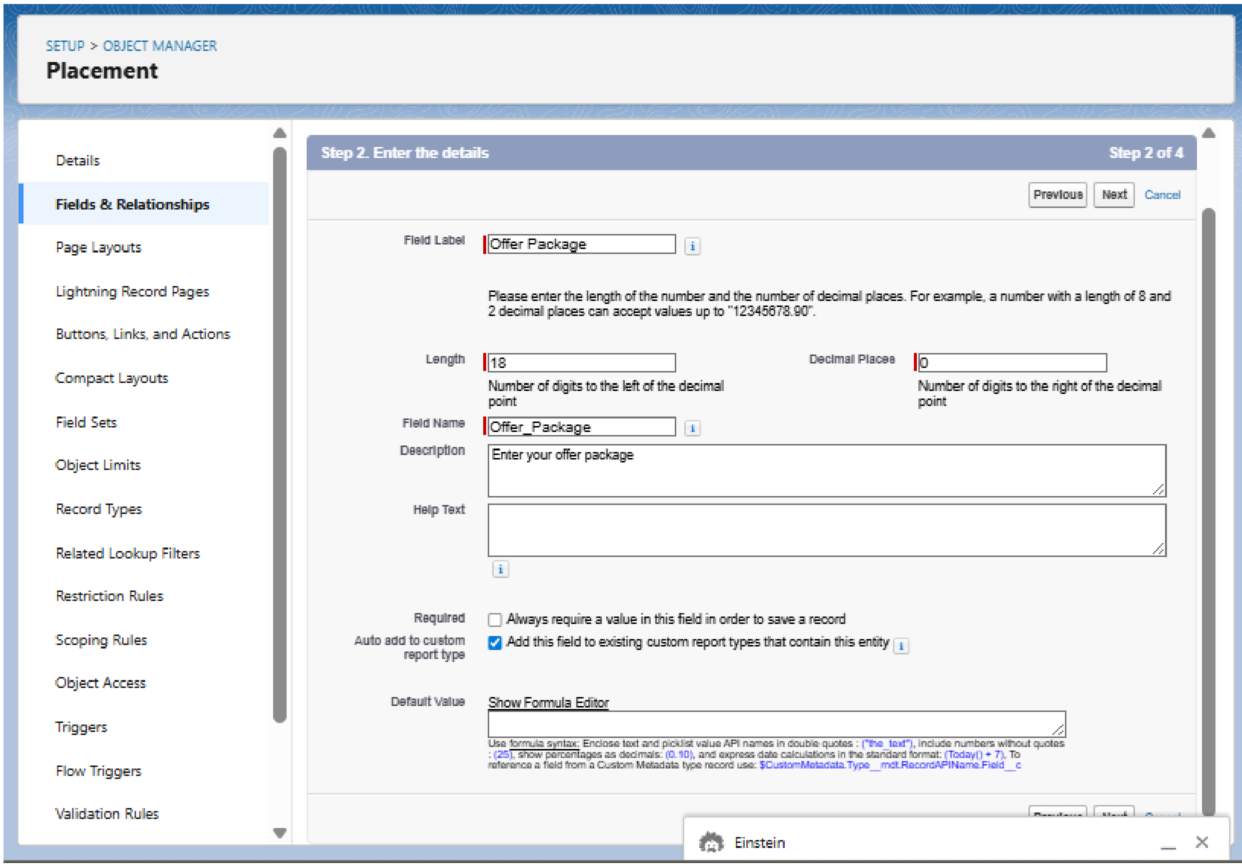
- **Student__c** — *Lookup(Contact)*



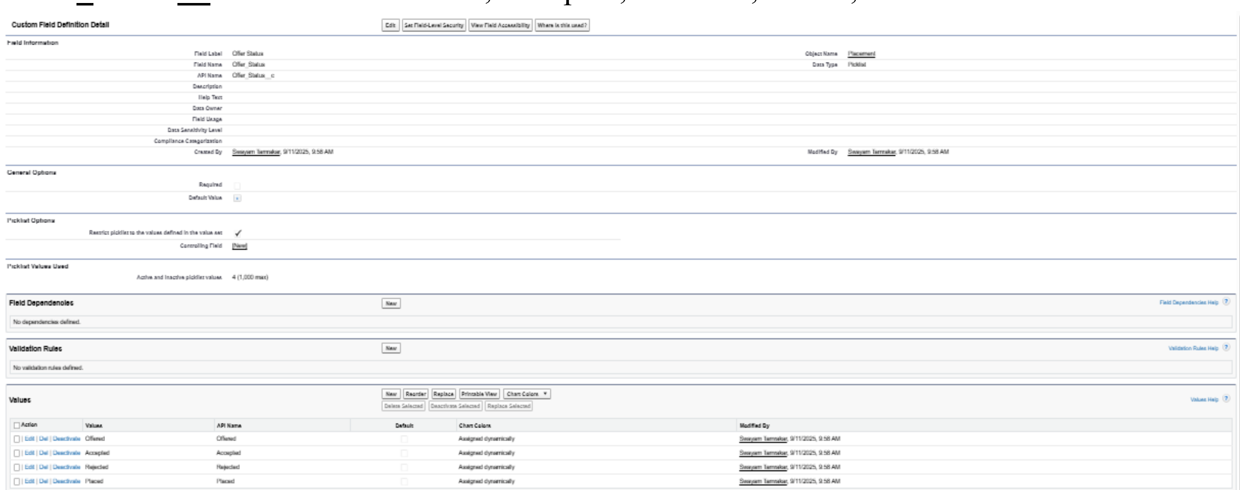
- **Company__c** — *Lookup(Account)*



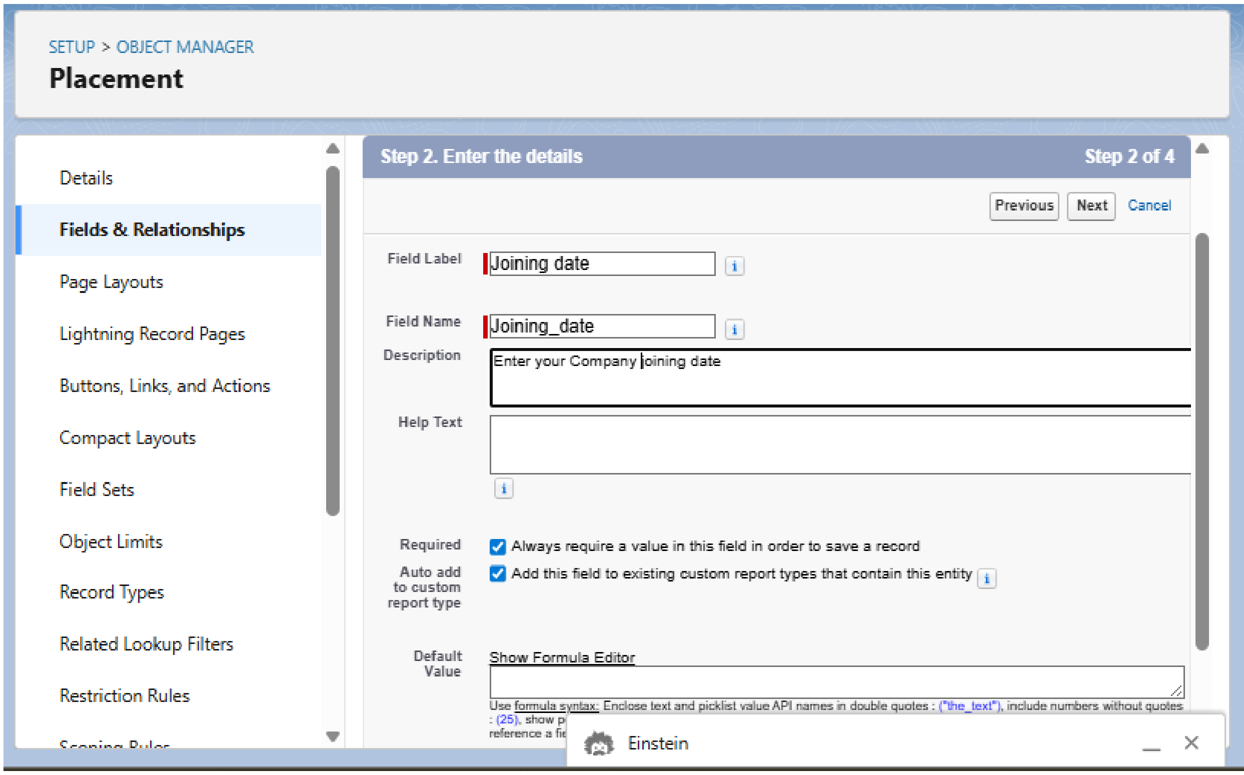
- **Offer_Package__c** — *Currency*



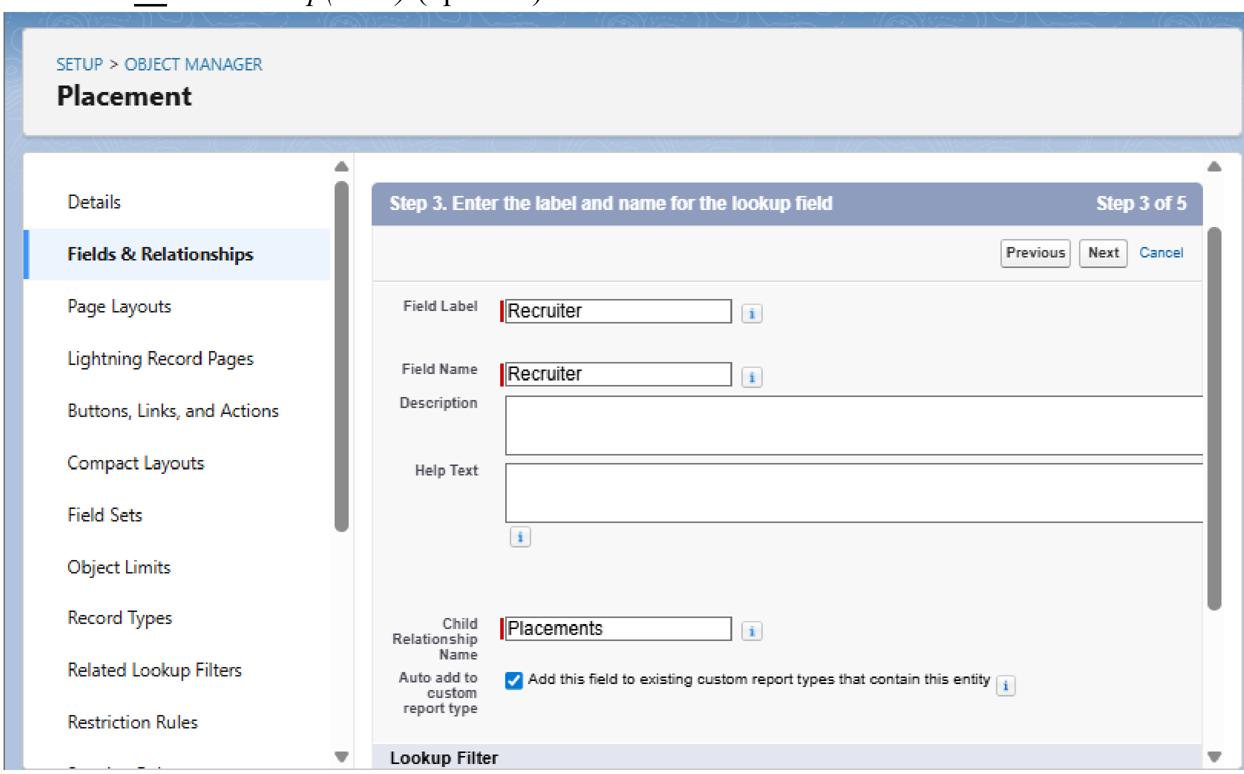
- **Offer_Status__c** — *Picklist*: Offered, Accepted, Declined, Joined, Withdrawn



- **Joining_Date__c** — *Date*

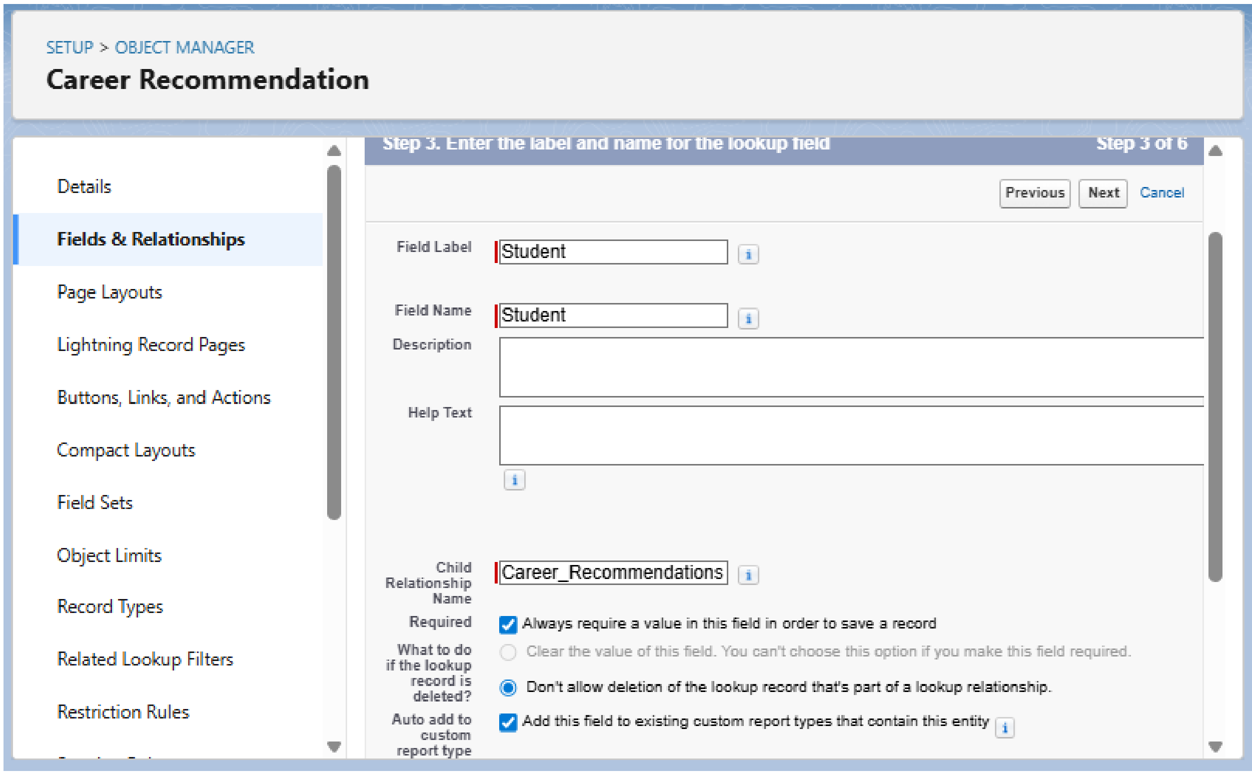


- **Recruiter__c** — *Lookup(User)* (optional)



6. **E. Career_Recommendation__c**

    - **Student__c** — *Lookup(Contact)*

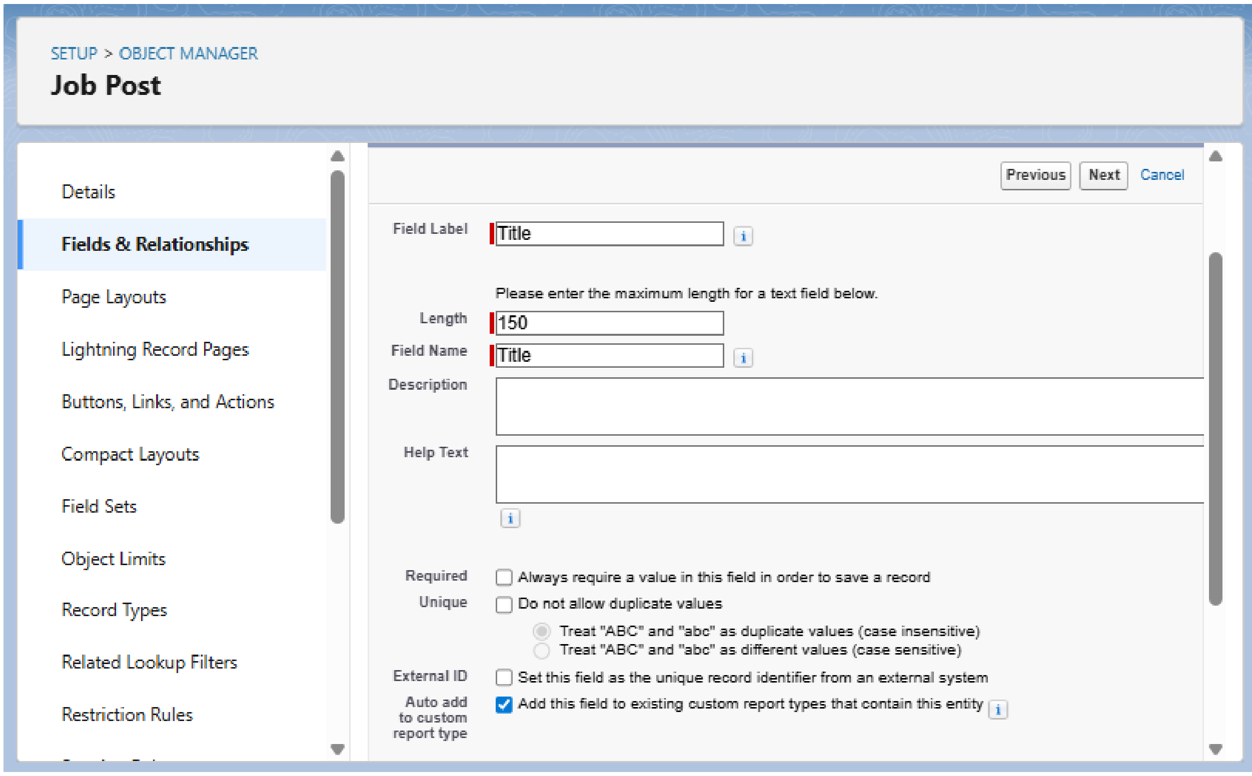


    - **Recommended_Path__c** — *Text* (e.g., “Data Science – Junior Analyst”)
    - **Confidence_Score__c** — *Number* (0 - 100)
    - **Generated_On__c** — *DateTime*
    - **Source__c** — *Picklist*: Einstein, RuleEngine, Manual

7. **F. Job_Post__c (optional)**

    - **Company__c** — Lookup(Account)

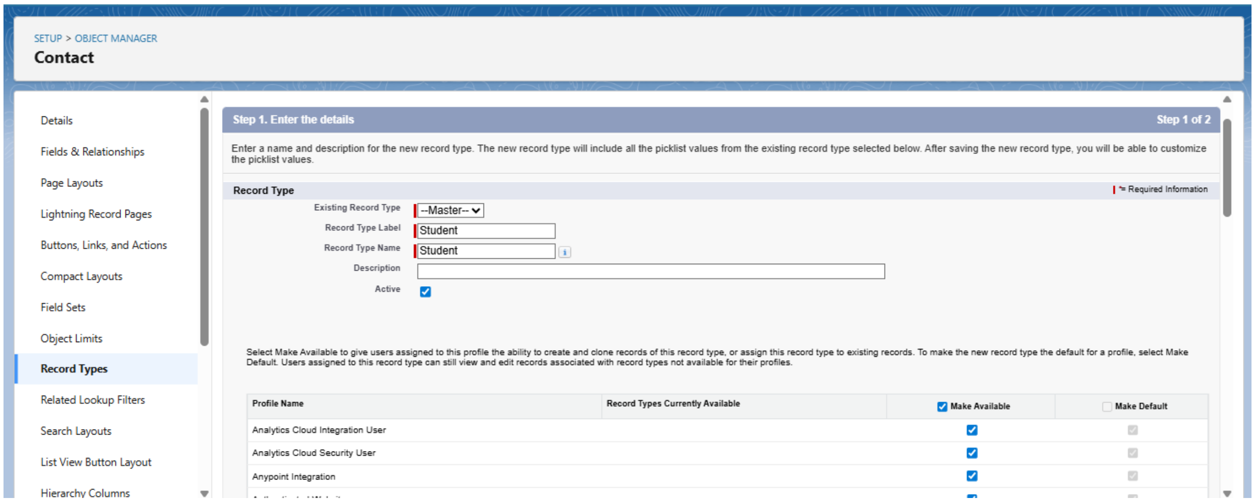
- **Title__c** — Text



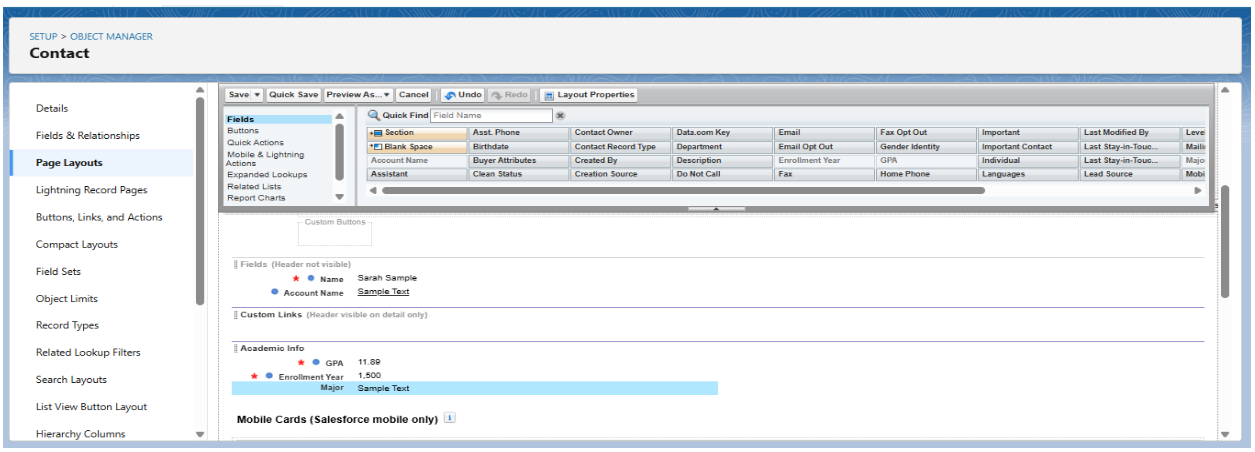
- **Description__c** — Long Text Area
- **Location__c** — Text
- **Skills_Required__c** — Long Text or multi-select picklist
- **Posted_On__c** — Date

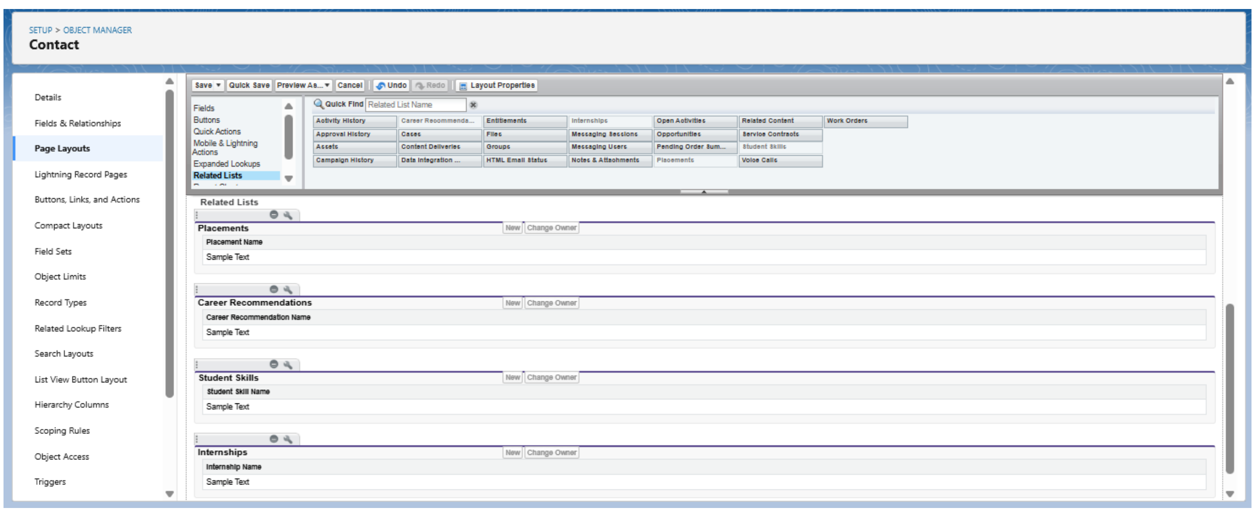
3.3 Record Types & Page Layouts

1. **Contact Object** → *Record Types* → **New** → Student. Add page layouts.



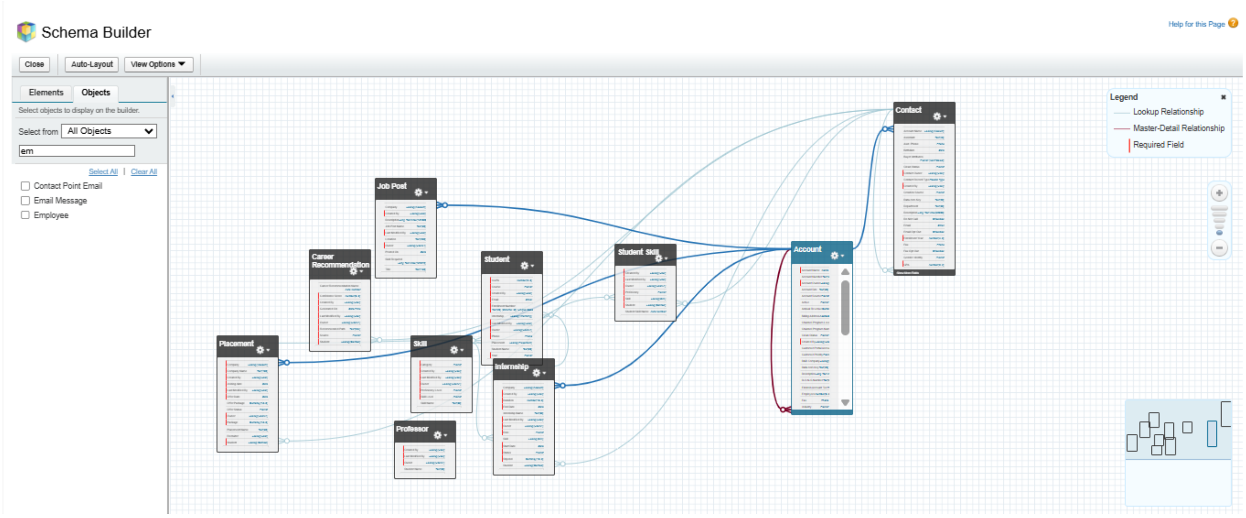
2. **Page Layouts** → include related lists: Internships, Student_Skills, Placements, Career_Recommendations. Drag fields like GPA__c, Career_Readiness__c.





3.4 Schema Builder

1. **Setup** → *Schema Builder* → drag objects to visualize relationships. Confirm junction object links.



3.5 Create sample data to test (validate your model)

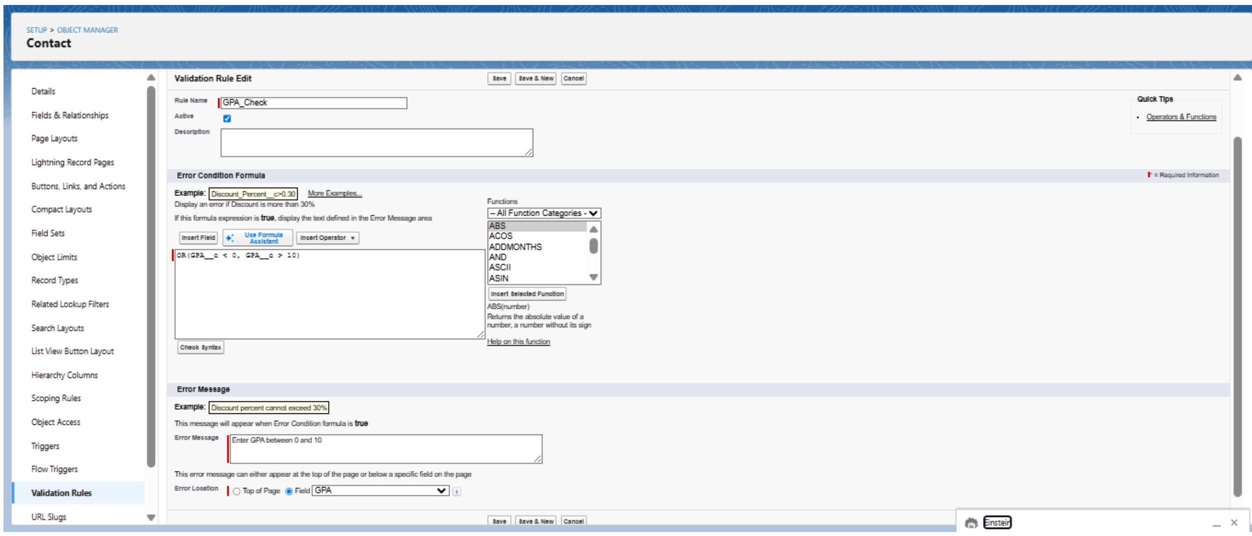
- **App Launcher → Accounts → New → create XYZ Engineering College.**
- **App Launcher → Contacts → New → create Swayam Tamrakar, select Record Type = Student, link Account = XYZ Engineering College, set GPA__c value.**
- **App Launcher → Skills tab → New → create Java, Python.**
- **App Launcher → Student Skills → New → select Student = Swayam, Skill = Java, set Proficiency.**
- **App Launcher → Internships → New → Student = Swayam, Company = TechCorp Pvt Ltd.**
- **Open Swayam's Contact page — confirm Student Skills, Internships appear in Related Lists.**

Phase 4: Process automation

- ➢ Use **Flow Builder** for modern automations

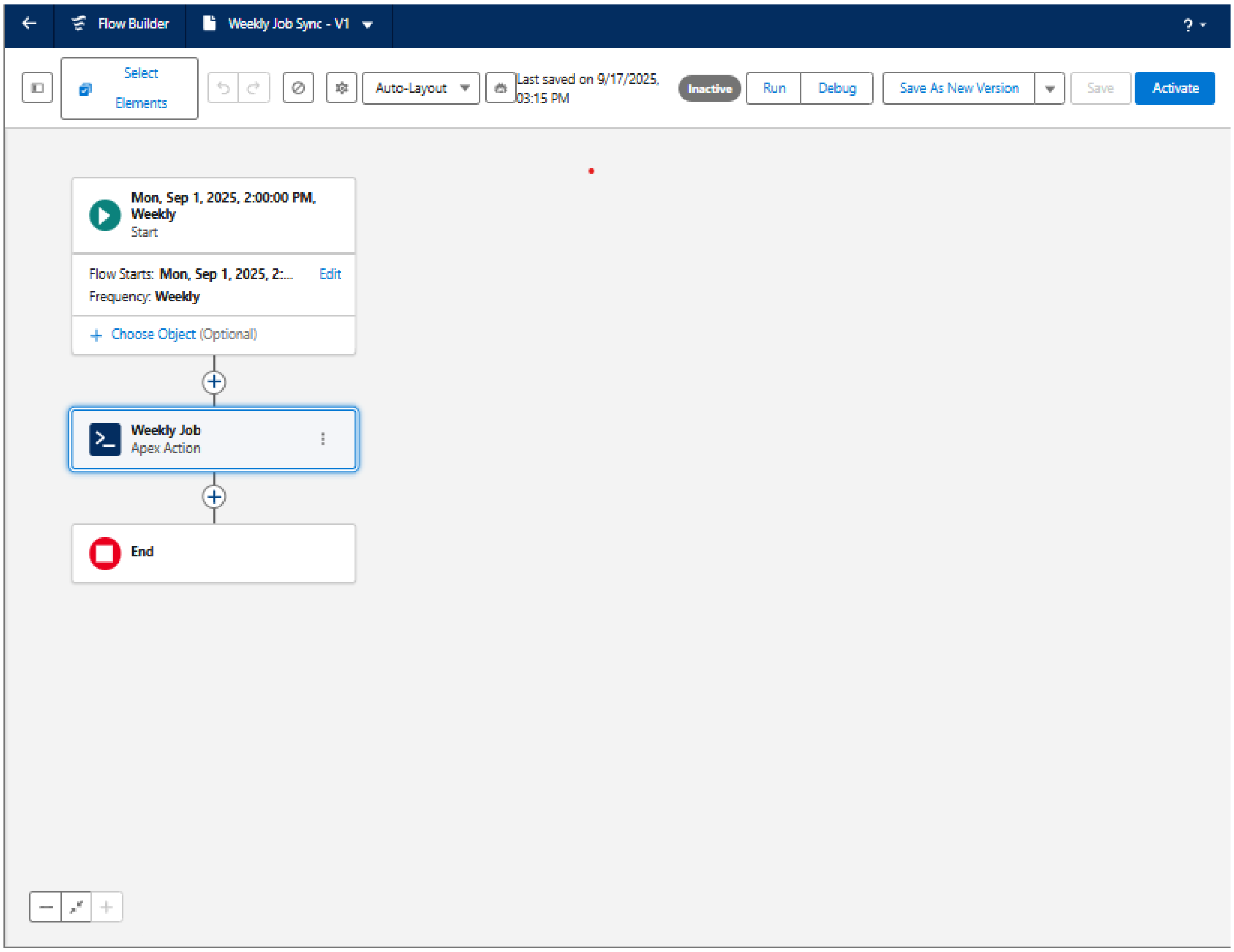
4.1 Validation Rules (examples)

1. **Contact** → *Validation Rules* → **New**
    - Rule: GPA__c >= 0 && GPA__c <= 10
    - Formula: OR(GPA__c < 0, GPA__c > 10)
    - Error message: "Enter GPA between 0 and 10."



4.2 Scheduled Flow — Weekly job aggregator

1. **New → Schedule-Triggered Flow** → run weekly.
2. Action: call an **Apex Action** (Batch job) or External Service to fetch jobs, create Job_Post__c records.



4.3 Approval Process (resume approval)

1. **Setup** → *Process Automation* → **Approval Processes** → New →**Jump Start Wizard**
2. Object: Contact.
3. **Name**:

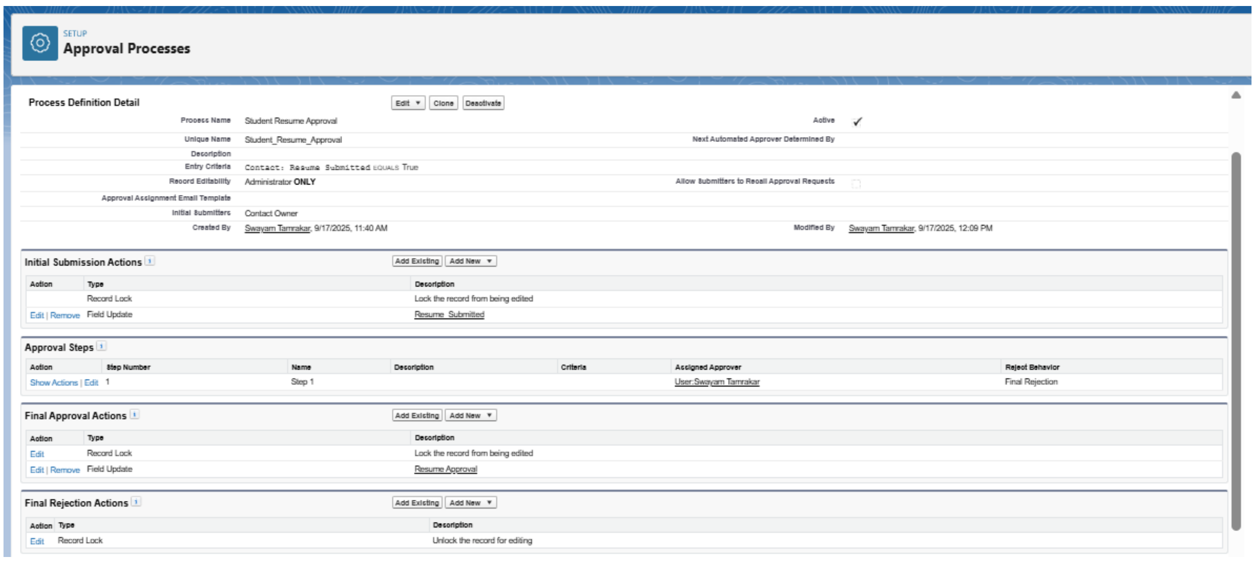
- Process Name: Student Resume Approval
- Unique Name: Student_Resume_Approval

4. **Entry Criteria**:

- Field: Resume_Submitted__c (Checkbox)

- Operator: Equals
- Value: TRUE

5. **Approver**: Select **Automatically assign to user** → Pick your **Placement Officer** user.
6. **Email Template**: You can skip or create a quick text-based template under Setup → Classic Email Templates.
7. **Initial Submission Action:** Field Update → set Resume_Status__c = Submitted
8. **Final Approval Action**: Field Update → Resume_Status__c = Approved.
9. **Final Rejection Action**: Field Update → Resume_Status__c = Rejected.
10. **Save** → Activate



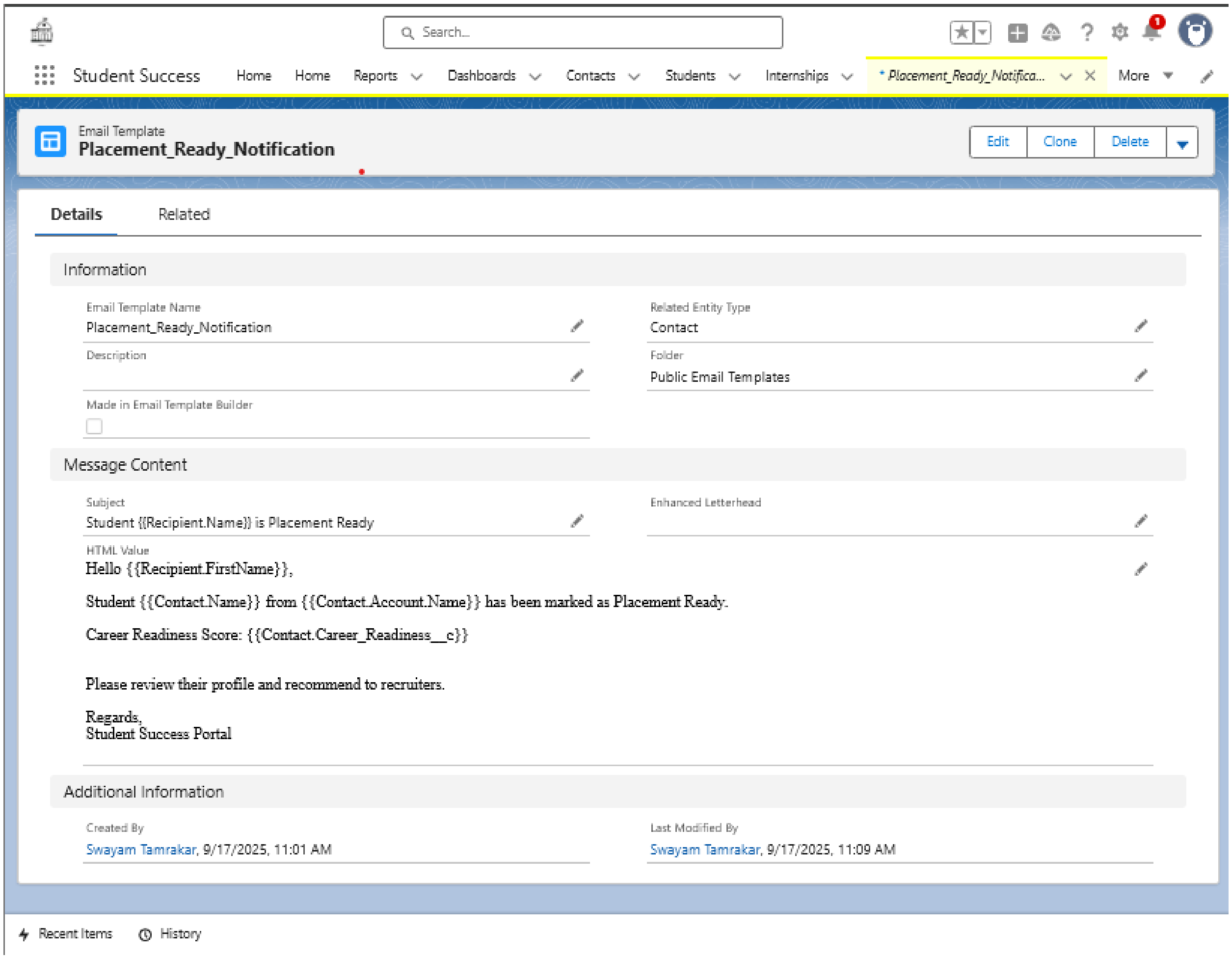
4.4 Email Templates & Notifications

- **Lightning templates.**

1. **In your org, click App Launcher → Search "Email Templates" → New Email Template**
2. **Fill details:**
    - **Name:** Placement_Ready_Notification.
    - **Related Entity Type:** Contact (since student is a Contact).
    - **Subject:** Student {{Recipient.Name}} is Placement Ready
    - **Folder:** Keep in Public Email Templates or create a folder Placement Notifications.
    - **In Email Content (HTML):**
    Hello {{Recipient.FirstName}},

Student {{Contact.Name}} from {{Contact.Account.Name}} has been marked as Placement Ready.
Career Readiness Score: {{Contact.Career_Readiness__c}}
Please review their profile and recommend to recruiters.
Regards,
Student Success Portal

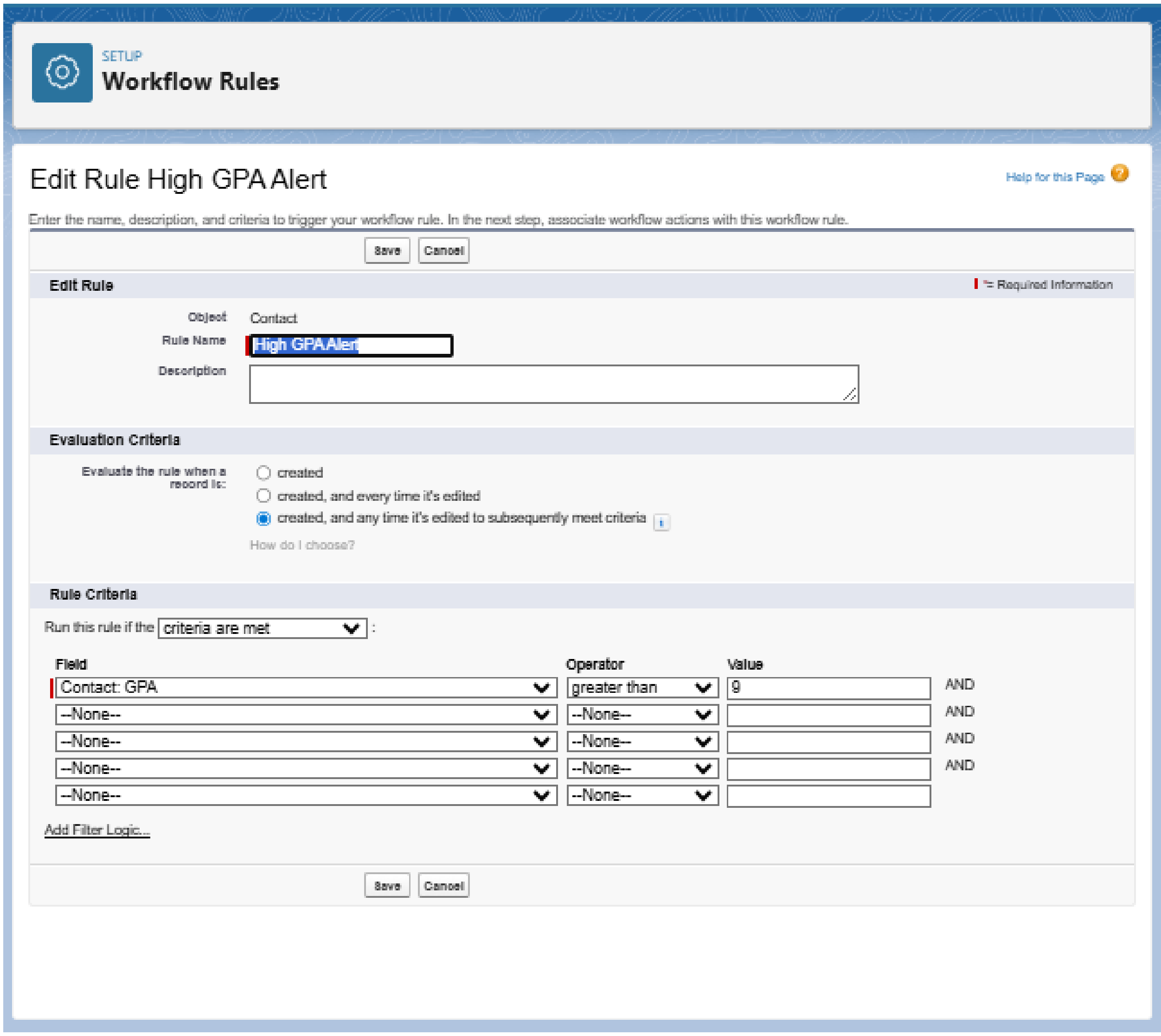
3. **Save**



4.5 Workflow Rules, Field Updates & Tasks

1. **Setup → Quick Find → Workflow Rules → New Rule.**
2. **Choose Object = Contact.**
3. **Rule Name = High GPA Alert.**
4. **Evaluation Criteria = "created, and every time it's edited to meet criteria".**
5. **Rule Criteria → Example: GPA__c >= 9.**

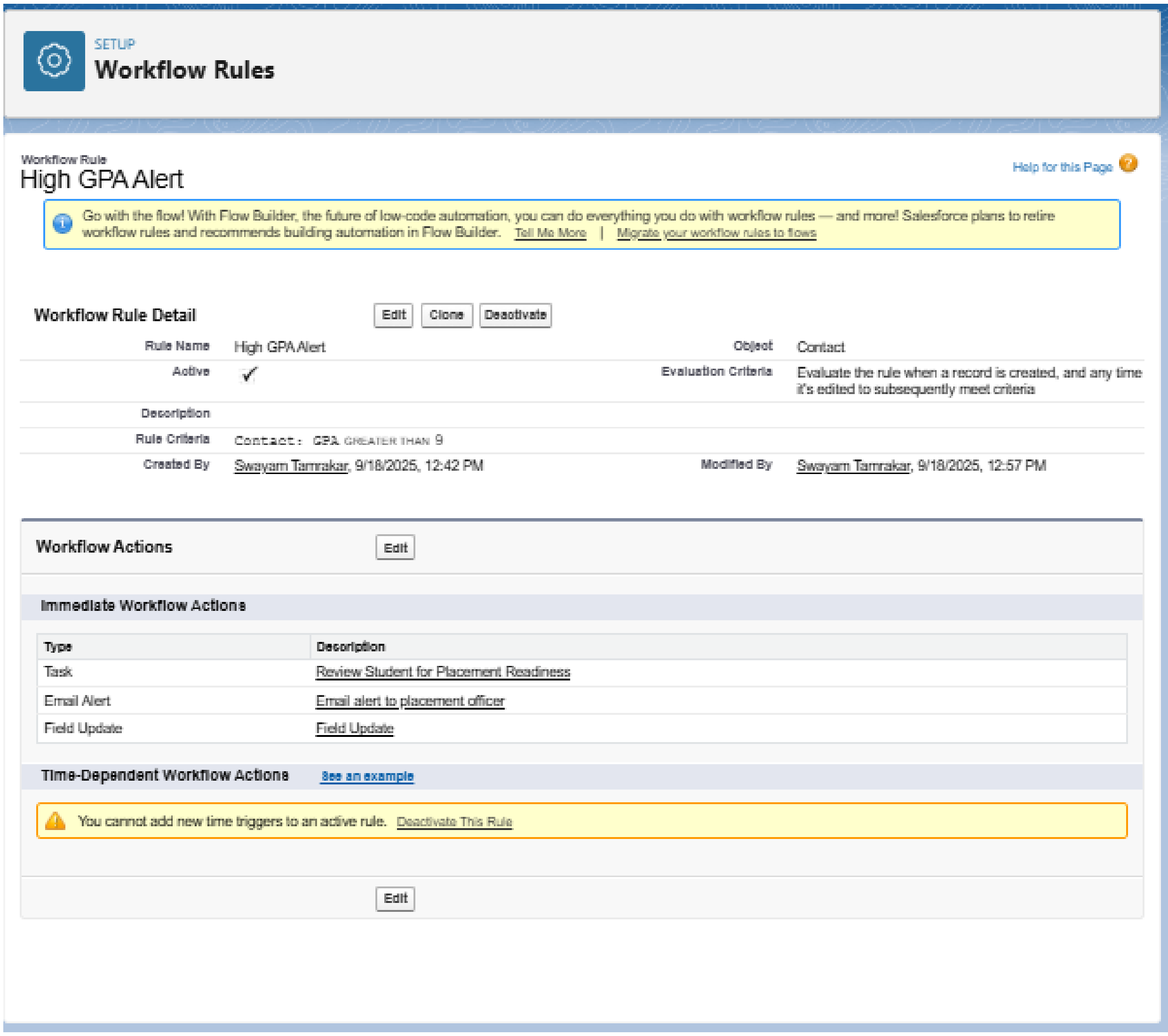
6. **Save**



7. **Add Workflow Action → choose:**

- **New Email Alert (send an email to Placement Officer).**
- **New Field Update (set Placement_Ready__c = TRUE).**
- **New Task (assign follow-up task to Placement Officer).**

8. **Activate the workflow rule.**

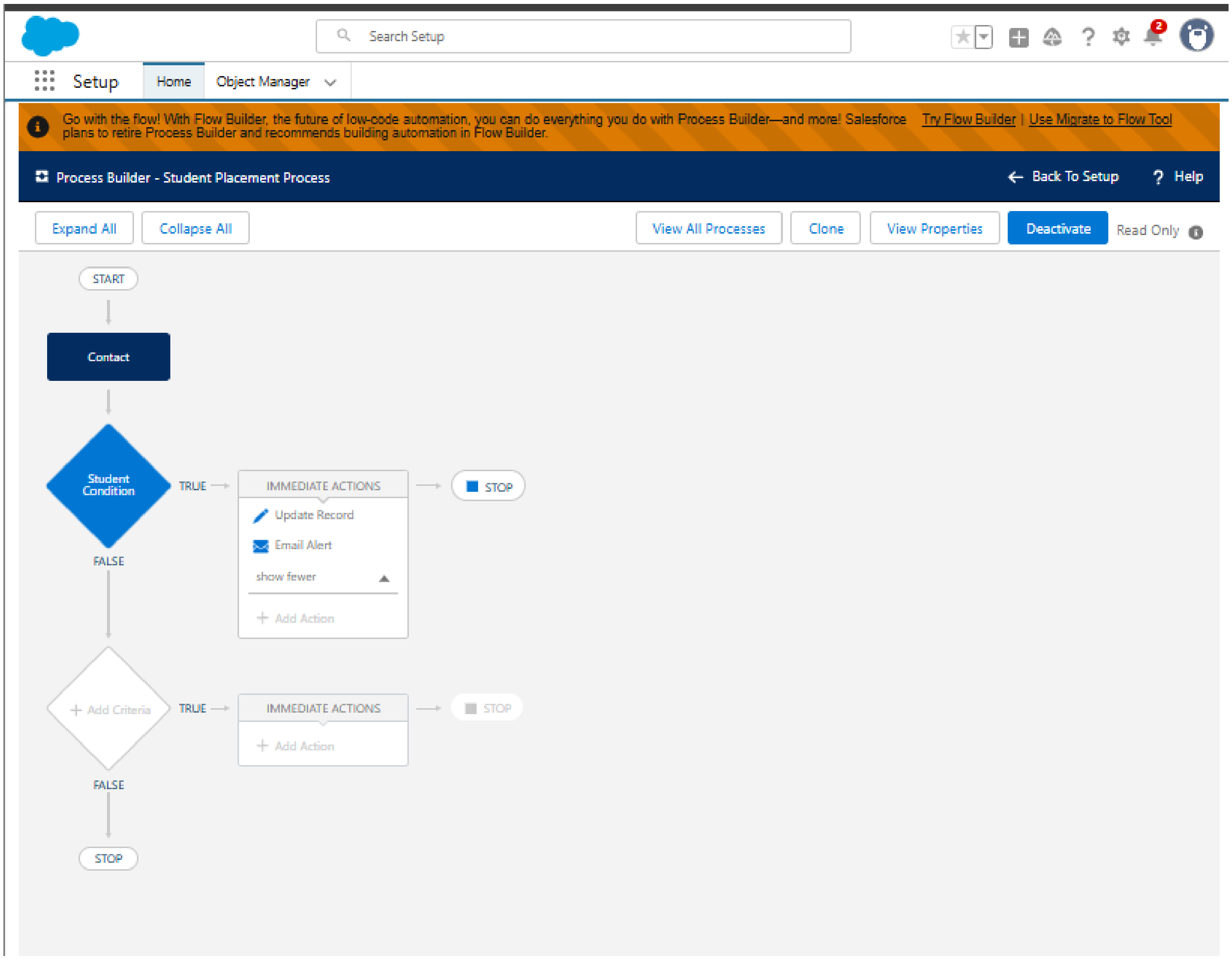


4.6 <u>Process Builder</u>

1. **Setup** → Quick Find → **Process Builder** → New.
2. Name: Student Placement Process.
3. Object: **Contact**. Start the process "when a record changes".
4. Criteria: GPA__c >= 8 AND Placement_Status__c = 'Not Ready'.
5. Actions:

- **Immediate Action** → Update Records → Set Placement_Status__c = Ready.
- **Immediate Action** → Send Alert → High GPA Alert

6. Activate process.



Phase 5: Apex development

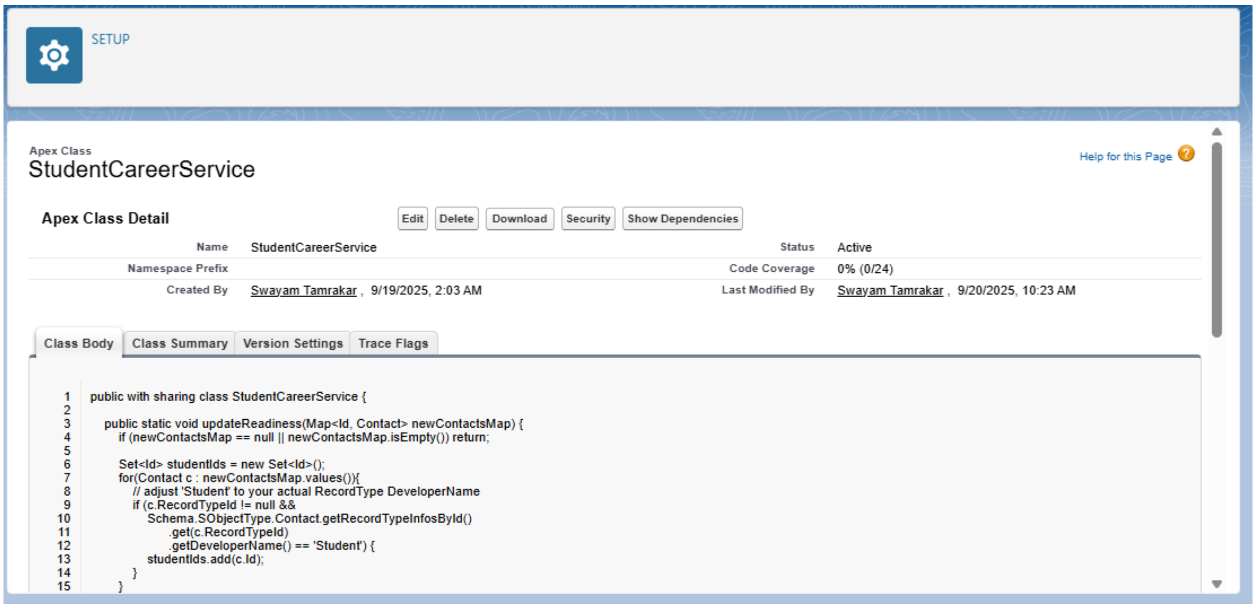
5.1 Trigger + Service + Test

1. **Create the Service Class (StudentCareerService)**

- **In Developer Console → File → New → Apex Class**.
- **Enter name:** StudentCareerService.
- **Code:**

```
public with sharing class StudentCareerService {
    public static void updateReadiness(Map<Id, Contact> newContactsMap) {
        if (newContactsMap == null || newContactsMap.isEmpty()) return;
        Set<Id> studentIds = new Set<Id>();
        for(Contact c : newContactsMap.values()){
            // adjust 'Student' to your actual RecordType DeveloperName
            if (c.RecordTypeId != null &&
                Schema.SObjectType.Contact.getRecordTypeInfosById()
                     .get(c.RecordTypeId)
                     .getDeveloperName() == 'Student') {
                studentIds.add(c.Id);
            }
        }
        if(studentIds.isEmpty()) return;
        // Aggregate skills
        Map<Id, Integer> skillCounts = new Map<Id, Integer>();
        for (AggregateResult ar : [
            SELECT Student__c studentId, COUNT(Id) cnt
            FROM Student_Skill__c
            WHERE Student__c IN :studentIds
            GROUP BY Student__c
        ]) {
            skillCounts.put((Id) ar.get('studentId'), (Integer) ar.get('cnt'));
        }
        // Calculate readiness
        List<Contact> updates = new List<Contact>();
        for(Contact s : [SELECT Id, GPA__c, Career_Readiness__c
                         FROM Contact
                         WHERE Id IN :studentIds]){
            Integer skills = skillCounts.containsKey(s.Id) ? skillCounts.get(s.Id) : 0;
            Decimal gpa = (s.GPA__c == null) ? 0 : s.GPA__c;
            Decimal readiness = Math.min(100, (gpa * 8) + (skills * 4));
```

```
            // Only update if value has actually changed
            if(s.Career_Readiness__c != readiness){
                updates.add(new Contact(Id = s.Id, Career_Readiness__c = readiness));
            }
        }
        if(!updates.isEmpty()) update updates;
    }
}
```

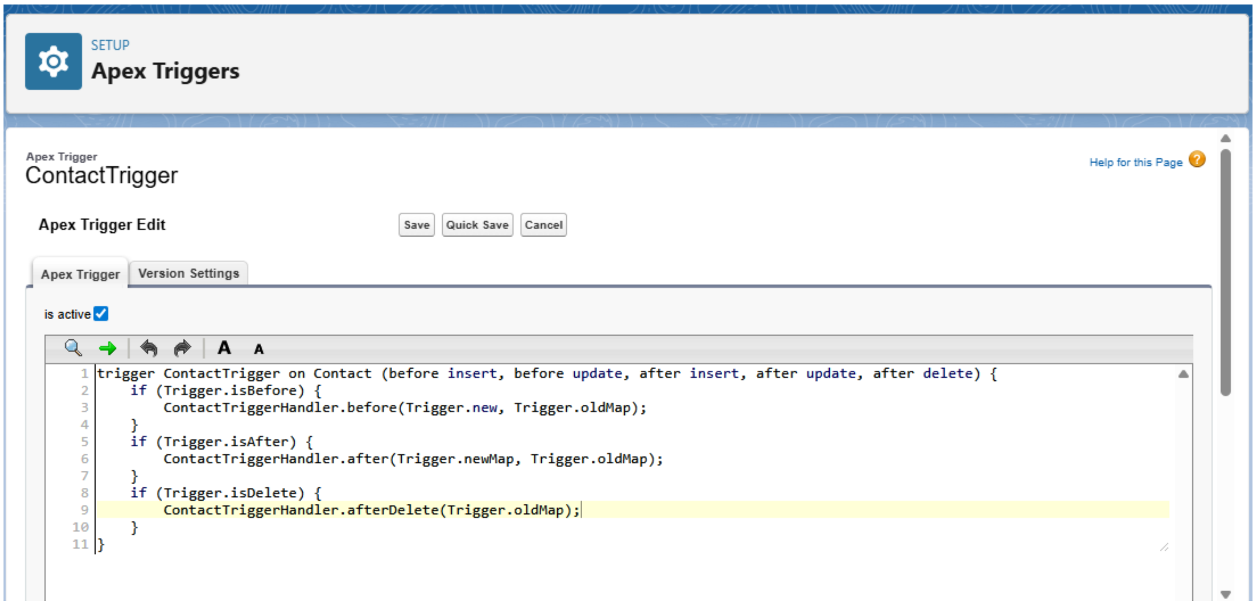


2. **Create the Trigger (ContactTrigger)**

- **Developer Console → File → New → Apex Trigger.**
- **Object:** Contact**. Name:** ContactTrigger.
- **Code:**

```
trigger ContactTrigger on Contact (before insert, before update, after insert, after update,
after delete) {
    if (Trigger.isBefore) {
        ContactTriggerHandler.before(Trigger.new, Trigger.oldMap);
    }
    if (Trigger.isAfter) {
        ContactTriggerHandler.after(Trigger.newMap, Trigger.oldMap);
    }
    if (Trigger.isDelete) {
        ContactTriggerHandler.afterDelete(Trigger.oldMap);
    }
```

```
}
```



3. Create the Test Class

- Developer Console → **File** → **New** → **Apex Class.**
- **Name:** StudentCareerServiceTest.
- **Code:**

```
@isTest
private class StudentCareerServiceTest {
    // 🔹 Test Setup - reusable data
    @testSetup
    static void setup(){
        // Create College (Account) for Student
        Account college = new Account(Name = 'Test College');
        insert college;

        // Create Company (Account) for Placement
        Account company = new Account(Name = 'TechCorp Pvt Ltd');
        insert company;

        // Get Student record type
        Id studentRtId = Schema.SObjectType.Contact.getRecordTypeInfosByName()
                          .get('Student').getRecordTypeId();

        // Dynamically get a valid Placement_Status__c picklist value (Contact)
        Schema.DescribeFieldResult placementStatusField = Contact.Placement_Status__c.
getDescribe();
        String validPlacementStatus = placementStatusField.getPicklistValues()[0].getValue
```

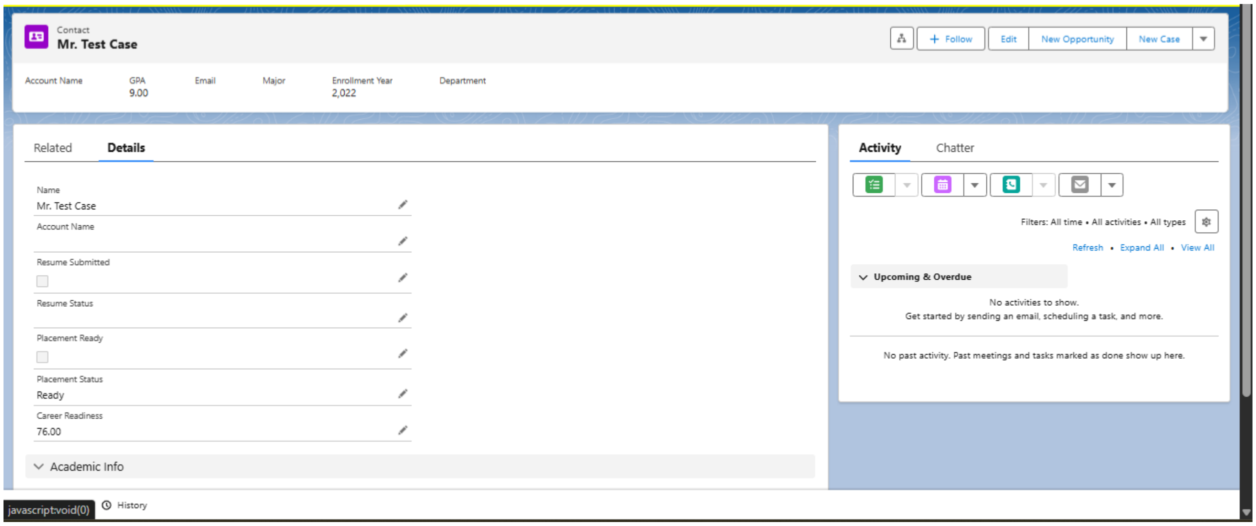
```
();
        // Create Student
        Contact stud = new Contact(
            LastName = 'TestStudent',
            FirstName = 'Unit',
            AccountId = college.Id,
            GPA__c = 7.5,
            RecordTypeId = studentRtId,
            Enrollment_Year__c = 2025,
            Placement_Status__c = validPlacementStatus
        );
        insert stud;
        // Dynamically get valid Category__c and Proficiency_Level__c for Skill
        Schema.DescribeFieldResult categoryField = Skill__c.Category__c.getDescribe();
        String validCategory = categoryField.getPicklistValues()[0].getValue();

        Schema.DescribeFieldResult proficiencyLevelField = Skill__c.Proficiency_Level__
c.getDescribe();
        String validProficiencyLevel = proficiencyLevelField.getPicklistValues()[0].getVal
ue();
        // Create Skill
        Skill__c skill = new Skill__c(
            Name = 'Apex Programming',
            Category__c = validCategory,
            Proficiency_Level__c = validProficiencyLevel,
            Skill_Level__c = 'Beginner'
        );
        insert skill;
        // Dynamically get a valid Proficiency__c value for Student_Skill__c
        Schema.DescribeFieldResult proficiencyField = Student_Skill__c.Proficiency__c.g
etDescribe();
        String validProficiency = proficiencyField.getPicklistValues()[0].getValue();
        // Link Student with Skill
        Student_Skill__c ss = new Student_Skill__c(
            Student__c = stud.Id,
            Skill__c = skill.Id,
            Proficiency__c = validProficiency
        );
        insert ss;
```

```
        // Dynamically get a valid Offer_Status__c value for Placement
        Schema.DescribeFieldResult offerStatusField = Placement__c.Offer_Status__c.get
Describe();
        String validOfferStatus = offerStatusField.getPicklistValues()[0].getValue();
        // Create Placement linked to Student and Company
        Placement__c placement = new Placement__c(
            Name = 'Software Engineer Placement',
            Company__c = company.Id,            // ✅ Required field
            Company_Name__c = company.Name,
            Joining_date__c = Date.today().addMonths(1),
            Offer_Date__c = Date.today(),
            Offer_Package__c = 600000,
            Offer_Status__c = validOfferStatus,
            Package__c = 600000,
            Student__c = stud.Id
        );
        insert placement;
    }
    // 🔹 Test Career Readiness Calculation (Trigger + Service)
    @isTest
    static void testCareerReadinessCalculation(){
        Contact stu = [SELECT Id, GPA__c FROM Contact WHERE LastName = 'TestStu
dent' LIMIT 1];
        // Fire trigger
        stu.GPA__c = 9;
        update stu;
        Contact updatedStu = [SELECT Career_Readiness__c FROM Contact WHERE Id
= :stu.Id];
        System.assert(updatedStu.Career_Readiness__c > 0,
            'Career readiness should be calculated.');
    }
    // 🔹 Test API Callout with Mock (Job Posting Service)
    @isTest
    static void testCalloutWithMockAndFutureMethod(){
        Test.setMock(HttpCalloutMock.class, new MockJobApiResponse());
        Test.startTest();
        JobApiService.fetchJobsAsync();  // assumes you have JobApiService class implem
ented
        Test.stopTest();
```

```
        Integer countJobs = [SELECT COUNT() FROM Job_Post__c];
        System.assert(countJobs >= 0,
            'Job posts should be inserted from mock response');
    }
    // 🔹 Mock API Response for Job Service
    private class MockJobApiResponse implements HttpCalloutMock {
        public HTTPResponse respond(HTTPRequest req){
            HttpResponse res = new HttpResponse();
            res.setHeader('Content-Type','application/json');
            res.setBody('[{"id":"123","title":"Intern Software Engineer","company":"TechCo
rp"}]');
            res.setStatusCode(200);
            return res;
        }
    }
}
```

4. **Run the Test**
    - In Developer Console → **Test** → **New Run**.
    - Select StudentCareerServiceTest.
    - Run it.
    - Expected Result: Test passes, Coverage shows green for StudentCareerService and ContactTrigger.
5. **Verify It Works with Real Data**
    - In App Launcher → Contacts → New.
    - Record Type: Student
    - Enter GPA = 7.
    - Create a Skill (App Launcher → Skills → New).
    - Create Student_Skill (App Launcher → Student Skills → New → link Student + Skill).

- Go back to Student record → refresh → you should see Career Readiness field updated.



5.2 Add Supporting Apex Classes

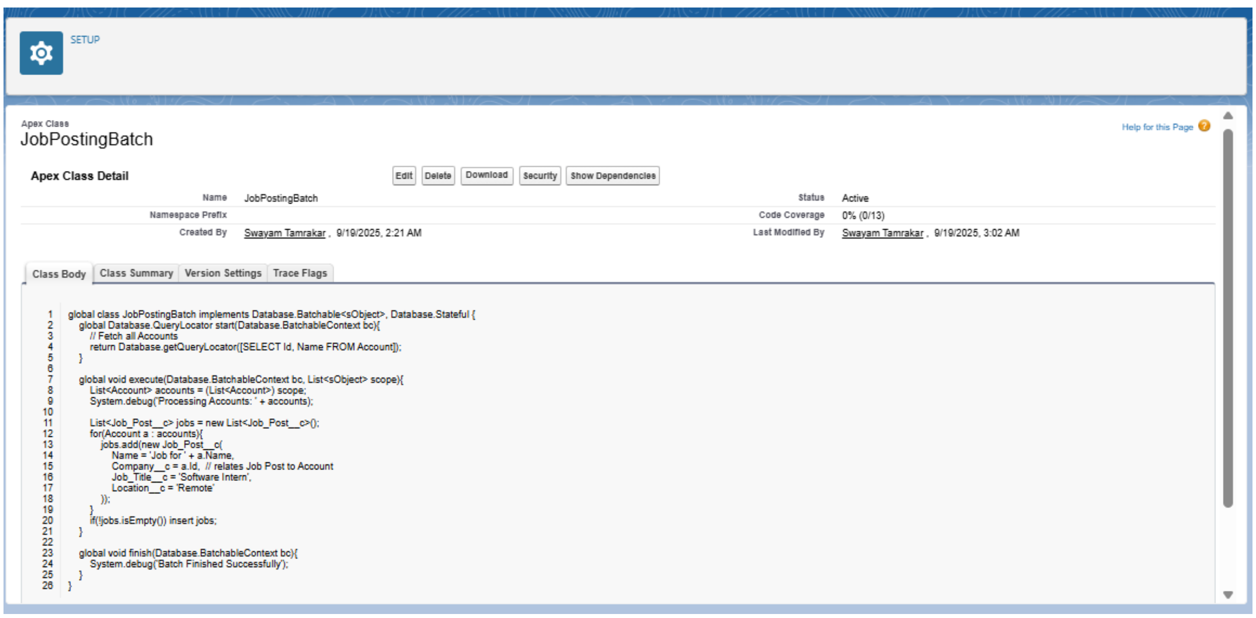
- Repeat **Setup** → **Apex Classes** → **New** for each:

1. **Batch Apex**

- **Code:**

```
global class JobPostingBatch implements Database.Batchable<sObject>, Database.Stateful {
    global Database.QueryLocator start(Database.BatchableContext bc){
        // Fetch all Accounts
        return Database.getQueryLocator([SELECT Id, Name FROM Account]);
    }
    global void execute(Database.BatchableContext bc, List<sObject> scope){
        List<Account> accounts = (List<Account>) scope;
        System.debug('Processing Accounts: ' + accounts);
        List<Job_Post__c> jobs = new List<Job_Post__c>();
        for(Account a : accounts){
            jobs.add(new Job_Post__c(
                Name = 'Job for ' + a.Name,
                Company__c = a.Id,  // relates Job Post to Account
```

```
            Job_Title__c = 'Software Intern',
            Location__c = 'Remote'
          ));
        }
        if(!jobs.isEmpty()) insert jobs;
      }
      global void finish(Database.BatchableContext bc){
        System.debug('Batch Finished Successfully');
      }
    }
```

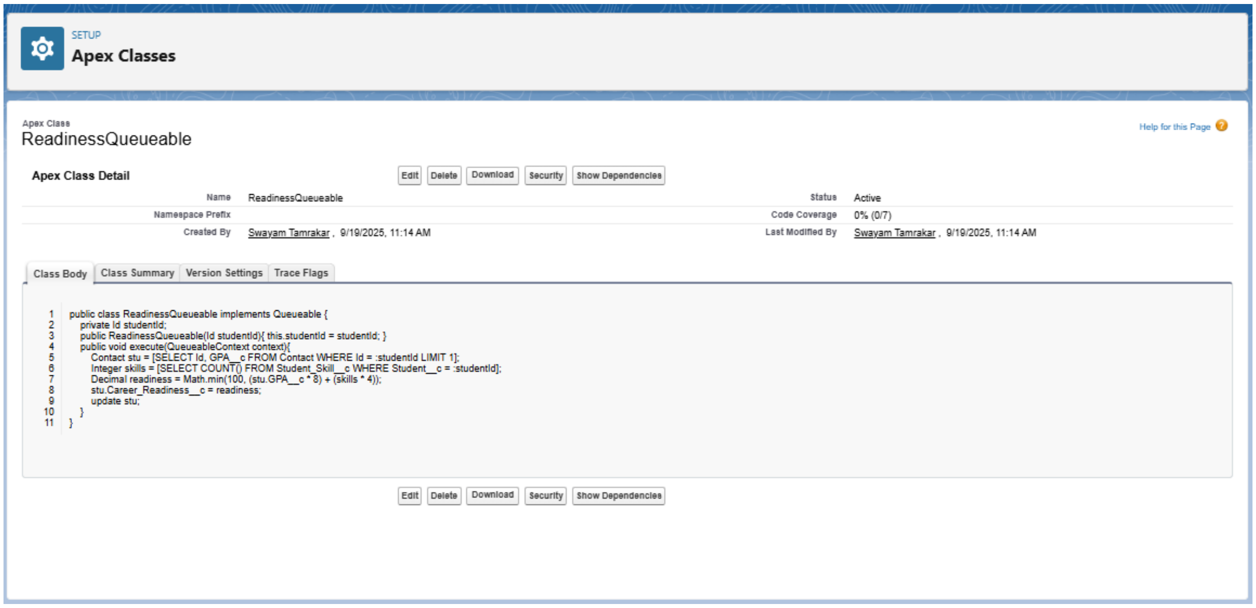


2. **Queueable Apex**

- **Code:**

```
public class ReadinessQueueable implements Queueable {
  private Id studentId;
  public ReadinessQueueable(Id studentId){ this.studentId = studentId; }
  public void execute(QueueableContext context){
    Contact stu = [SELECT Id, GPA__c FROM Contact WHERE Id = :studentId LI
MIT 1];
    Integer skills = [SELECT COUNT() FROM Student_Skill__c WHERE Student
__c = :studentId];
```

```
        Decimal readiness = Math.min(100, (stu.GPA__c * 8) + (skills * 4));
        stu.Career_Readiness__c = readiness;
        update stu;
    }
}
```



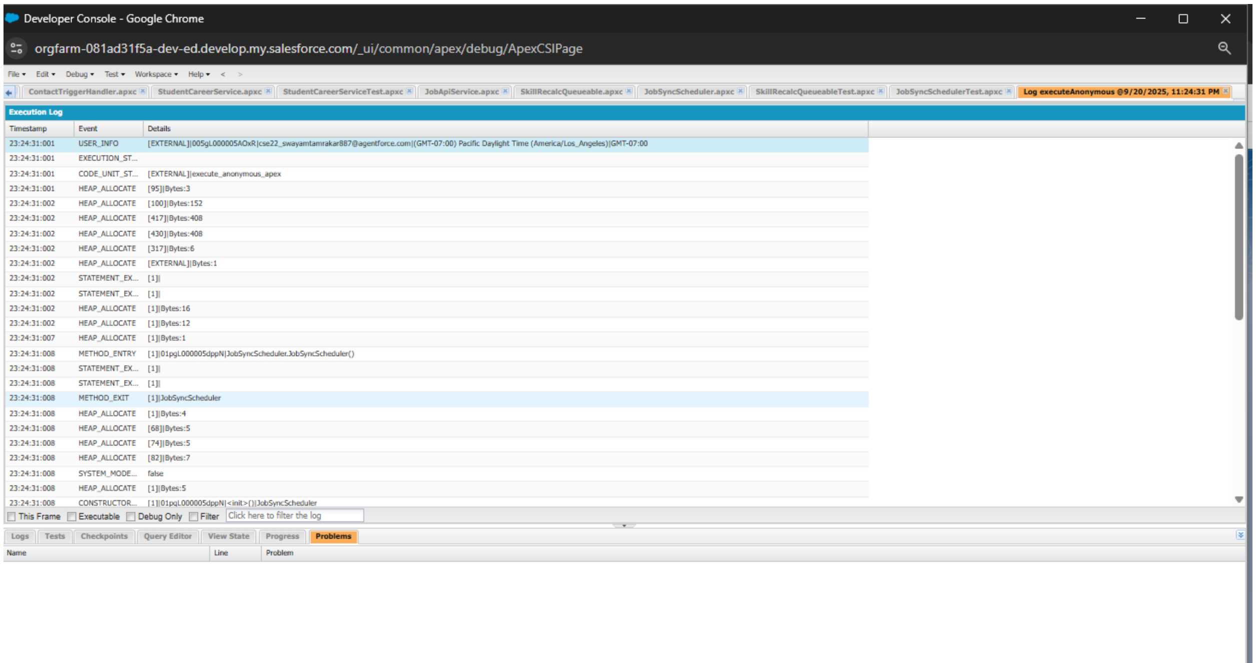
3. **Scheduled Apex**

- **Code:**

```
global class JobSyncScheduler implements Schedulable {
    global void execute(SchedulableContext sc){
        Database.executeBatch(new JobPostingBatch(), 100);
    }
}
```

- Run in **Execute Anonymous** (Developer Console → Debug → Open Execute Anonymous)**:**

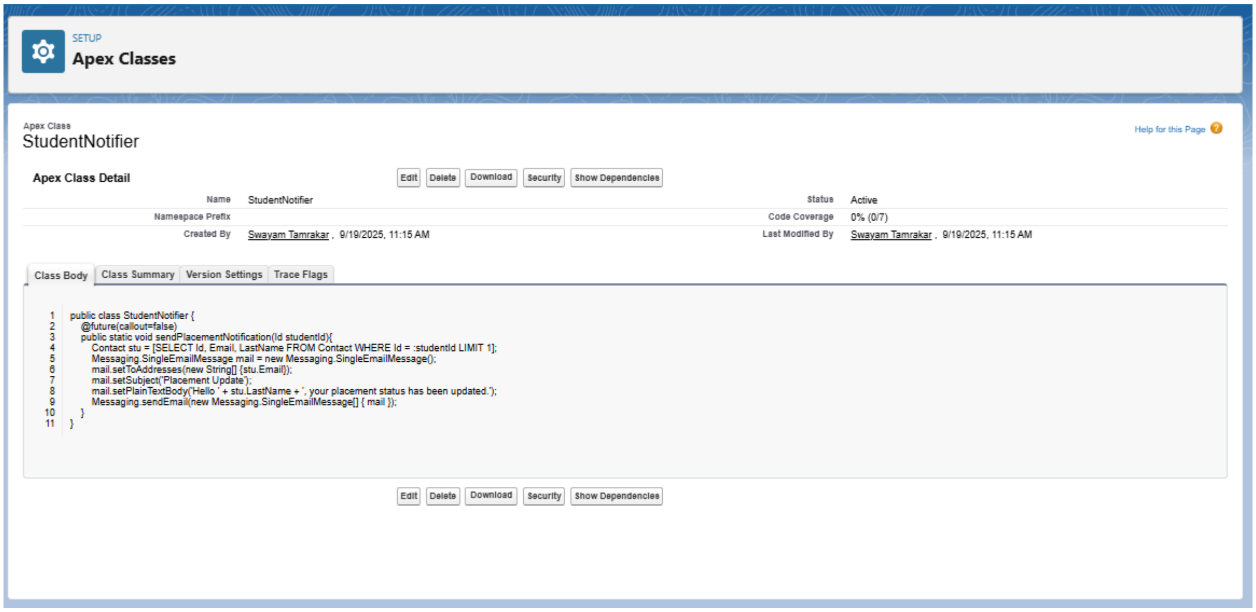
System.schedule('Nightly Job Sync', '0 0 23 * * ?', new JobSyncScheduler());



4. Future Method

- **Code:**

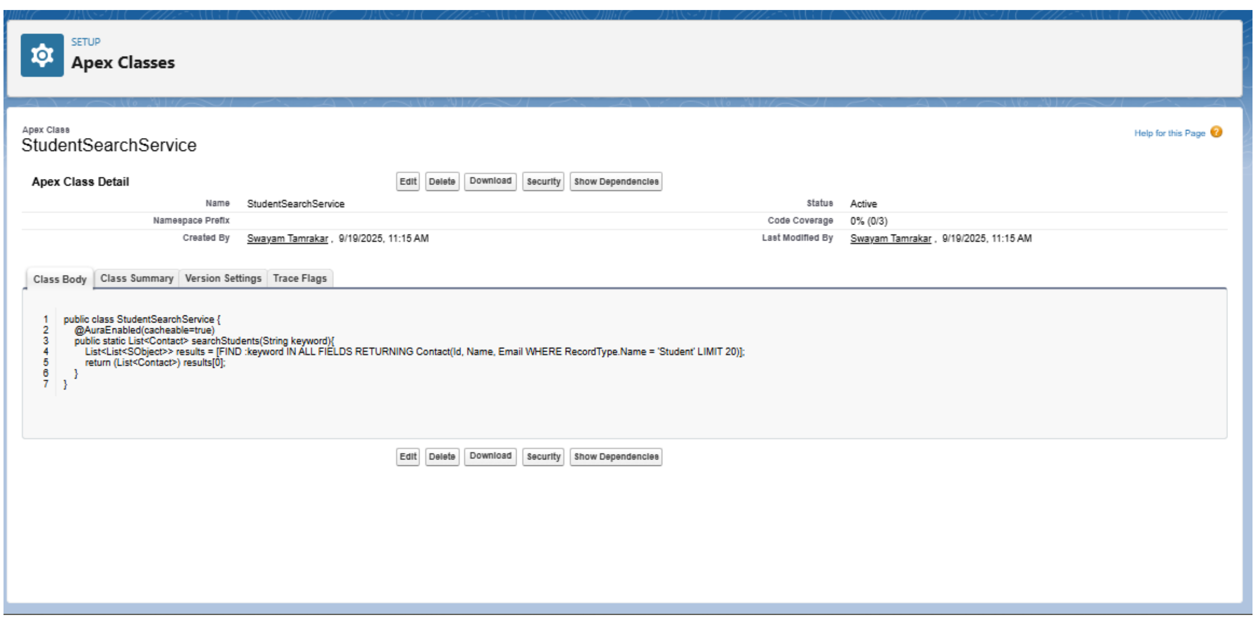
```
public class StudentNotifier {
   @future(callout=false)
   public static void sendPlacementNotification(Id studentId){
      Contact stu = [SELECT Id, Email, LastName FROM Contact WHERE Id = :studentId LIMIT 1];
      Messaging.SingleEmailMessage mail = new Messaging.SingleEmailMessage();
      mail.setToAddresses(new String[] {stu.Email});
      mail.setSubject('Placement Update');
      mail.setPlainTextBody('Hello ' + stu.LastName + ', your placement status has been updated.');
      Messaging.sendEmail(new Messaging.SingleEmailMessage[] { mail });
   }
}
```



5. SOSL

- **Code:**

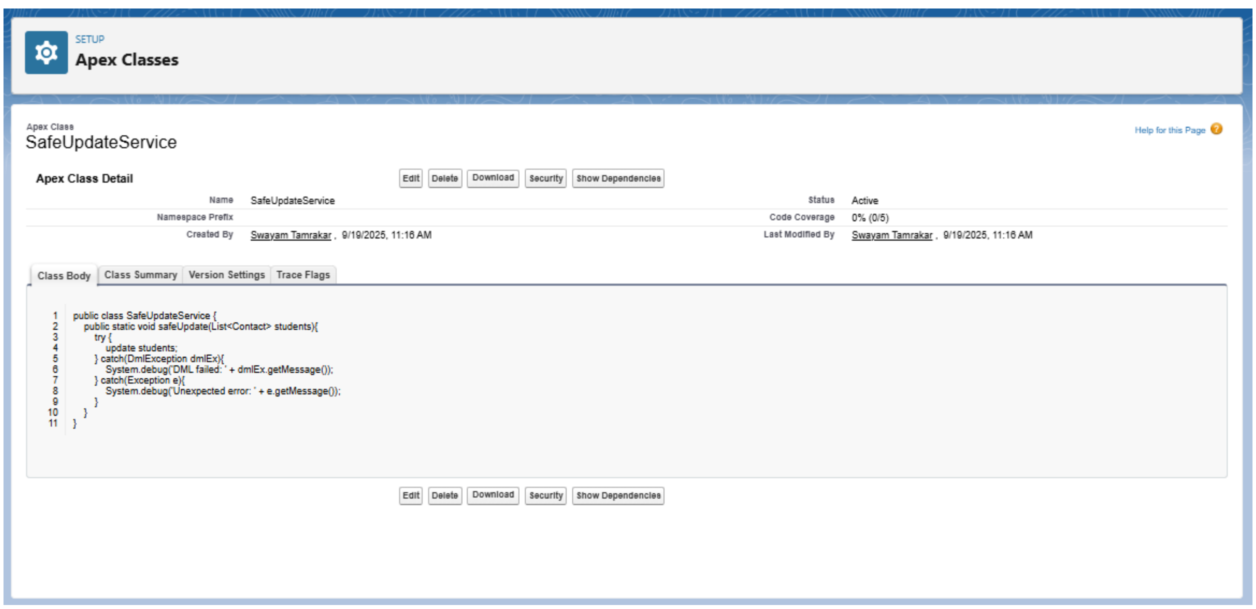
```
public class StudentSearchService {
   @AuraEnabled(cacheable=true)
   public static List<Contact> searchStudents(String keyword){
      List<List<SObject>> results = [FIND :keyword IN ALL FIELDS RETURNING Contact(Id
, Name, Email WHERE RecordType.Name = 'Student' LIMIT 20)];
      return (List<Contact>) results[0];
   }
}
```



6. **SOSL**

- **Code:**

```
public class SafeUpdateService {
   public static void safeUpdate(List<Contact> students){
      try {
         update students;
      } catch(DmlException dmlEx){
         System.debug('DML failed: ' + dmlEx.getMessage());
      } catch(Exception e){
         System.debug('Unexpected error: ' + e.getMessage());
      }
   }
}
```

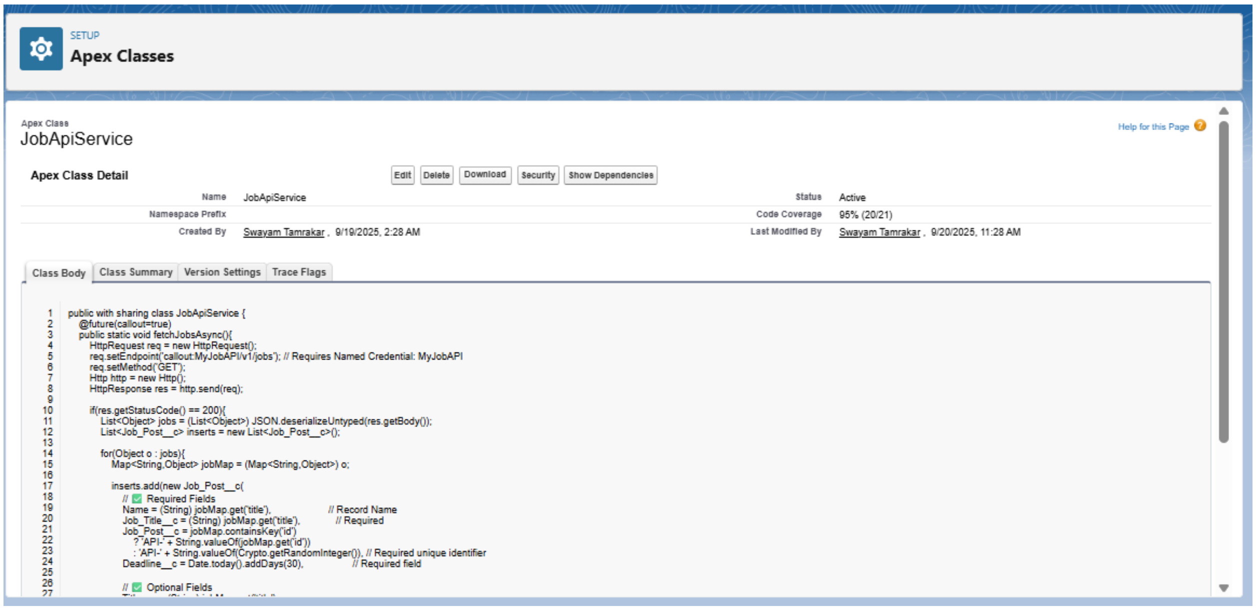


7. **JobApiService**

- **Code:**

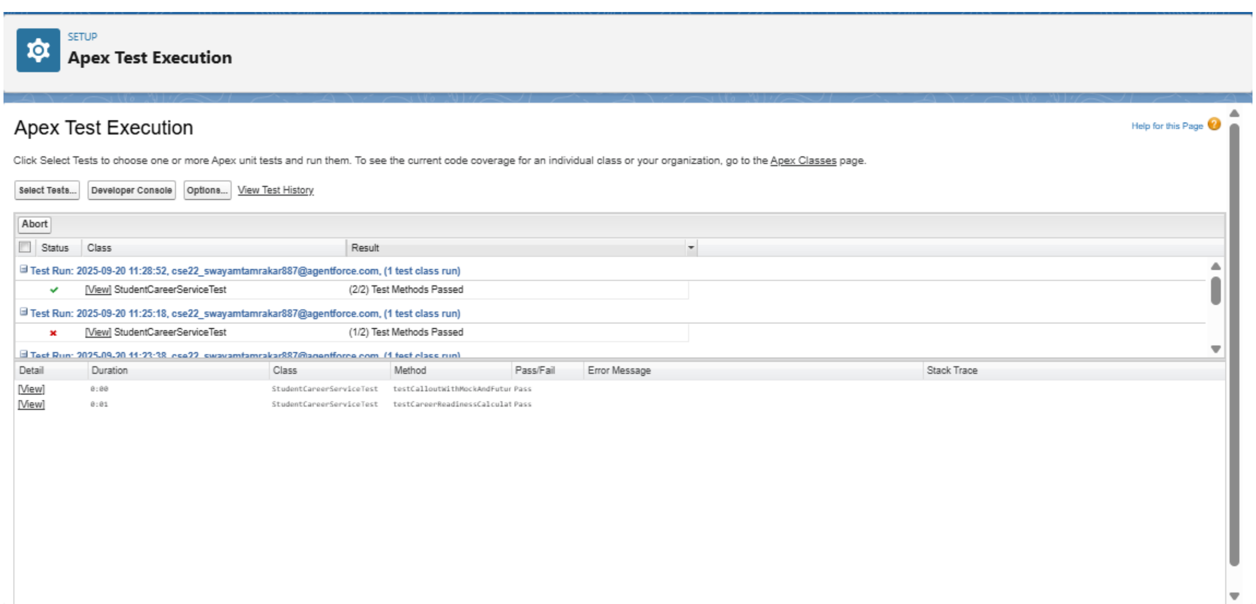
```
public with sharing class JobApiService {
   @future(callout=true)
   public static void fetchJobsAsync(){
      HttpRequest req = new HttpRequest();
      req.setEndpoint('callout:MyJobAPI/v1/jobs'); // Requires Named Credential:
MyJobAPI
      req.setMethod('GET');
      Http http = new Http();
      HttpResponse res = http.send(req)
      if(res.getStatusCode() == 200){
         List<Object> jobs = (List<Object>)
JSON.deserializeUntyped(res.getBody());
         List<Job_Post__c> inserts = new List<Job_Post__c>();
         for(Object o : jobs){
            Map<String,Object> jobMap = (Map<String,Object>) o;
            inserts.add(new Job_Post__c(
               // ✅ Required Fields
               Name = (String) jobMap.get('title'),                  // Record Name
               Job_Title__c = (String) jobMap.get('title'),          // Required
               Job_Post__c = jobMap.containsKey('id')
                  ? 'API-' + String.valueOf(jobMap.get('id'))
                  : 'API-' + String.valueOf(Crypto.getRandomInteger()), // Required
unique identifier
               Deadline__c = Date.today().addDays(30),               // Required field
               // ✅ Optional Fields
               Title__c = (String) jobMap.get('title'),
               Location__c = jobMap.containsKey('location') ? (String)
jobMap.get('location') : null
            ));
```

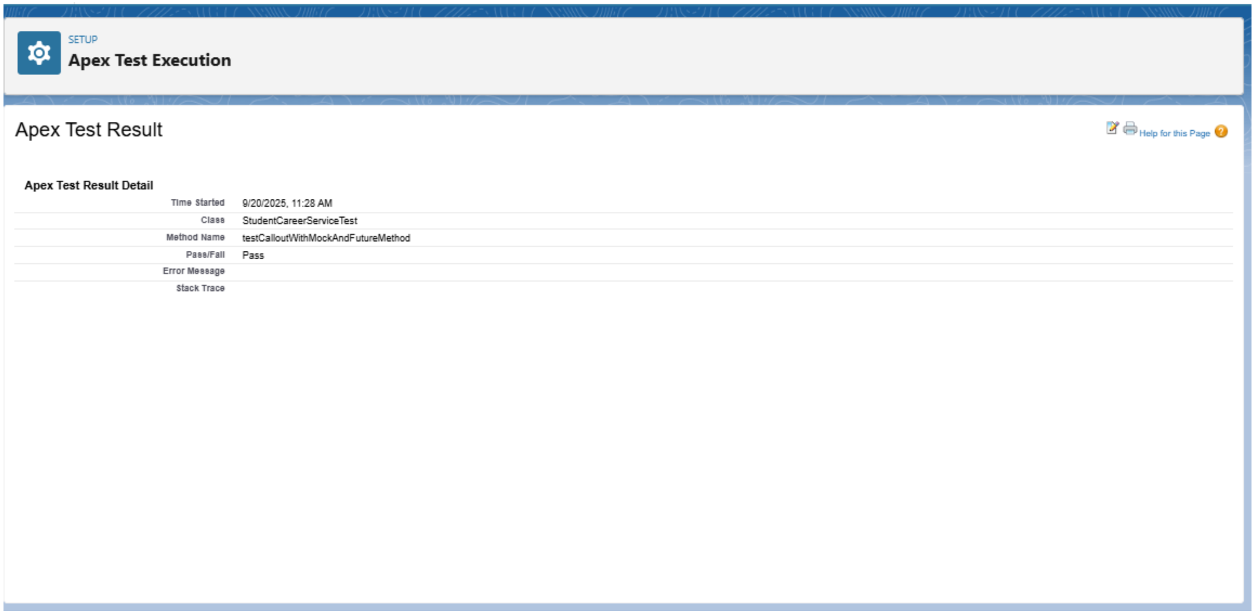
```
            }
            if(!inserts.isEmpty()) insert inserts;
        }
    }
}
```



8. Run Tests

- Setup → Apex Test Execution → Select StudentCareerServiceTest → Run.
- Expected: Pass with ≥75% coverage.




SETUP
Apex Test Execution
Apex Test Result
Help for this Page
Apex Test Result Detail
Time Started
9/20/2025, 11:28 AM
Class
StudentCareerServiceTest
Method Name
testCalloutWithMockAndFutureMethod
Pass/Fail
Pass
Error Message
Stack Trace

Phase 6: User Interface Development

6.1 Create Apex controller

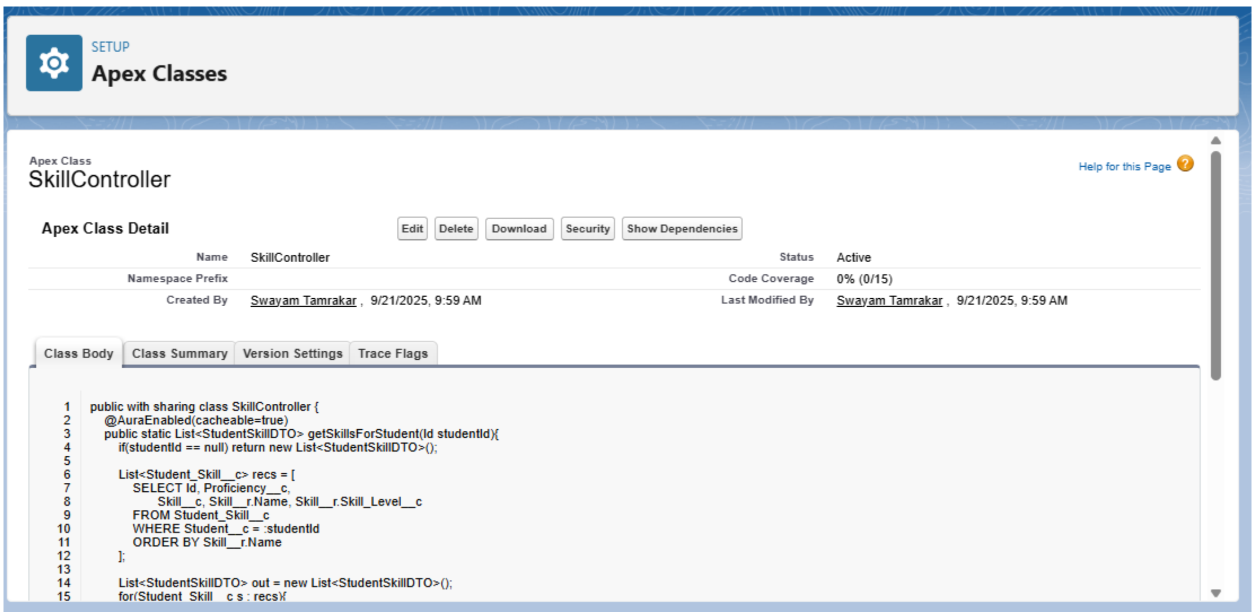
1. In the org (Setup) — quick option:

- Setup → Quick Find → **Apex Classes** → **New** → write a code → **Save**.

Apex

```
public with sharing class SkillController {
  @AuraEnabled(cacheable=true)
  public static List<StudentSkillDTO> getSkillsForStudent(Id studentId){
    if(studentId == null) return new List<StudentSkillDTO>();
    List<Student_Skill__c> recs = [
      SELECT Id, Proficiency__c,
          Skill__c, Skill__r.Name, Skill__r.Skill_Level__c
      FROM Student_Skill__c
      WHERE Student__c = :studentId
      ORDER BY Skill__r.Name
    ];
    List<StudentSkillDTO> out = new List<StudentSkillDTO>();
    for(Student_Skill__c s : recs){
      String skillName = (s.Skill__r != null) ? s.Skill__r.Name : null;
      String skillLevel = (s.Skill__r != null) ? s.Skill__r.Skill_Level__c : null;
      out.add(new StudentSkillDTO(s.Skill__c, skillName, skillLevel, s.Proficiency__c));
    }
    return out;
  }
  public class StudentSkillDTO {
    @AuraEnabled public Id skillId;
    @AuraEnabled public String skillName;
```

```
    @AuraEnabled public String skillLevel;
    @AuraEnabled public String proficiency;
    public StudentSkillDTO(Id skillId, String skillName, String skillLevel, String proficiency)
{
        this.skillId = skillId;
        this.skillName = skillName;
        this.skillLevel = skillLevel;
        this.proficiency = proficiency;
    }
  }
}
```



Notes:

- This returns a small DTO list ready for LWC.
- cacheable=true lets the wire adapter cache results for performance.

6.2 Create the LWC (skillTracker) in VS Code

1. In VS Code

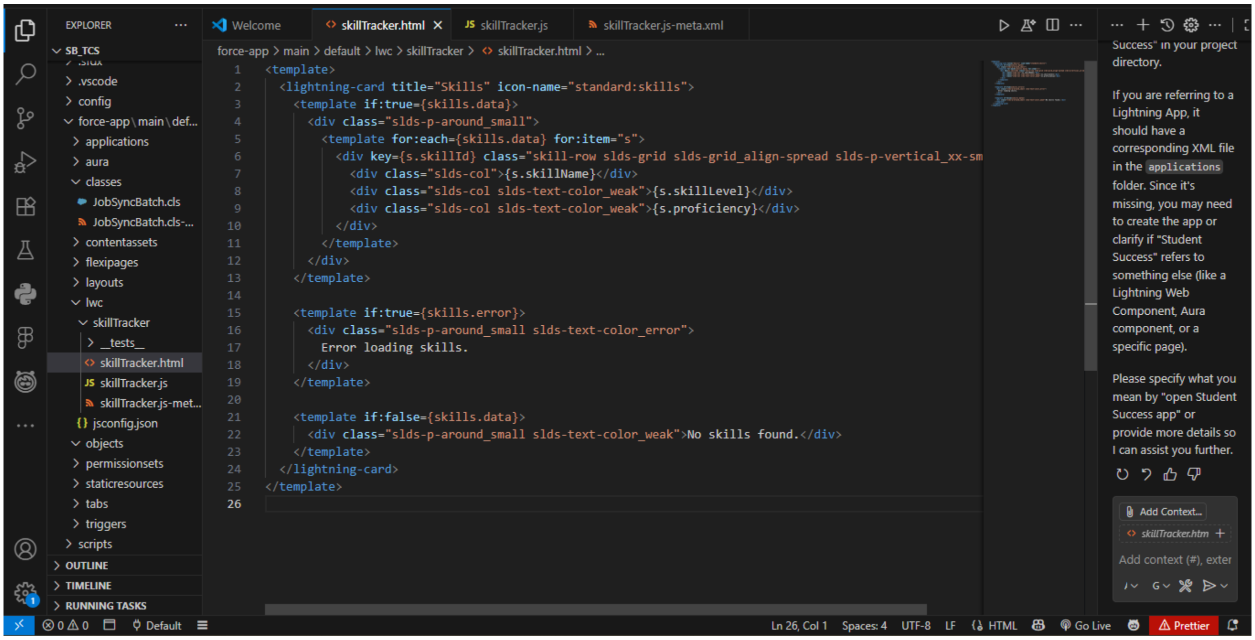
- Command palette → **SFDX: Create Project**.
- Connect to org: **SFDX: Authorize an Org** → choose your Developer Org.
- In the force-app/main/default/lwc folder → right-click → **SFDX: Create Lightning Web Component** → name it skillTracker.

You’ll get three files. Replace them with the code below.

- **skillTracker.html**

```
<template>
  <lightning-card title="Skills" icon-name="standard:skills">
    <template if:true={skills.data}>
      <div class="slds-p-around_small">
        <template for:each={skills.data} for:item="s">
          <div key={s.skillId} class="skill-row slds-grid slds-grid_align-spread slds-p-vertical_xx-small">
            <div class="slds-col">{s.skillName}</div>
            <div class="slds-col slds-text-color_weak">{s.skillLevel}</div>
            <div class="slds-col slds-text-color_weak">{s.proficiency}</div>
          </div>
        </template>
      </div>
    </template>
    <template if:true={skills.error}>
      <div class="slds-p-around_small slds-text-color_error">
        Error loading skills.
      </div>
    </template>
```

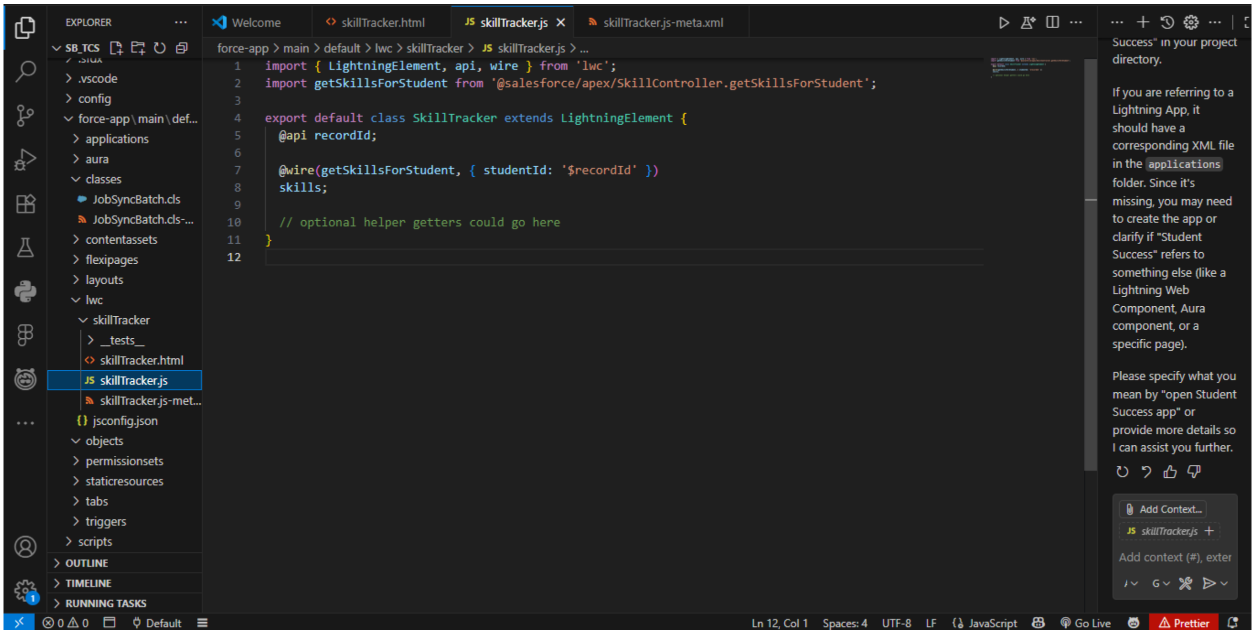
```
    <template if:false={skills.data}>
      <div class="slds-p-around_small slds-text-color_weak">No skills found.</div>
    </template>
  </lightning-card>
</template>
```



- **skillTracker.js**

```
import { LightningElement, api, wire } from 'lwc';
import getSkillsForStudent from '@salesforce/apex/SkillController.getSkillsForStudent';
export default class SkillTracker extends LightningElement {
  @api recordId;
  @wire(getSkillsForStudent, { studentId: '$recordId' })
  skills;
  // optional helper getters could go here
```

}



- **skillTracker.js-meta.xml**

```
<?xml version="1.0" encoding="UTF-8"?>

<LightningComponentBundle xmlns="http://soap.sforce.com/2006/04/metadata">

  <apiVersion>59.0</apiVersion>

  <isExposed>true</isExposed>

  <targets>

    <target>lightning__RecordPage</target>

    <target>lightning__AppPage</target>

  </targets>

  <targetConfigs>

    <targetConfig targets="lightning__RecordPage">

      <objects>

        <object>Contact</object>

      </objects>

    </targetConfig>
```

</targetConfigs>

</LightningComponentBundle>

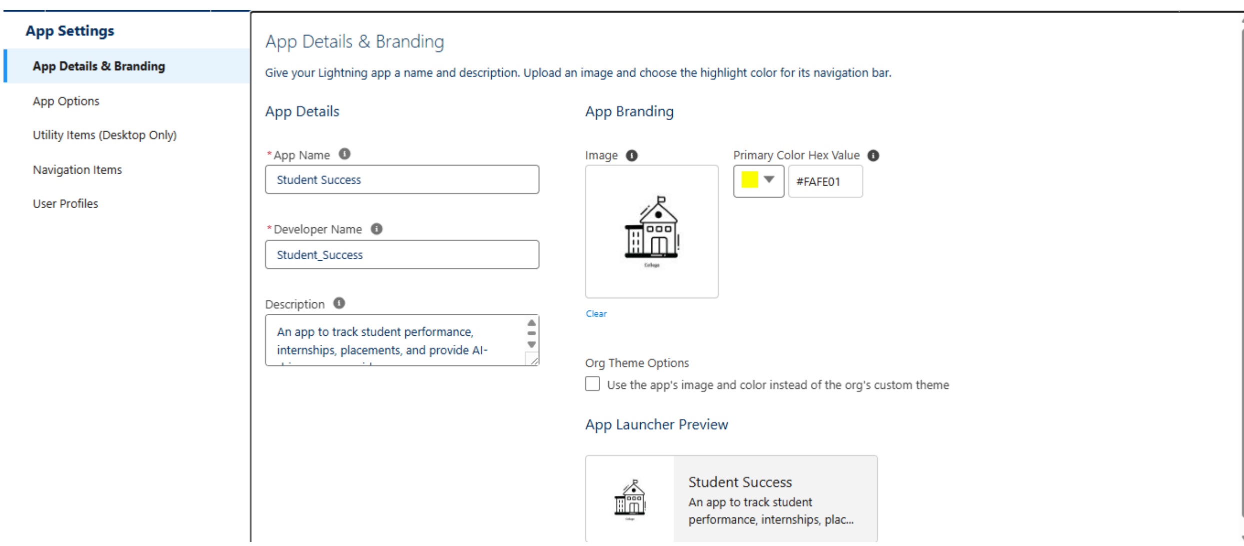
2. Deploy to Org

- In VS Code explorer → right-click component folder → **SFDX: Deploy Source to Org**.
- Or run CLI: sfdx force:source:deploy -p force-app/main/default/lwc/skillTracker.

6.3 Add the LWC & Components to Contact Record Page

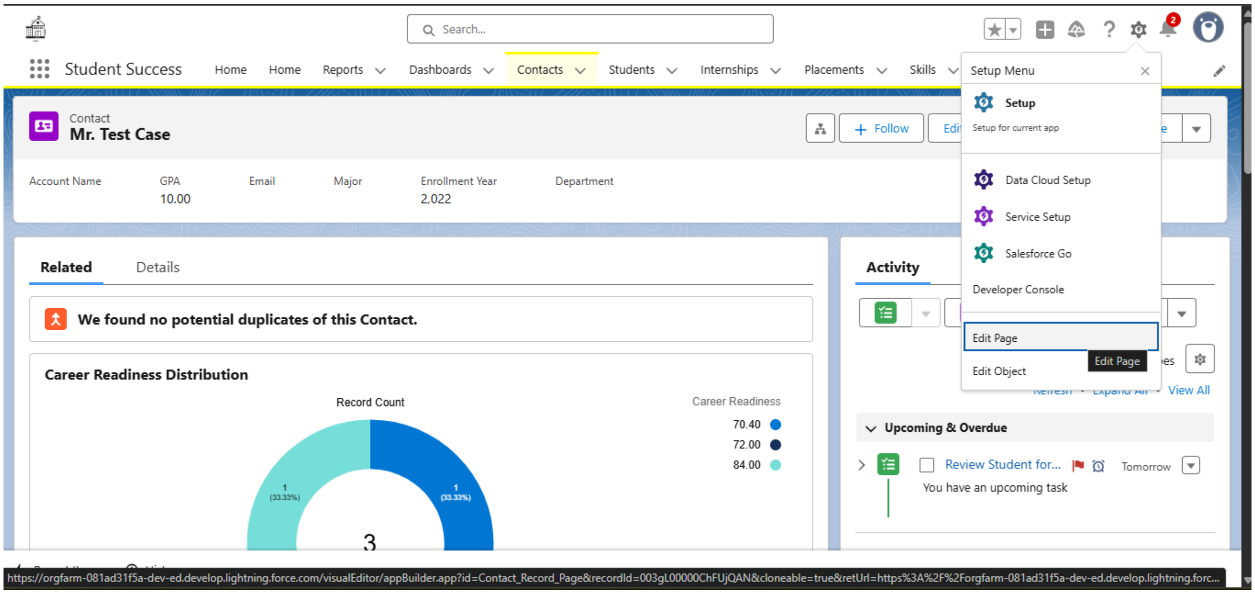
1. Create Lightning App

- Setup → **App Manager → New Lightning App**
  - App Name: Student Success → Next → choose branding (color/logo) → Next.
  - Add Navigation Items: **Contacts**, **Internships** (your custom object), **Placements**, **Job_Posts**, **Reports**, **Dashboards, Student Skills, Skills, e.t.c** → Finish.



2. Edit Contact Record Page

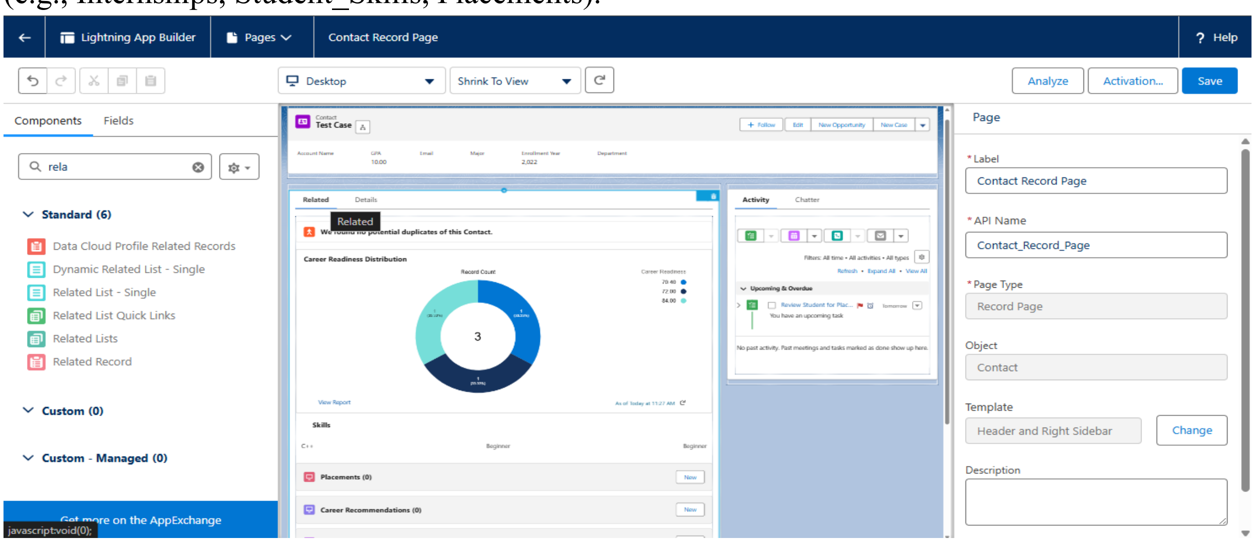
- App Launcher → open **Contacts** → open any Contact record → click the gear icon (⚙️) top-right → **Edit Page**. This opens **Lightning App Builder**.



- You're now editing the Contact Record Page. Do these:

Add Related Lists

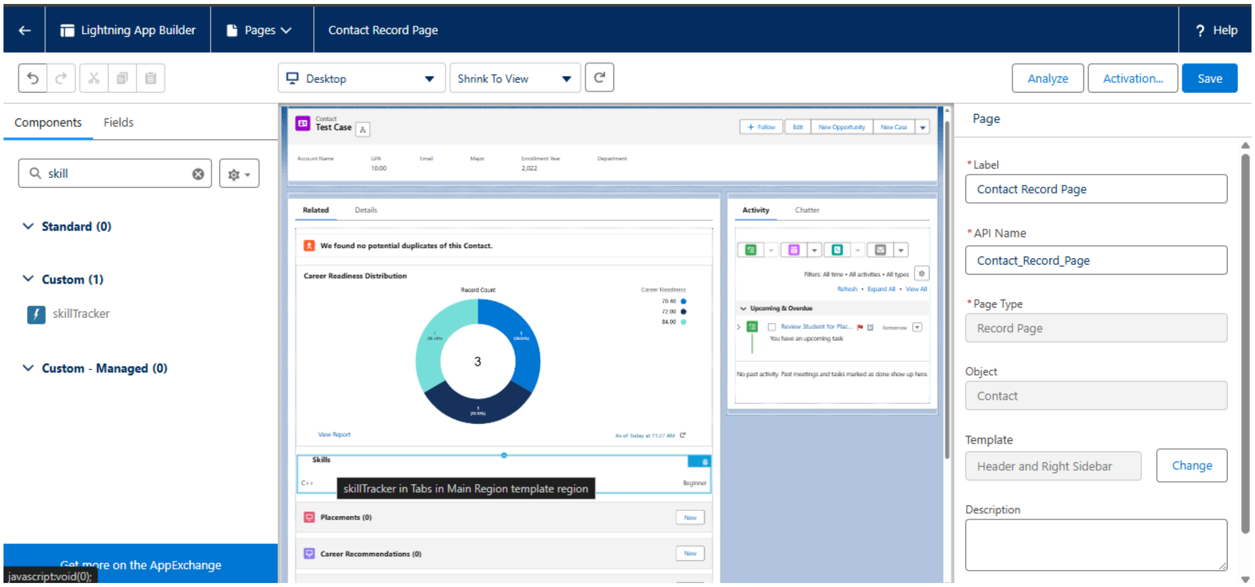
- From left components panel → drag **Related Lists** (or **Related List - Single**) onto the page where you want them.
- Click the related list component → in the right pane choose which related list to show (e.g., Internships, Student_Skills, Placements).



- Save & Activate.

Add the skillTracker LWC

- In left panel → find **Custom** section → your skillTracker component → drag it onto the page (a column/region).



Add a Report Chart (Career Readiness distribution)

- Create a report.

    I. Create a Custom Report Type for Student Skills
    - Go to **Setup.**
    - Quick Find → **Report Types** → click **New Custom Report Type**.
    - Configure:
        - ✓ **Primary Object**: Contacts
        - ✓ **Report Type Label**: Contacts with Student Skills
        - ✓ **Description**: Report showing Contacts and their related Student Skills.
        - ✓ **Category**: Choose Other Reports
        - ✓ **Deployment Status**: Deployed
    - Click **Next.**
    - Related Object: Student_Skill__c.
    - Choose: "Each A record must have at least one related B record."
    - **Save**

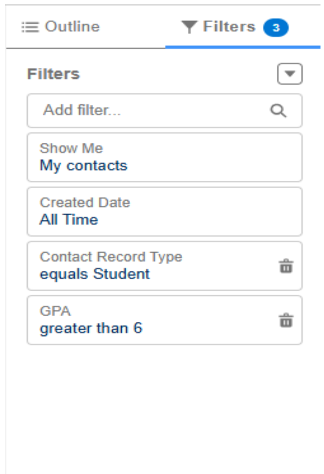
    II. Create a Report from this New Report Type
    - App Launcher → **Reports** → **New Report**.
    - In Report Type selector, search for **Contacts with Student Skills**.
    - Add Fields:
        - ✓ Name

- ✓ Career_Readiness__c
- ✓ Skill__r.Name
- ✓ Proficiency__c

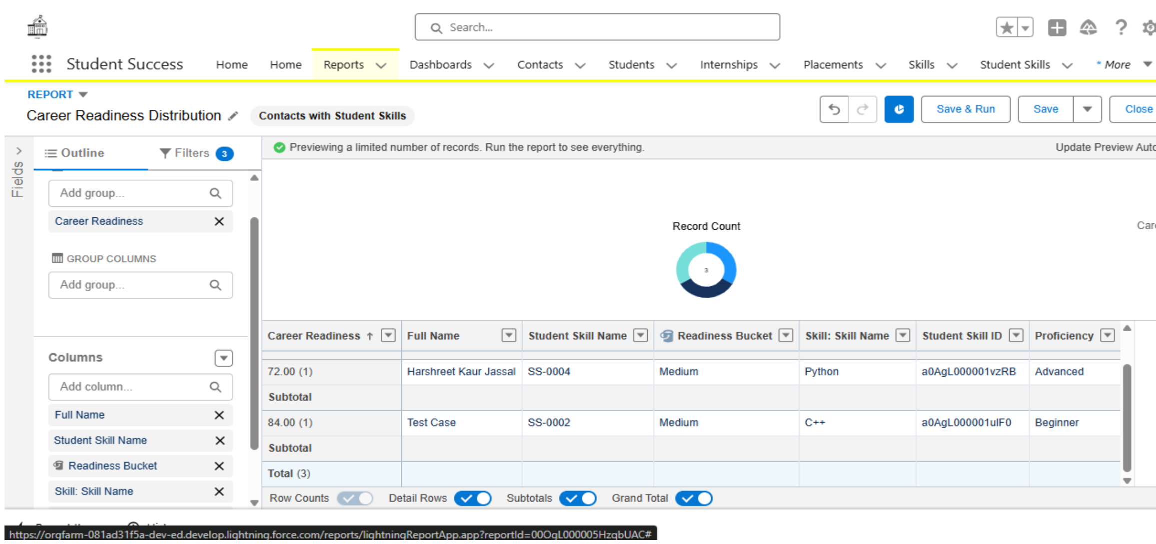
- o Buckets:
  - ✓ In the left **Fields** pane, find the Career_Readiness__c field.
  - ✓ Click the chevron (▾) next to that field and choose **Bucket this field** (or click **Add | Bucket Field** at top of Fields pane).
  - ✓ Give the bucket a name: **Readiness Bucket**.
  - ✓ Choose **Bucket By: Numeric Range** (or the UI will allow you to create ranges).
  - ✓ Create ranges, for example:
    - Low : 0 to 30
    - Medium : 31 to 60
    - High : 61 to 100

- o Group rows:
  - ✓ Drag the **Readiness Bucket** from **Fields** into the **Group Rows** area on the left (where it says *Add group…*).
  - ✓ You should now see **GROUP ROWS** populated — the report preview updates.

- o Add Filter:
  - ✓ On the right side, open the **Filters** pane.
  - ✓ By default you may see a Date filter (e.g., Show Me / Created Date). Remove or set it to *All Time*.
  - ✓ Click **Add Filter**.
    - Field: type **Record Type** → choose **Record Type** (Contact.Record Type).
    - Operator: **equals**.
    - Value: choose **Student**
    - Click **Apply**.



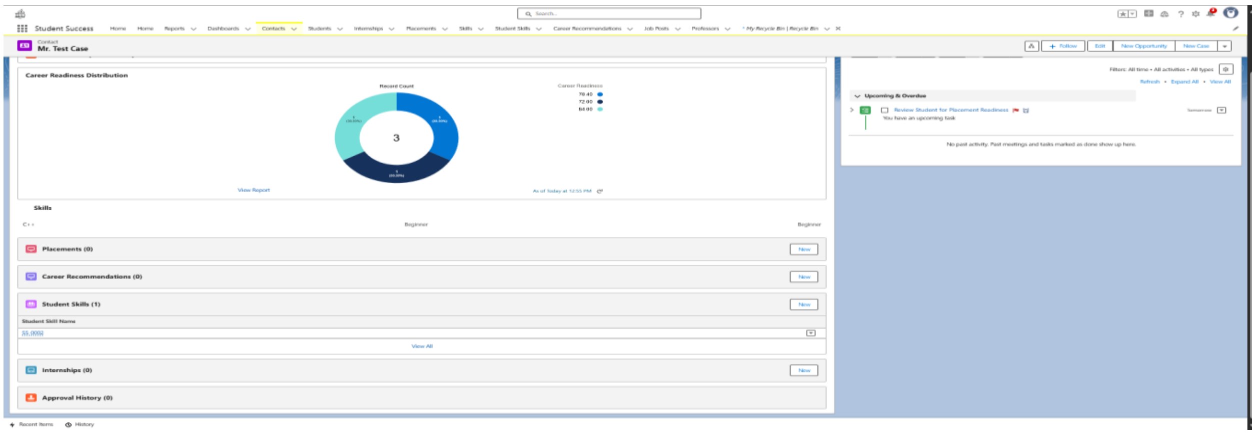
- ✓ Optional: add any other filters (Department, GPA > 6.0, etc).
- ○ Add Chart:
  - ✓ Once you have a group, **Add Chart** becomes active (top bar). Click **Add Chart**.
  - ✓ In the chart configuration panel:
    - **Chart Type**: choose **Donut** (good for distribution) or **Bar**.
    - **Grouping**: should already be set to the group you made
    - **Values**: default is **Record Count**
    - Optional: show percentage or values in chart options.
- ○ Click Done



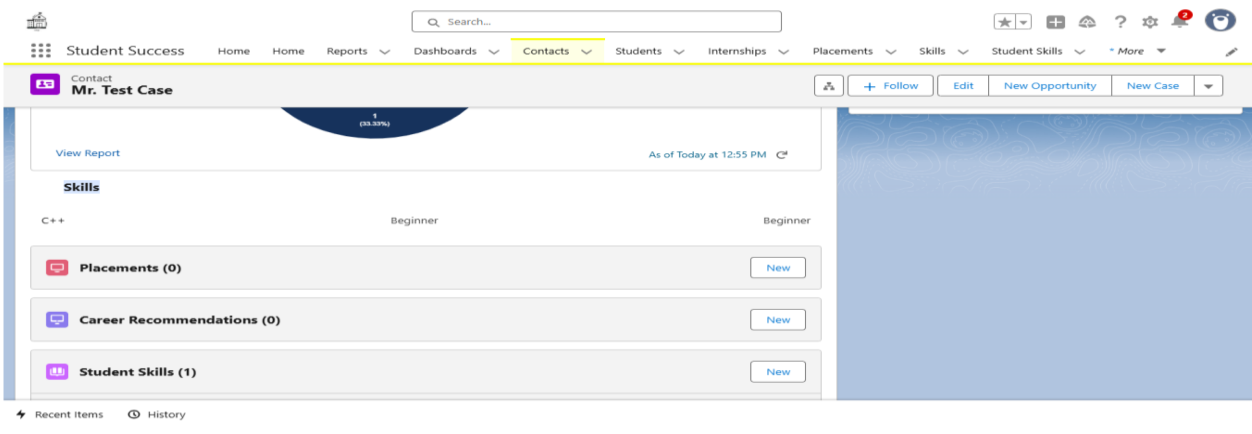
- Save & Run the report.
  - ○ Click **Save & Run**.
  - ○ Name the report: **Career Readiness Distribution**.
  - ○ Choose a folder (e.g., My Personal Reports or a team folder).
  - ○ Click **Save** → then **Run**.

6.4 Test everything in Org

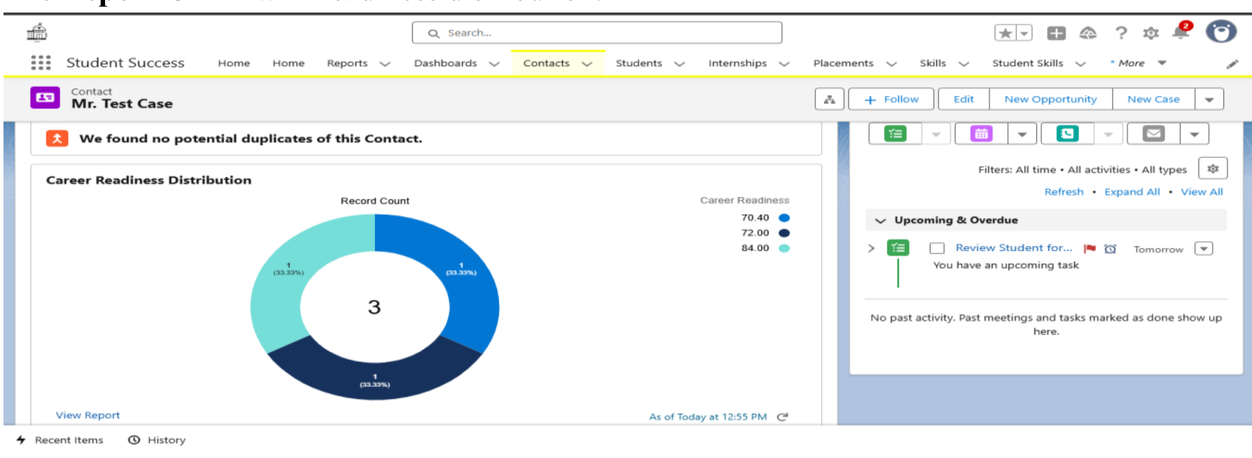
1. Create a **Contact** with Record Type Student. Add GPA and other fields.
2. Create a **Skill__c** record and a Student_Skill__c linking the Student.
3. Open the Student Contact record. The page should show:
    - Related lists (Internships, Skills).



    - The **Skills** LWC showing skill name, skill level, proficiency.



    - The **Report Chart** with readiness distribution.



Phase 7: Integration & External Access

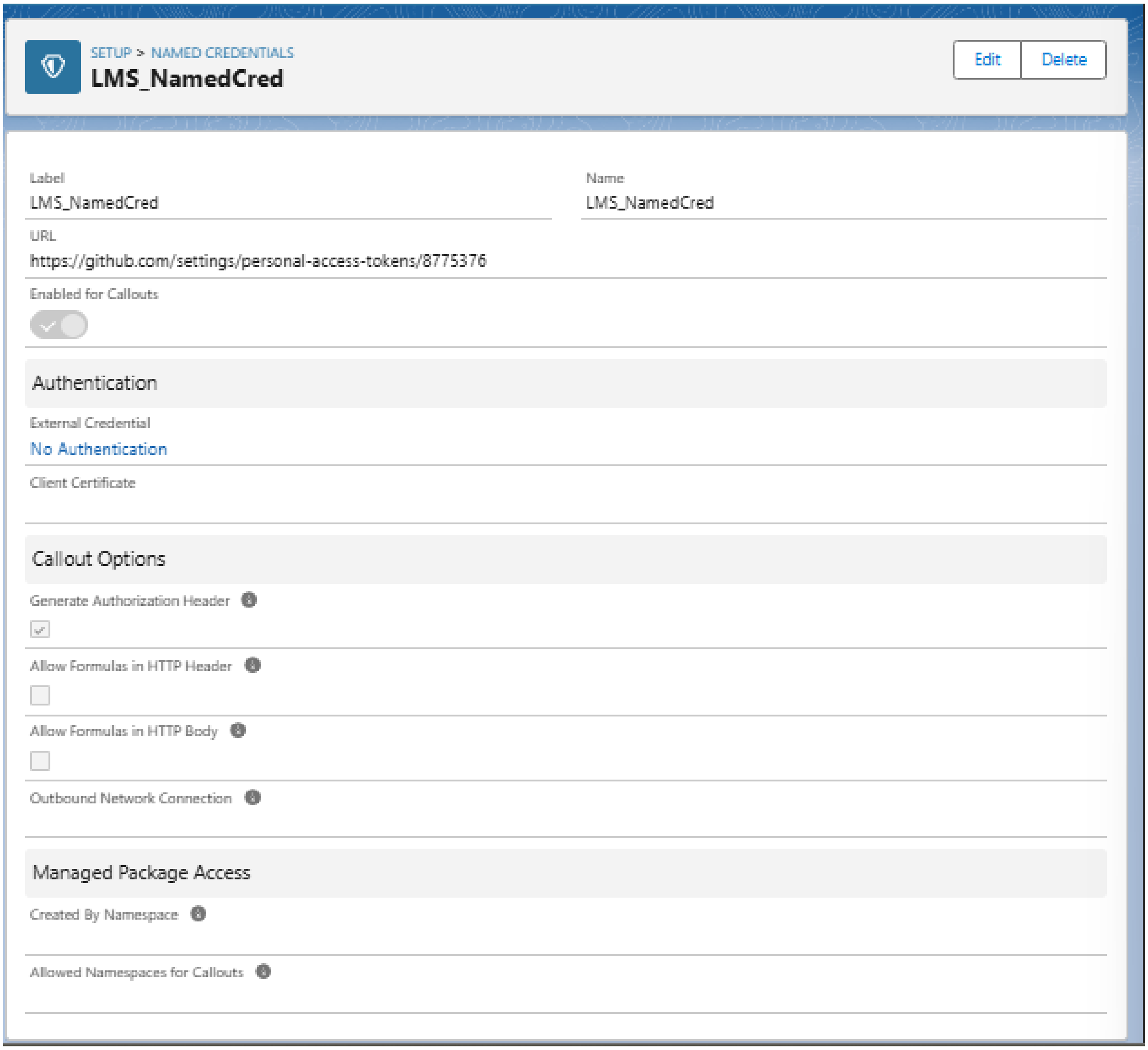
7.1 Named Credentials (preferred for callouts)

When to use: Salesforce makes *callouts to external services* (LMS, Job APIs). **Named Credentials** centralize the endpoint + auth. Use them instead of hard-coding endpoints.

A. If external API is public or uses simple credentials

1. Setup → **Named Credentials** → **New Named Credential**.
2. Fill:
   - **Label:** LMS_NamedCred
   - **Name:** LMS_NamedCred
   - **URL:** https://api.mylms.com (replace with actual API root)
   - **Identity Type: Named Principal** (single identity for all callouts)
   - **Authentication Protocol: Password Authentication** (or **No Authentication** for public)
   - Enter username/password if using password auth.

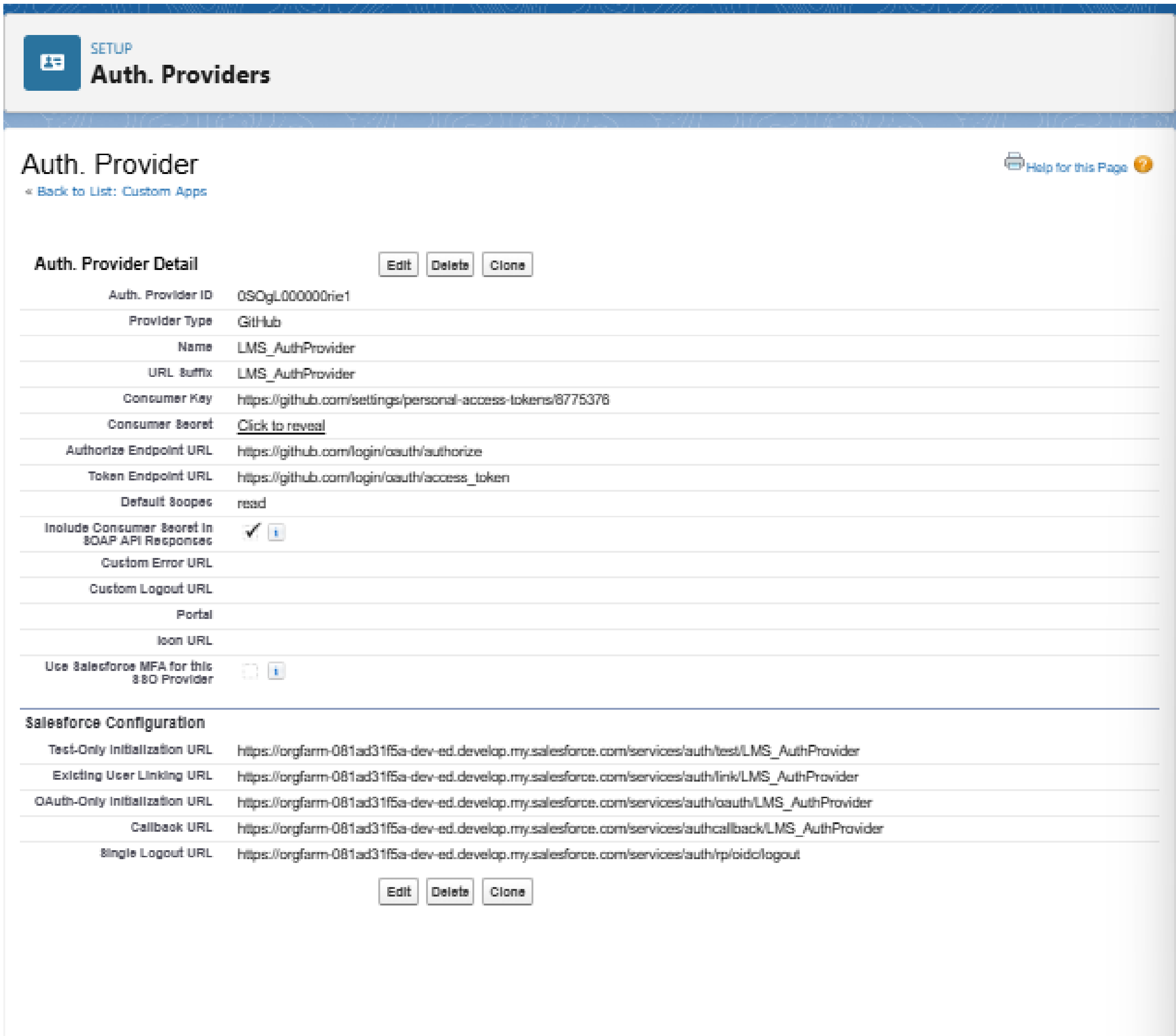
3. Save.



B. If external API uses OAuth 2.0 (recommended) — create an Auth. Provider first

1. Setup → **Auth. Providers** → **New**.
    - **Provider Type:** choose the provider that matches the API (e.g., **GitHub** or the specific vendor if listed).
    - **Name:** LMS_AuthProvider
    - **Consumer Key / Secret:** get these from the external API developer console (not Salesforce).
    - **Authorize Endpoint URL / Token Endpoint URL / User Info Endpoint URL:** paste the external API's OAuth URLs.

- **Default Scopes:** e.g., read write (depends on external API).
- Save.



2. After Save, on the Named Credential detail, click **Authenticate** (or **Authorize**). That opens the external provider's login page. Complete the OAuth flow and the Named Credential will store the access token/refresh token.

Use in Apex (callout):

```
HttpRequest req = new HttpRequest();

req.setEndpoint('callout:LMS_NamedCred/v1/courses'); // callout: + NamedCredential + path

req.setMethod('GET');

Http http = new Http();
```

HttpResponse res = http.send(req);

System.debug(res.getStatus());

System.debug(res.getBody());

Notes:

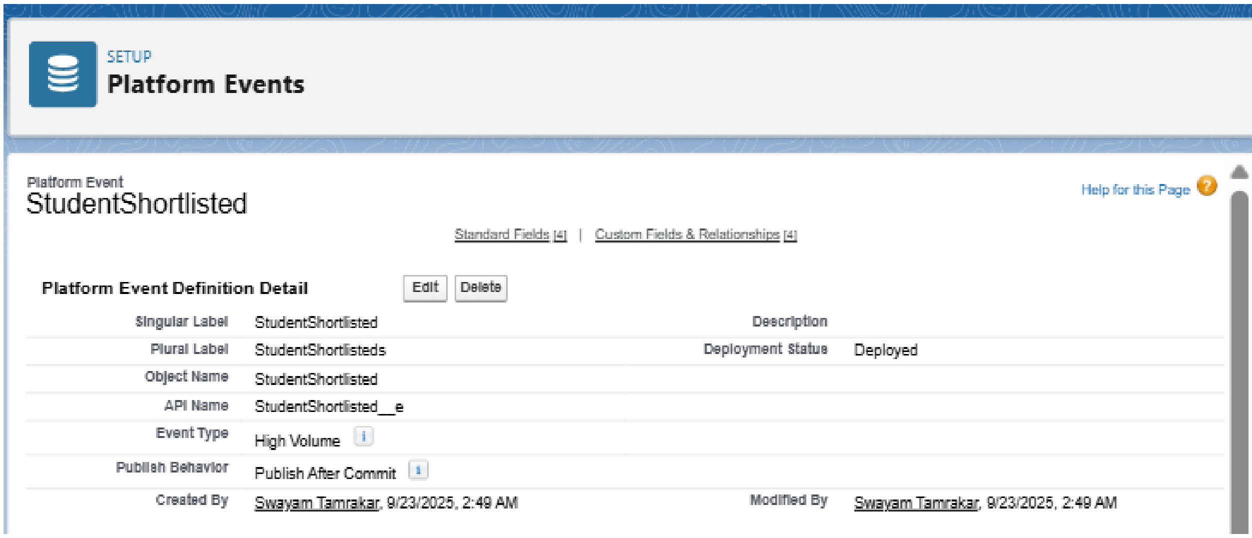
- Use callout:NamedCredentialName/path as the endpoint — Salesforce replaces host & auth.
- Named Credentials handle token refresh for OAuth providers automatically.

7.2 Platform Events

When to use: publish real-time notifications to external systems or internal subscribers (e.g., notify recruiter system when a student is shortlisted).

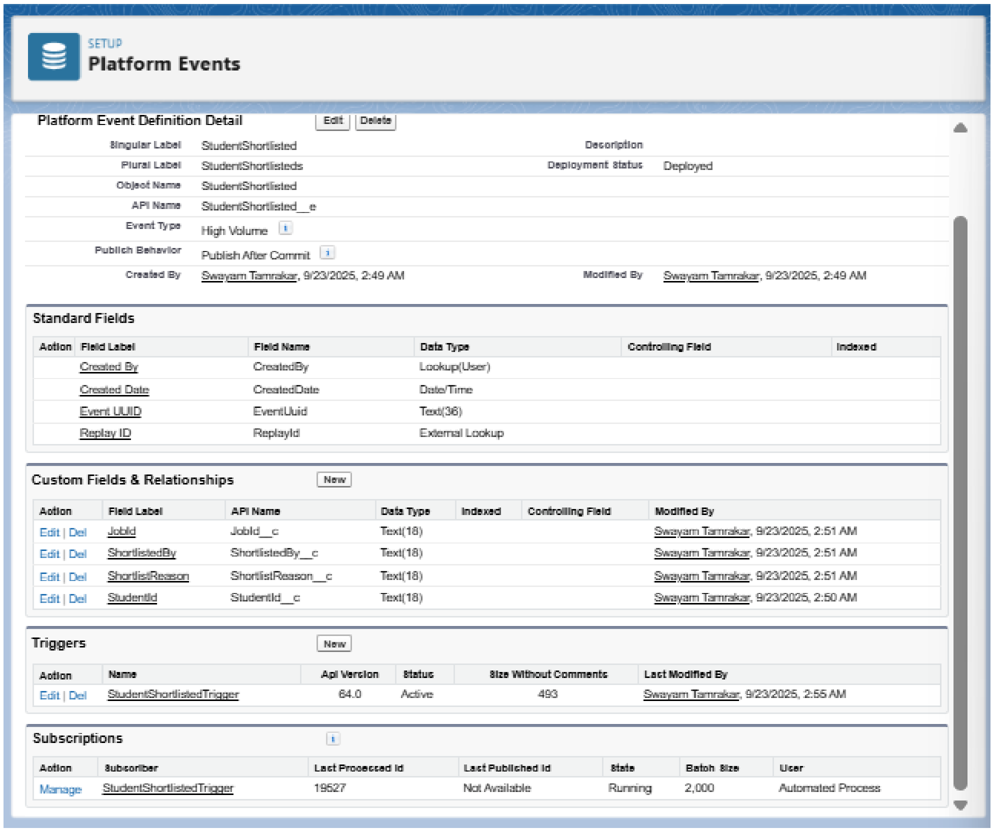
Create the Platform Event

1. Setup → **Platform Events → New Platform Event**.
    - **Label:** StudentShortlisted
    - **Object Name:** StudentShortlisted__e (platform events end with __e)
    - **Publish Behavior: Publish After Commit** (recommended)
    - Save.



2. Add fields: while on the event detail → **Fields & Relationships → New**:
    - **Field 1:** StudentId__c — Data Type: Text (Length: 18)

- **Field 2:** JobId__c — Data Type: Text (Length: 18)
- (Add any other fields like ShortlistReason__c, ShortlistedBy__c)



Publish a Platform Event (Apex example)

StudentShortlisted__e evt = new StudentShortlisted__e(

StudentId__c = '0033t00002ABCDE',

JobId__c = 'a0E3t00000XYZ12'

);

// Publish

Database.SaveResult sr = EventBus.publish(evt);

System.debug('Published: ' + sr.isSuccess());

Subscribe to Platform Events

- **Apex trigger subscriber** (internal processing):

trigger StudentShortlistedTrigger on StudentShortlisted__e (after insert) {

for (StudentShortlisted__e evt : Trigger.New) {

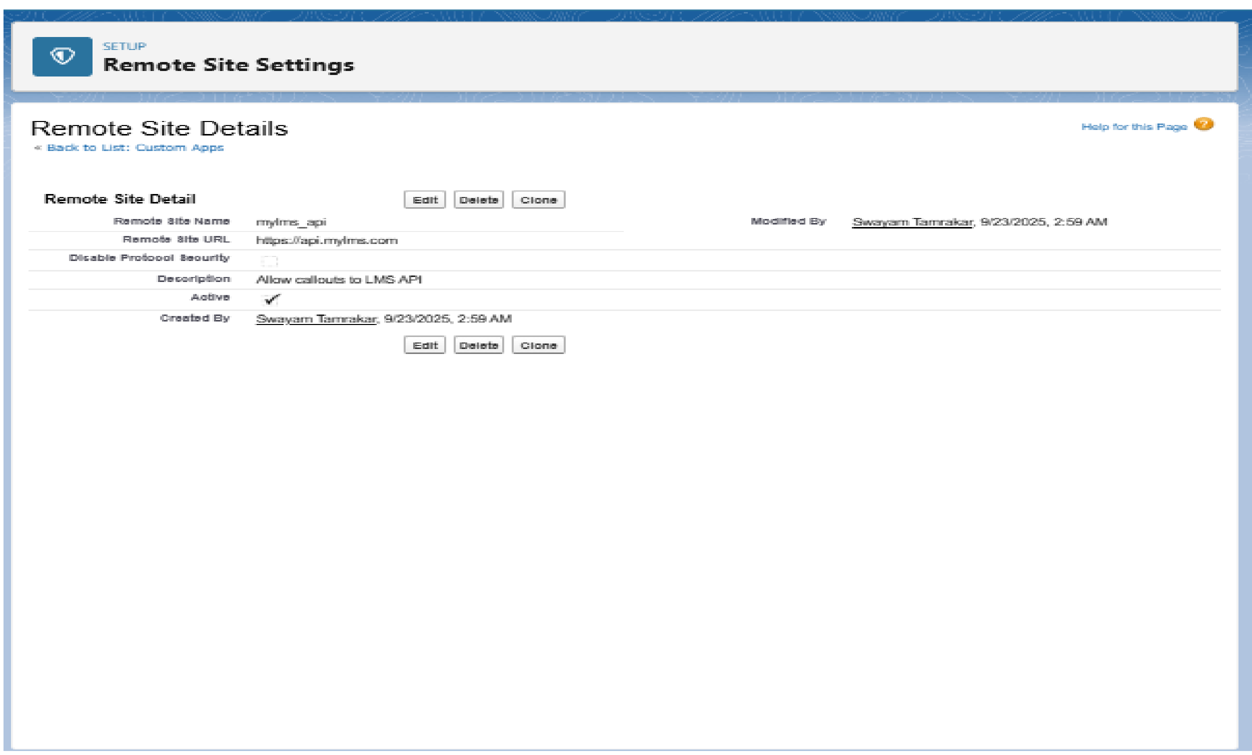
```
    // e.g., create a Task for Placement Officer
    // evt.StudentId__c, evt.JobId__c
  }
}
```

Test: use Execute Anonymous to run the Apex publish code. Check subscriber behavior (e.g., a Trigger that creates a Task).

7.3 Remote Site Settings (when Named Credentials not used)

When to use: quick test callouts or older patterns — *less secure* than Named Credentials.

1. Setup → **Remote Site Settings** → **New Remote Site**.
    - **Remote Site Name:** mylms_api
    - **Remote Site URL:** https://api.mylms.com
    - **Description:** Allow callouts to LMS API
    - Save.



Use: In Apex callouts without Named Credential, you use the full https URL:

HttpRequest req = new HttpRequest();

```
req.setEndpoint('https://api.mylms.com/v1/courses');

req.setMethod('GET');

Http http = new Http();

HttpResponse res = http.send(req);
```

Recommendation: prefer **Named Credentials** — they centralize, secure, and make token refresh automatic.

Quick Testing checklist (do these after setup)

1. **Connected App:** verify Consumer Key & Secret are visible on the Connected App detail page. If you used Admin-approved, add your admin profile under **Manage Profiles**.
2. **Named Credential:** after creating + authenticating, run this sample from **Developer Console → Execute Anonymous**:

```
HttpRequest req = new HttpRequest();

req.setEndpoint('callout:LMS_NamedCred/v1/ping'); // replace with an endpoint that exists

req.setMethod('GET');

Http http = new Http();

HttpResponse res = http.send(req);

System.debug(res.getStatus() + ' ' + res.getBody());
```

3. **External Service:** build a simple Flow that calls one External Service action and run it.
4. **Platform Event:** execute the Apex publish snippet. Then check that your Subscriber Trigger executed (e.g., created a Task or wrote to debug log).
5. **Remote Site:** if using Remote Site, try a plain Apex callout to the external URL.

Phase 8: Data Management & Deployment

8.1 Data import — Data Import Wizard

Prepare: Students.csv example headers:

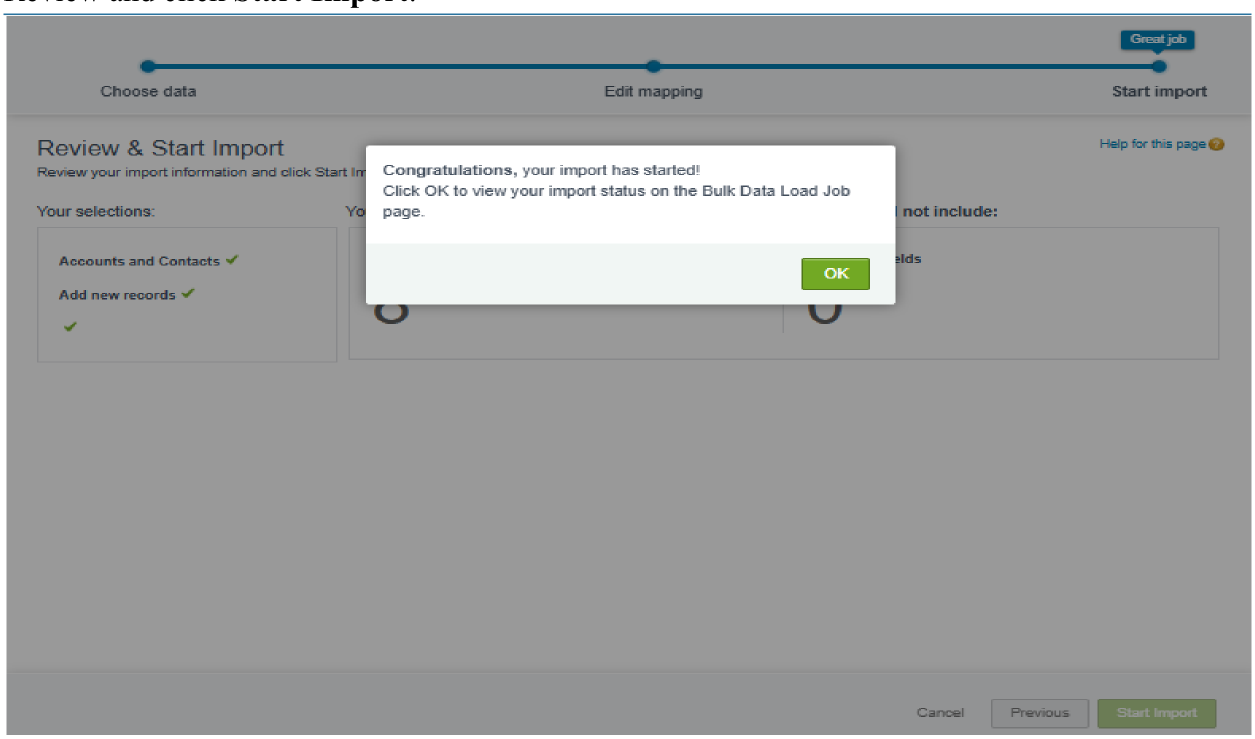
FirstName,LastName,Email,Student_ID__c,AccountName,GPA__c

Swayam,Tamrakar,swayam@example.com,STU001,XYZ Engineering College,8.2

Note: You can import Contacts and specify Account by **Account Name** (Data Import Wizard will try to match or create).

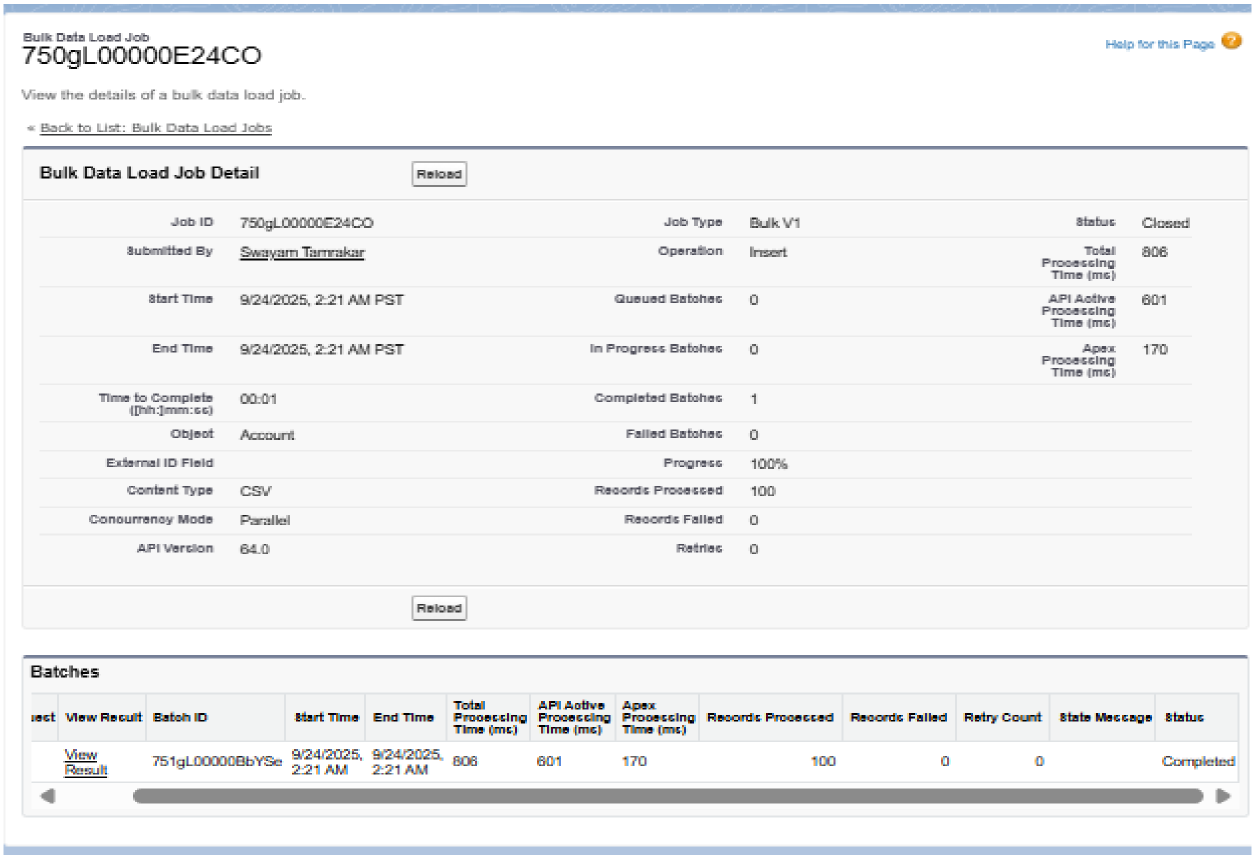
Steps

1. In Salesforce: click ⚙ → **Setup**.
2. Quick Find → **Data Import Wizard** → **Launch Wizard**.
3. Choose the object group (e.g., **Standard Objects** → Accounts and Contacts).
4. Select operation: **Add new records** (or Update existing if you include Salesforce IDs).
5. Choose CSV file (Students.csv) and follow the mapping screen: map CSV columns to Salesforce fields.
   - Map AccountName → Account Name (if importing Contacts tied to Account).
6. Review and click **Start Import**.



7. After run completes, check the email notification and import results page for success/error counts.

8. Fix errors (download error CSV), correct CSV, re-import failed rows.



Bulk Data Load Job
750gL00000E24CO

View the details of a bulk data load job.

« Back to List: Bulk Data Load Jobs

Help for this Page

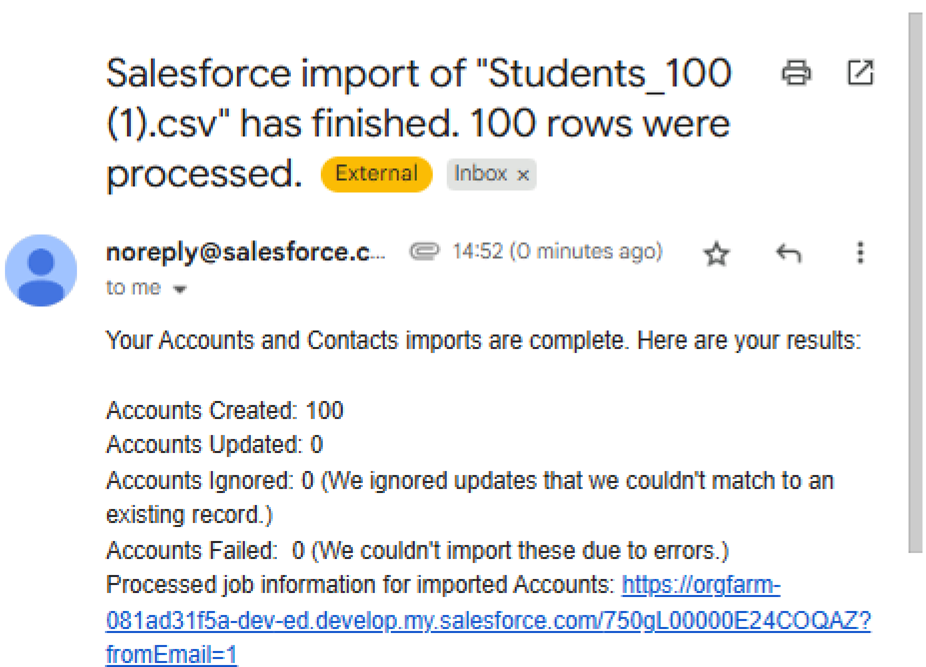
Bulk Data Load Job Detail Reload

Job ID	750gL00000E24CO	Job Type	Bulk V1	Status	Closed
Submitted By	Swayam Tamrakar	Operation	Insert	Total Processing Time (ms)	806
Start Time	9/24/2025, 2:21 AM PST	Queued Batches	0	API Active Processing Time (ms)	601
End Time	9/24/2025, 2:21 AM PST	In Progress Batches	0	Apex Processing Time (ms)	170
Time to Complete ([hh:]mm:ss)	00:01	Completed Batches	1		
Object	Account	Failed Batches	0		
External ID Field		Progress	100%		
Content Type	CSV	Records Processed	100		
Concurrency Mode	Parallel	Records Failed	0		
API Version	64.0	Retries	0		

Reload

Batches

ject	View Result	Batch ID	Start Time	End Time	Total Processing Time (ms)	API Active Processing Time (ms)	Apex Processing Time (ms)	Records Processed	Records Failed	Retry Count	State Message	Status
	View Result	751gL00000BbYSe	9/24/2025, 2:21 AM	9/24/2025, 2:21 AM	806	601	170	100	0	0		Completed



Salesforce import of "Students_100 (1).csv" has finished. 100 rows were processed. External Inbox ×

noreply@salesforce.c... 14:52 (0 minutes ago)
to me

Your Accounts and Contacts imports are complete. Here are your results:

Accounts Created: 100
Accounts Updated: 0
Accounts Ignored: 0 (We ignored updates that we couldn't match to an existing record.)
Accounts Failed: 0 (We couldn't import these due to errors.)
Processed job information for imported Accounts: https://orgfarm-081ad31f5a-dev-ed.develop.my.salesforce.com/750gL00000E24COQAZ?fromEmail=1

Tips

- For initial data load, import Accounts first (Colleges/Companies), then Contacts (Students) referencing accounts.
- If you plan to use Data Loader later, you can import basic data with the Wizard and use Data Loader for complex objects.

8.1 Data import — Data Loader (bulk, lookups, upsert)

Best for: bulk loads, custom objects, junction objects, upsert using External IDs.

Install & Login

1. Install Data Loader and open it.
2. Click **Login** → choose **Username-Password** (enter security token if required) OR use **OAuth** flow (recommended).
    - For Developer Org, if using username&password, append your security token to password.

Common operations: Insert, Update, Upsert, Export.

Example: Load Student_Skill__c (junction)

Problem: Student_Skill__c has lookup to Contact and Skill. You likely don't have Salesforce Ids initially.

Use IDs

1. Export Contacts (after Contacts import) to get Contact IDs: Setup → Data Loader → Export → select Contact → export Id, Email, Student_ID__c to CSV.
2. Prepare Student_Skill.csv:

Student__c,Skill__c,Proficiency__c

0033t00002ABCDe, a0H3t00000Skill1,Intermediate

0033t00002FGHIj, a0H3t00000Skill2,Beginner

3. In Data Loader: **Insert** → select Student_Skill__c → choose CSV → Map fields (or auto map) → Finish → check success/error files.

Use External ID upsert (no Salesforce IDs)

1. Create an External ID field on Contact:
   - Setup → Object Manager → Contact → Fields & Relationships → **New** → Text → Student_External_Id__c and check **External ID**.
2. Ensure Skill__c has External ID (or create Skill_Code__c External ID).
3. Prepare Student_Skill_Upsert.csv:

Student_External_Id__c,Skill_Code__c,Proficiency__c

STU001,SKL_JAVA,Intermediate

STU002,SKL_PYTHON,Advanced

4. In Data Loader: choose **Upsert** → select Student_Skill__c → for the lookup fields use the ExternalId mapping: map Student_External_Id__c → Student__r.Student_External_Id__c and Skill_Code__c → Skill__r.Skill_Code__c (Data Loader supports parent external id mapping using relationship notation).
5. Run import and review success/error files.

Tips

- Always **export a small sample** first to confirm mappings.
- Keep the **success.csv** and **error.csv** produced by Data Loader — they tell you row-level outcome and error messages.
- For large loads split into batches < 10k if necessary. Use Bulk API mode in Data Loader for huge data.

8.2 Duplicate Rules — create matching & duplicate rules

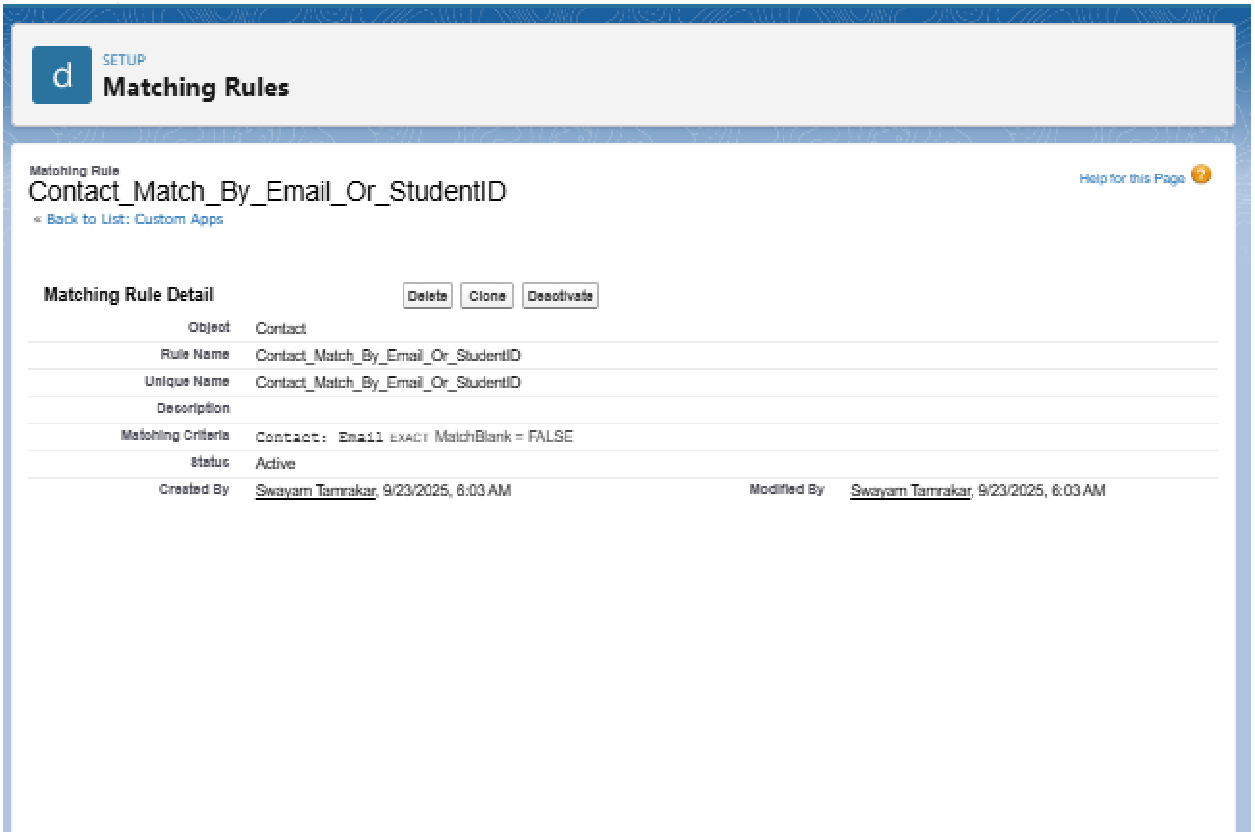
Goal: Prevent or alert on duplicate Student (Contact) records by Email or Student_ID__c.

Create Matching Rule

1. Setup → Quick Find → **Matching Rules** → **New Rule**.
   - Select **Contact** as Object.
   - Rule Name: Contact_Match_By_Email_Or_StudentID.
   - Add Matching Criteria:
     - Field: Email Match Method: Exact

- OR add Student_ID__c Match Method: Exact (if you have Student_ID__c).

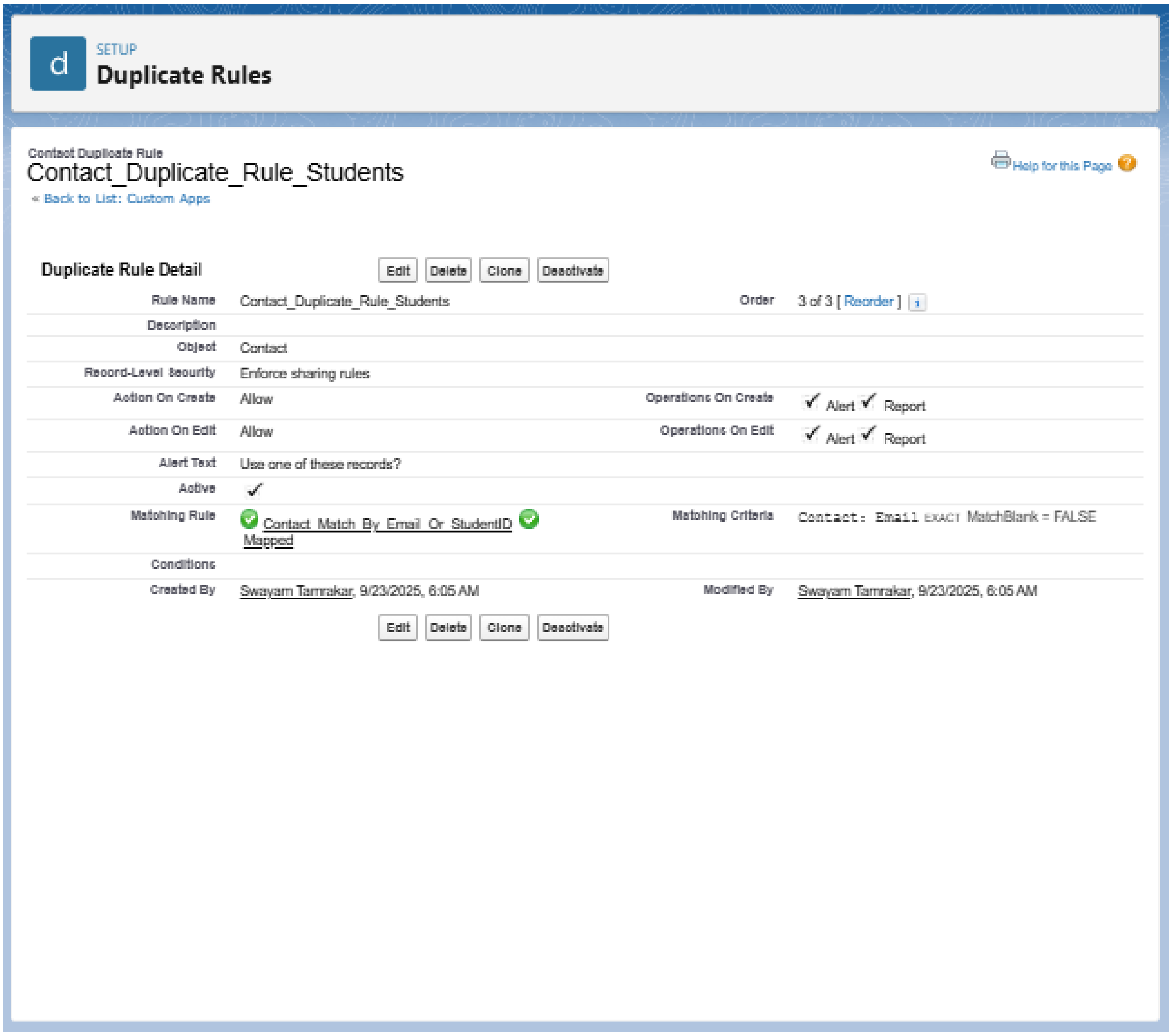
2. Save and **Activate** the matching rule (must be active to use).



Create Duplicate Rule

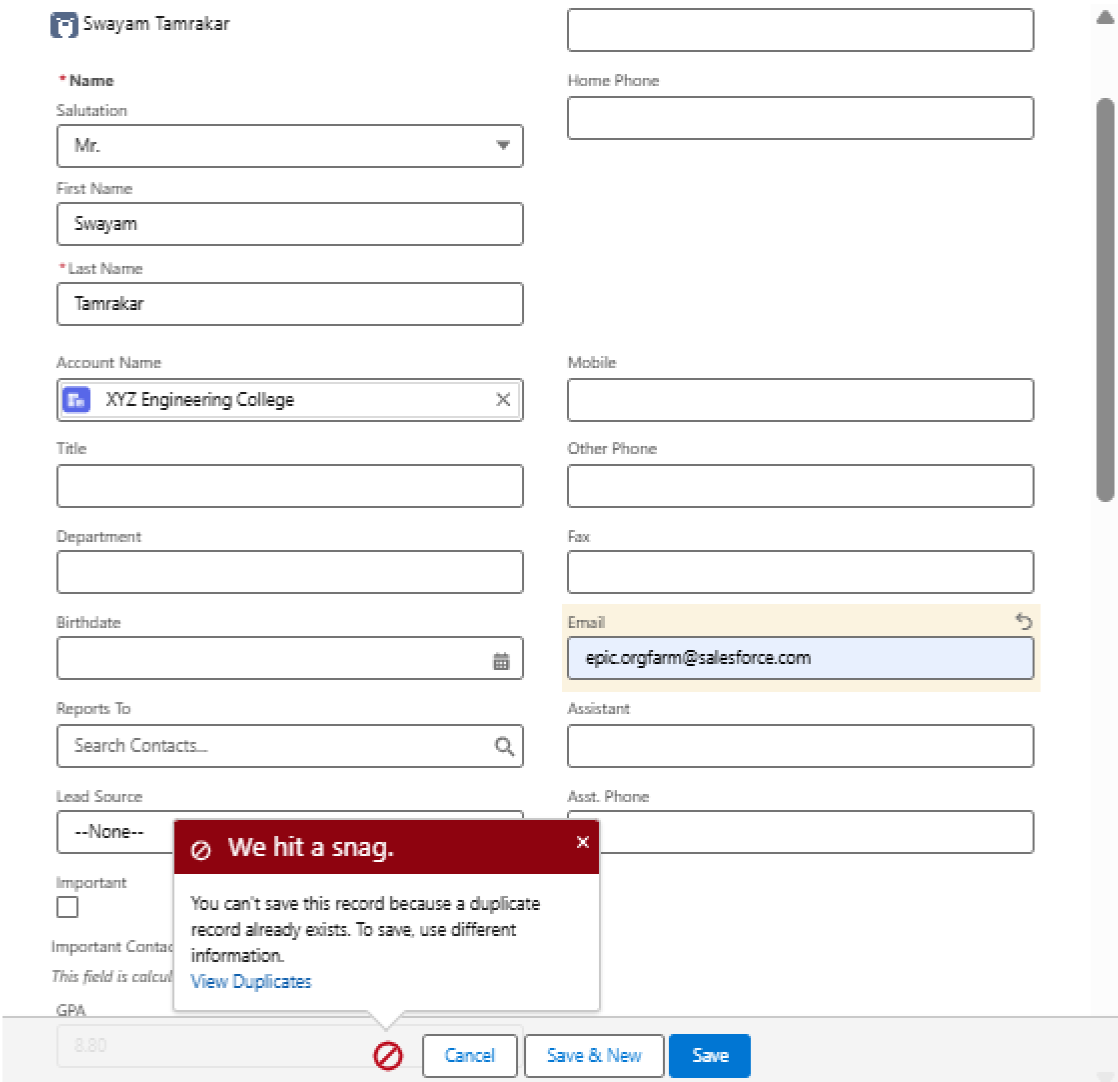
1. Setup → Quick Find → **Duplicate Rules** → **New Rule** → choose **Contact**.
2. Rule Name: Contact_Duplicate_Rule_Students.
3. Under **Matching Rule**, select the matching rule you created.
4. Action on create/edit:
   - **Alert** (allow save but show alert to user) or **Block** (prevent save) — choose based on policy.
5. Under **Actions**, specify what to do on create/edit — you can create a report, allow users to bypass with reason, or block.

6. Save and **Activate** the duplicate rule.



Test

- Attempt to create a Contact with same email or Student_ID__c — verify the behavior (alert or block).

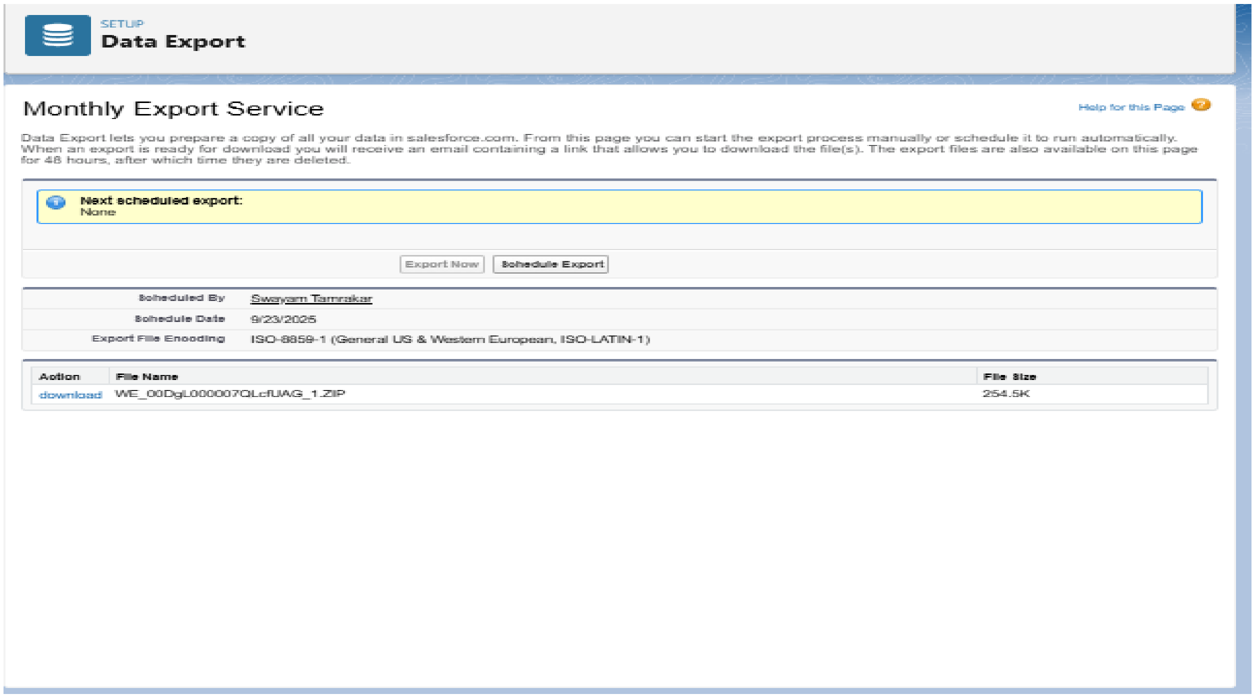


8.3 Backups — Data Export (manual & scheduled)

Manual Export (Developer Org)

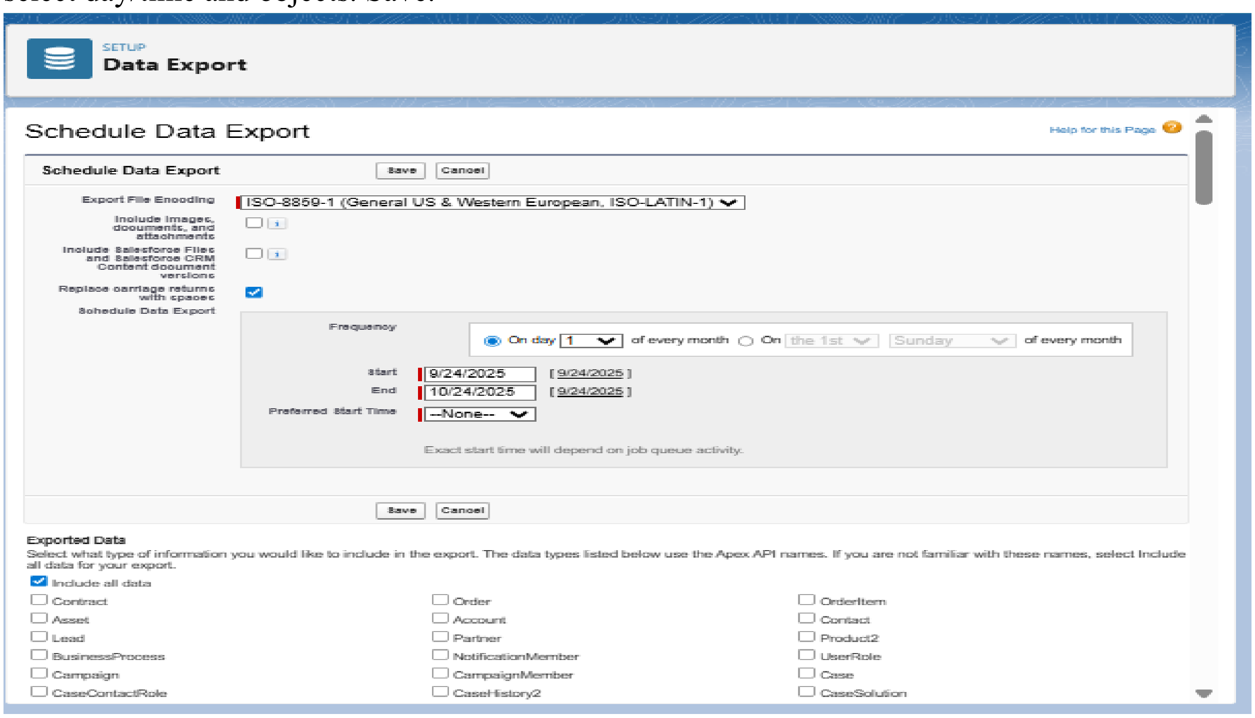
1. Setup → Quick Find → **Data Export** → Click **Export Now**.
2. Choose objects to export (or select **Include all data**). For faster test, select Accounts, Contacts, Internship__c, Student_Skill__c, etc.

3. Click **Start Export**. Salesforce prepares a ZIP file and sends email when available. Download the ZIP and store it securely.



Scheduled Export (if available in your edition)

1. On **Data Export** page → **Schedule Export** (if enabled): choose **Weekly** or **Monthly**, select day/time and objects. Save.



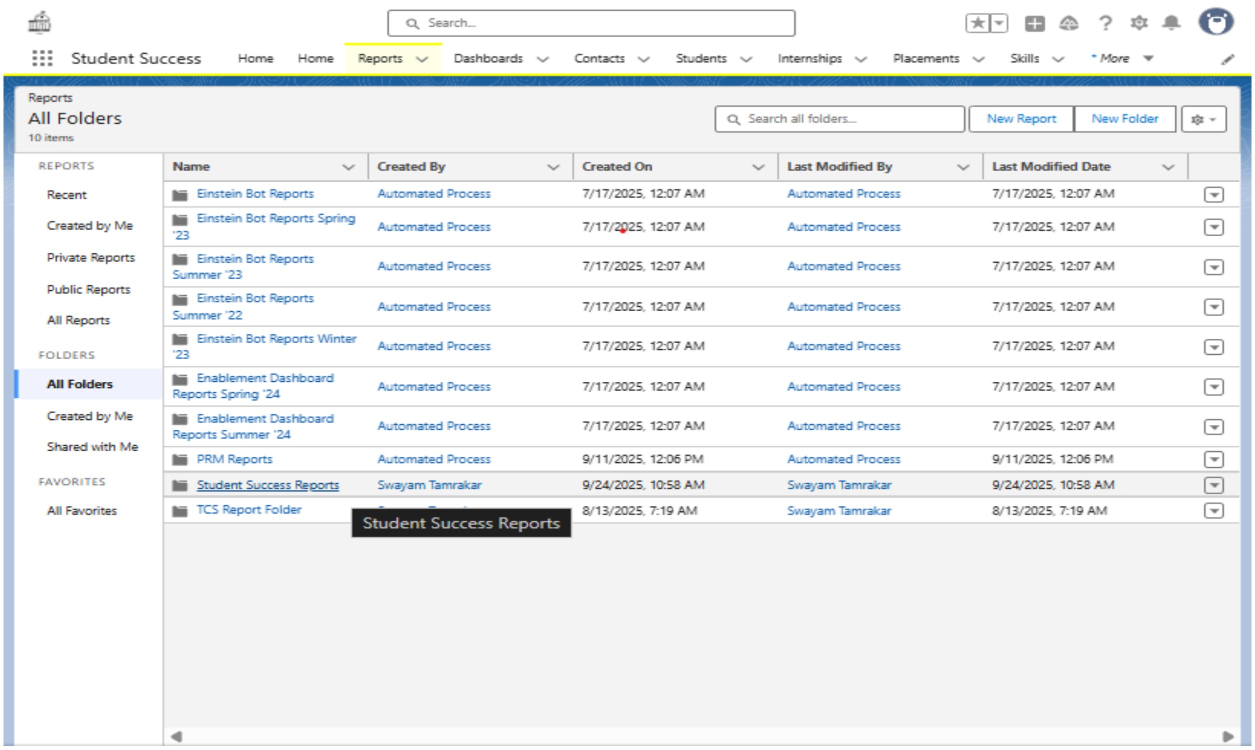
Tip: Always download and keep backup before any mass delete or destructive deploy.

Phase 9: Reporting, Dashboards & Security Review

9.1. Create a Report Folder

1. App Launcher → **Reports** → click **All Folders** (left pane).
2. Click **New Folder**.
    - Name: Student Success Reports
3. Click **Save**.

Why: keeps reports & dashboards secure and easy to share.



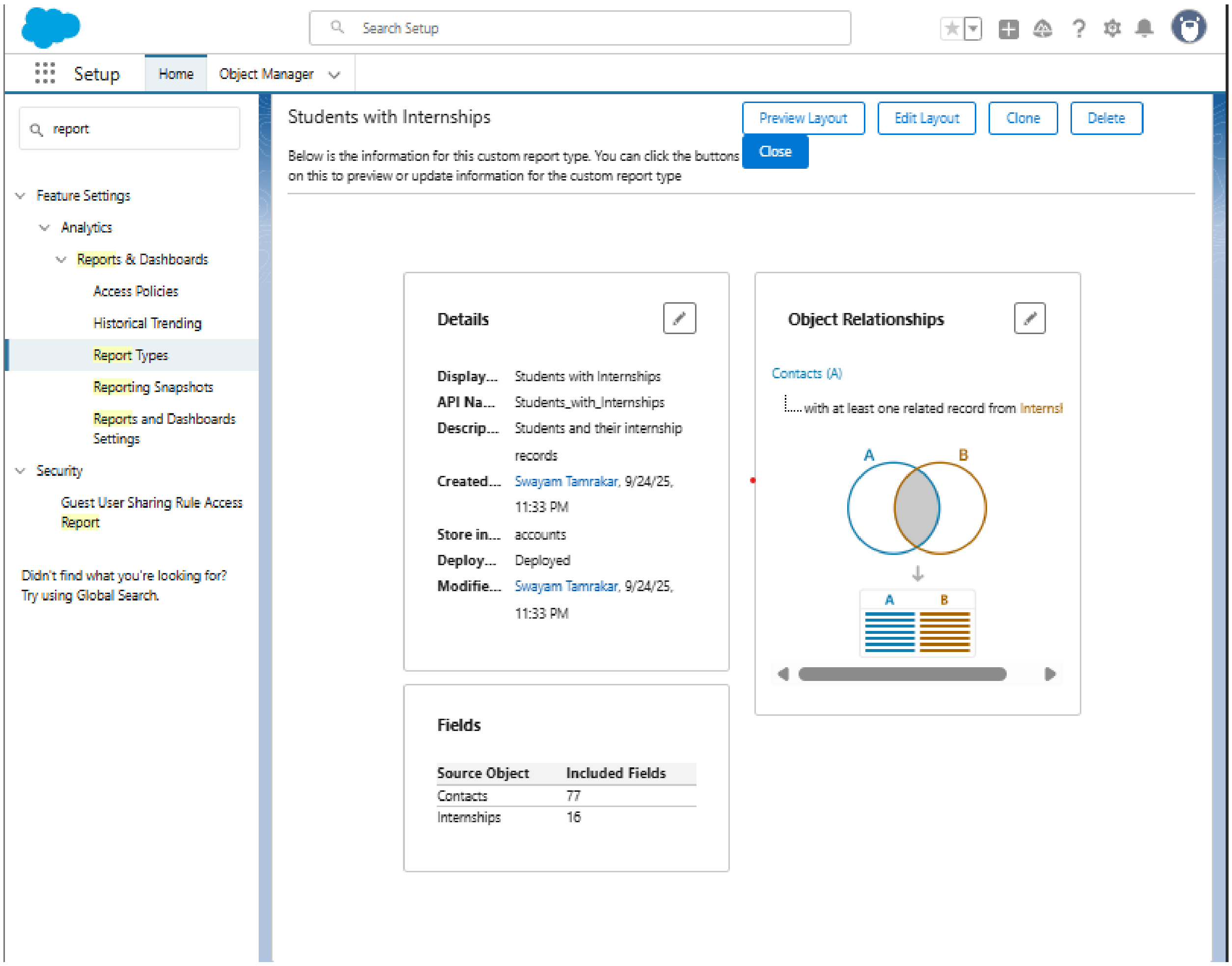
9.2. Create Custom Report Types

Contacts with Internships

1. Setup (⚙) → **Quick Find** → **Report Types** → **New Custom Report Type**.
2. Fill:
    - **Primary Object**: Contacts
    - **Report Type Label**: Students with Internships
    - **Report Type Name**: Students_with_Internships
    - **Category**: Contacts & Accounts (or Other Reports)

- **Deployment Status**: **Deployed**
- Description: Students and their internship records

3. Click **Next** → On the next screen click **Click to relate another object** and pick your custom object Internship__c.
4. Choose relationship: **Each "A" record must have at least one "B" record** (choose *with or without* if you want students even without internships).
5. Save.

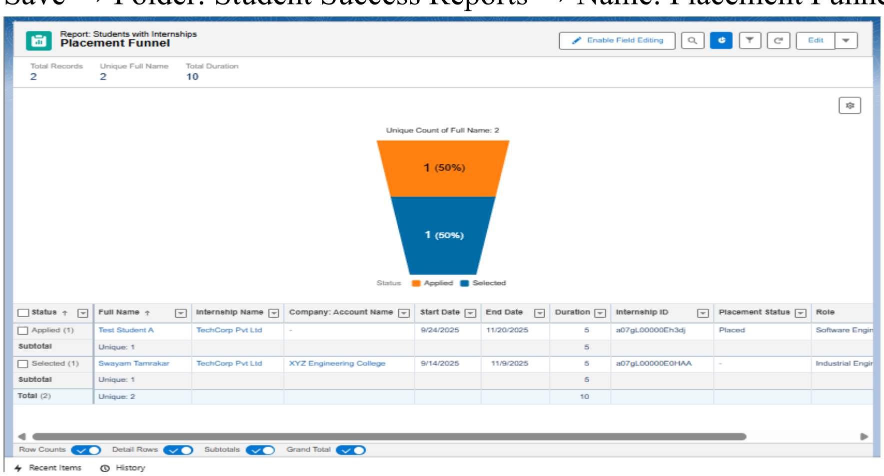


9.3. Create Reports

A — Placement Funnel

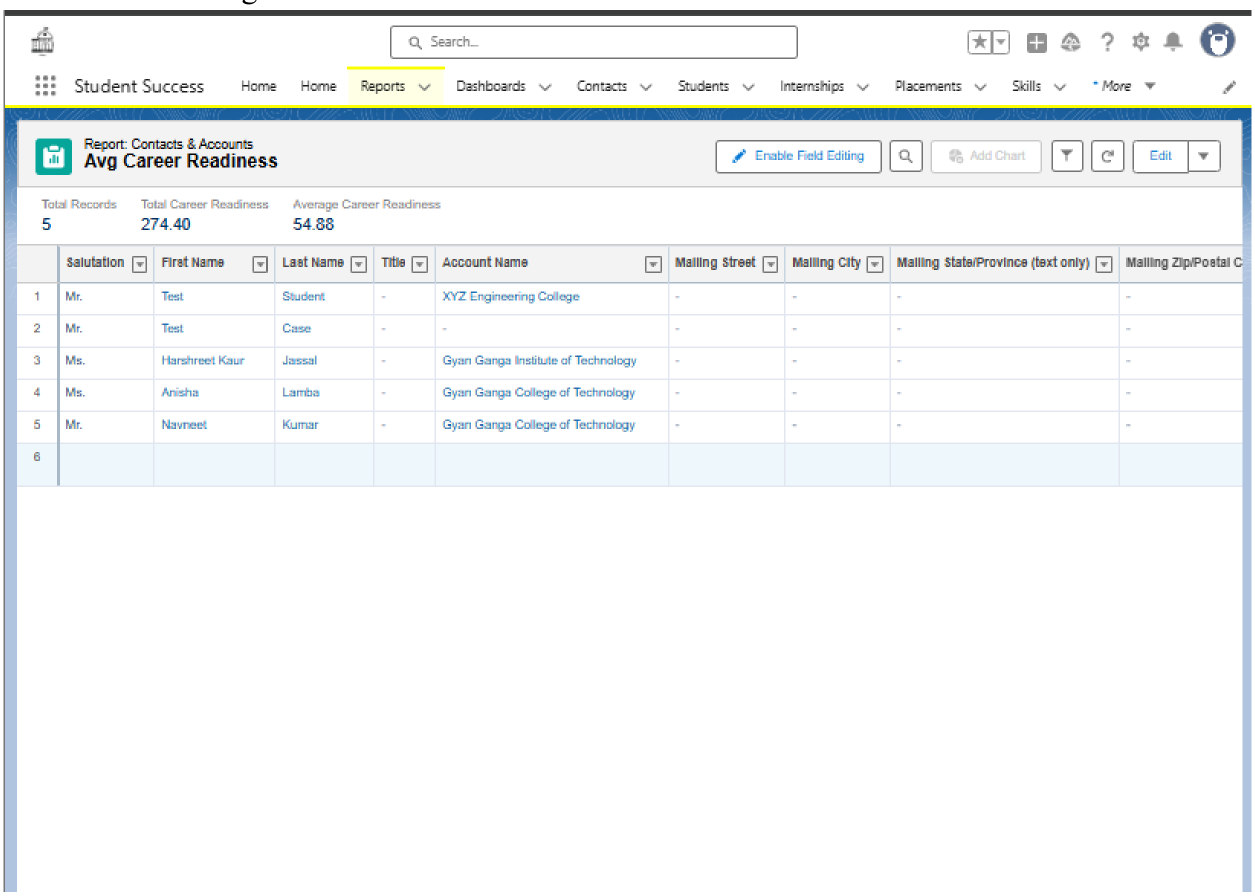
App Launcher → **Reports** → **New Report**.

1. In the report type chooser select: **Students with Internships** (the custom report type you created). Click **Continue**.
2. Filters:
   - **Show**: All students (or specify Record Type = Student)
   - **Date Range**: All Time (or internship Start Date filter)
   - Add filter: Internship__c.Status__c (or Offer_Status__c) **is not empty**.
3. Grouping:
   - Drag the Internship: Status (Offer_Status__c) to **Group Rows**. This will create rows per status (Applied, Interviewing, Selected, Completed, Rejected).
4. Columns: keep Student Name, Company (Account), Start Date.
5. Summaries:
   - Click the dropdown on Student Name column → **Summarize** → select **Count**. This gives count of students per status.
6. Add Chart (within report):
   - Click the chart icon (top right of the report).
   - Chart Type: **Funnel**
7. Save → Folder: Student Success Reports → Name: Placement Funnel.



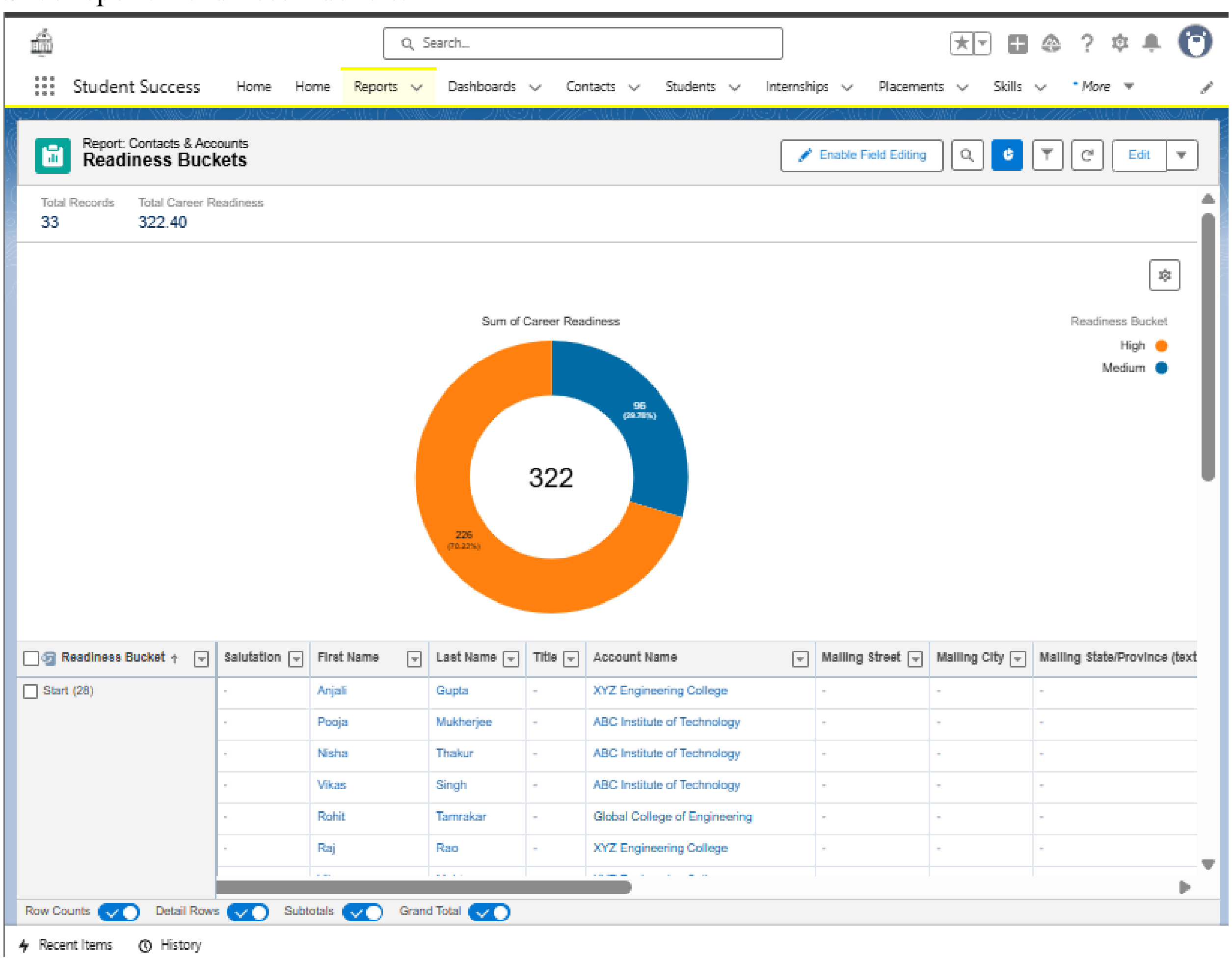
B — Average Career Readiness

1. New Report → choose **Contacts** (or custom report type that includes Career_Readiness__c).
2. Filters: Record Type = Student.
3. Add column: Career_Readiness__c (numeric).
4. Click column dropdown → **Summarize** → choose **Average**. (This computes avg readiness.)
5. Remove row grouping (leave as tabular or summary with no grouping).
6. Save → Name: Avg Career Readiness.



9.4. Use Bucket Fields to create Readiness Buckets

1. Edit Avg Career Readiness report or create a new Contacts report with Career_Readiness__c.
2. In the Columns header, click ▼ → **Add Bucket Column**.
   - Bucket Field Source: Career_Readiness__c.
   - Create buckets:
     - Low (0-30) → 0 to 30
     - Medium (31-60) → 31 to 60
     - High (61-100) → 61 to 100
3. Save the bucket as Readiness Bucket.
4. Add a chart: Chart Type **Donut**, Grouping: Readiness Bucket, Metric: Record Count.
5. Save report: Readiness Buckets.

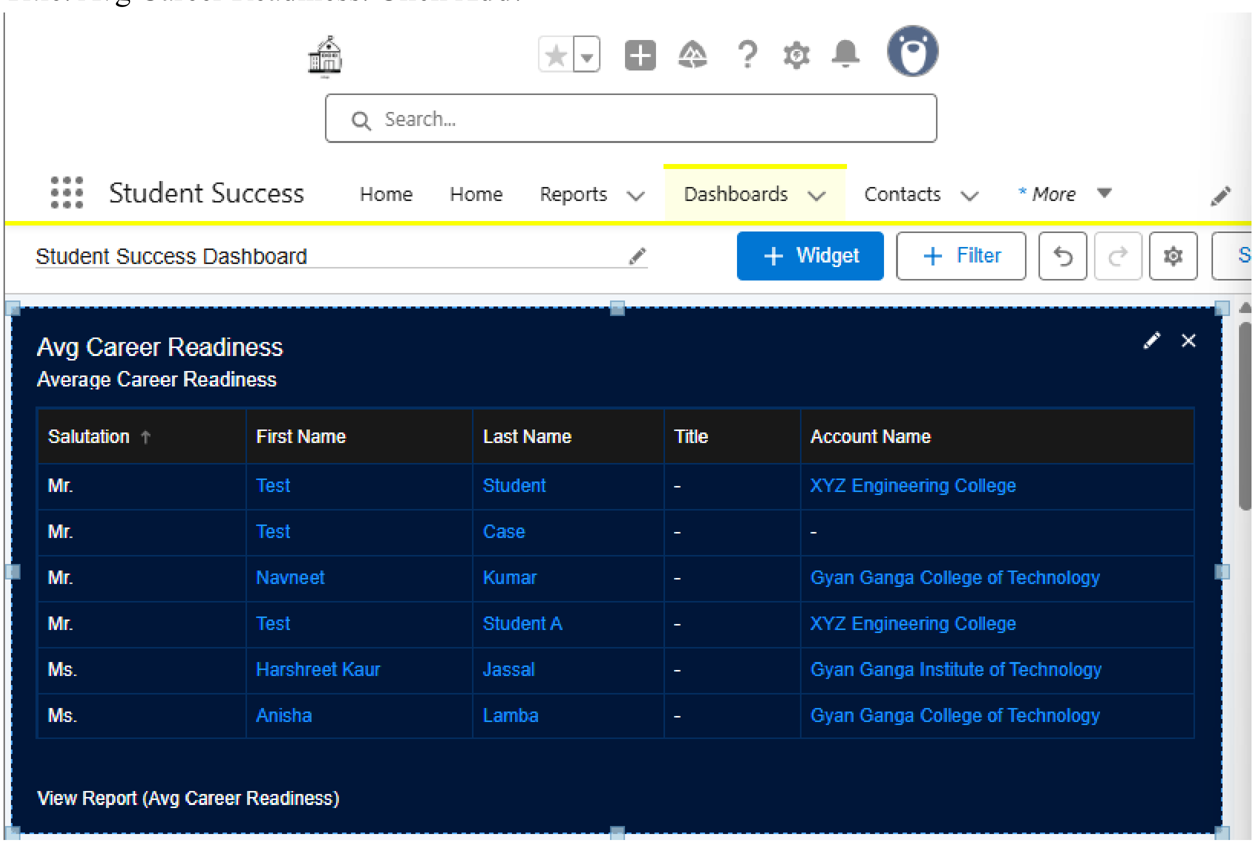


9.5. Build Dashboard & Add Components

1. App Launcher → **Dashboards** → **New Dashboard**.
    - Dashboard Name: Student Success Dashboard
    - Folder: Student Success Reports
    - Click **Create**.

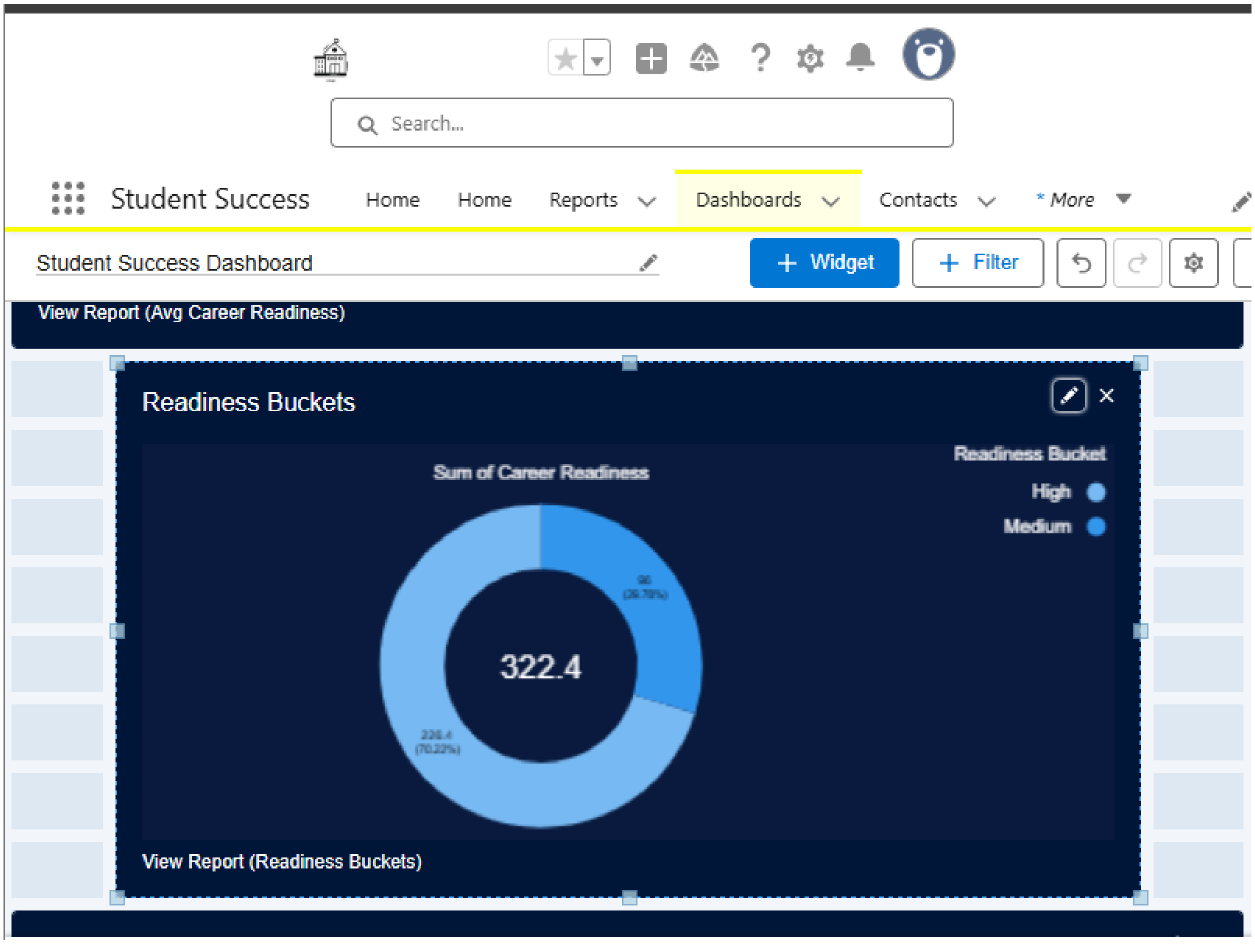
A — Add Table: Average Career Readiness

1. Click + **Component**.
2. Select Report: Avg Career Readiness (the report with Average summary).
3. Select **Table** component type.
4. Data: choose Metric = Average of Career_Readiness__c.
5. Title: Avg Career Readiness. Click **Add**.



B — Add Donut: Readiness Buckets

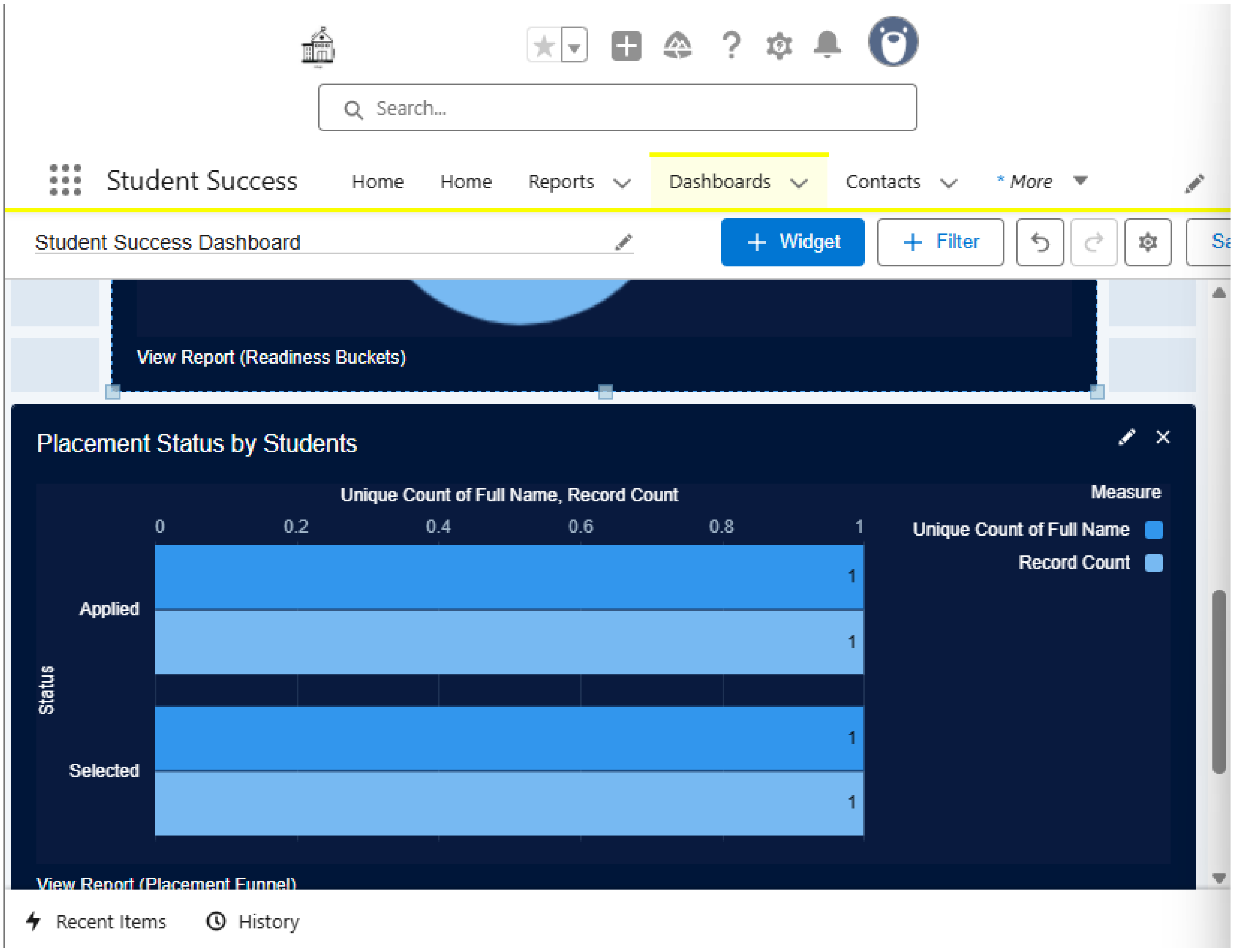
1. + **Component** → select report Readiness Buckets.
2. Component type: **Donut**.
3. Configure to show Readiness Bucket groups and set labels/legend.
4. Title: Readiness Distribution. Click **Add**.



C — Add Bar: Placement

1. + **Component** → select Top Skills by Department report.
2. Component type: **Bar Chart** (or horizontal bar).
3. In component properties— choose Top 10 by Record Count.

4. Title: Top Skills → Add.



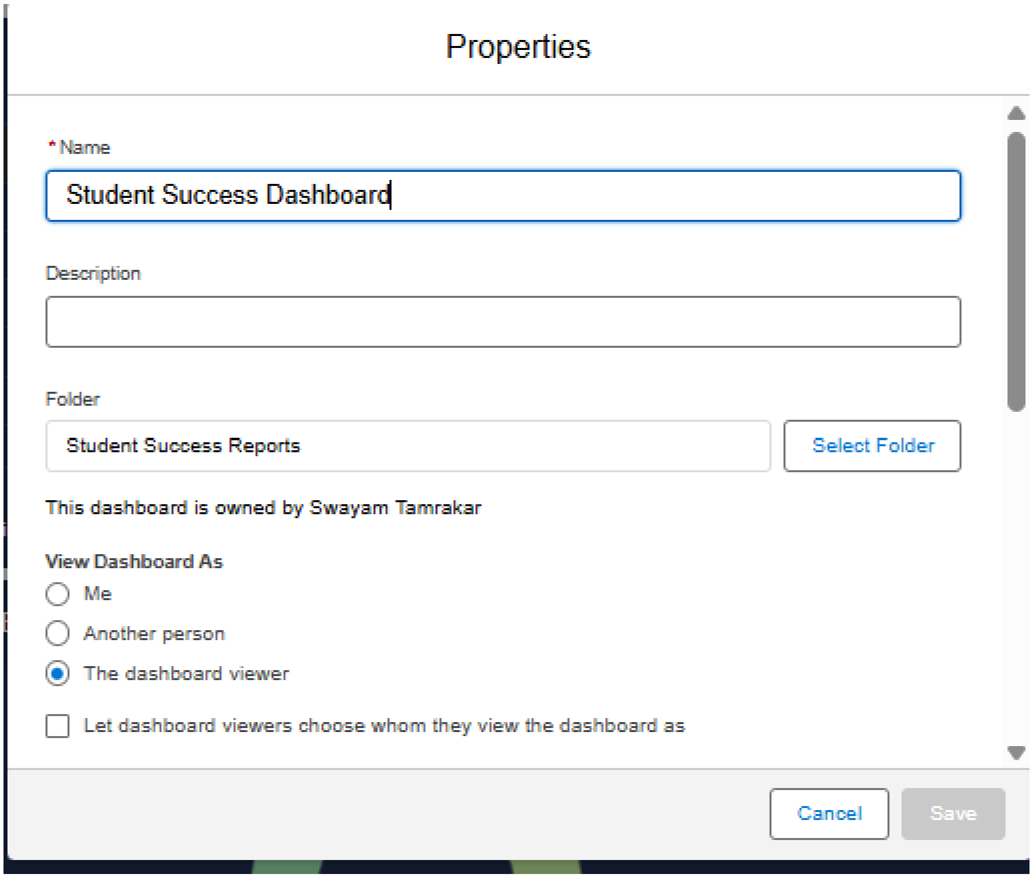
D — Arrange & Save

1. Drag components to arrange.
2. Click **Save** → **Done**.

Tip: in Dashboard component settings you can choose to display most recent data, enable "Show as table" or "Show labels".

9.6. Make Dashboard Dynamic

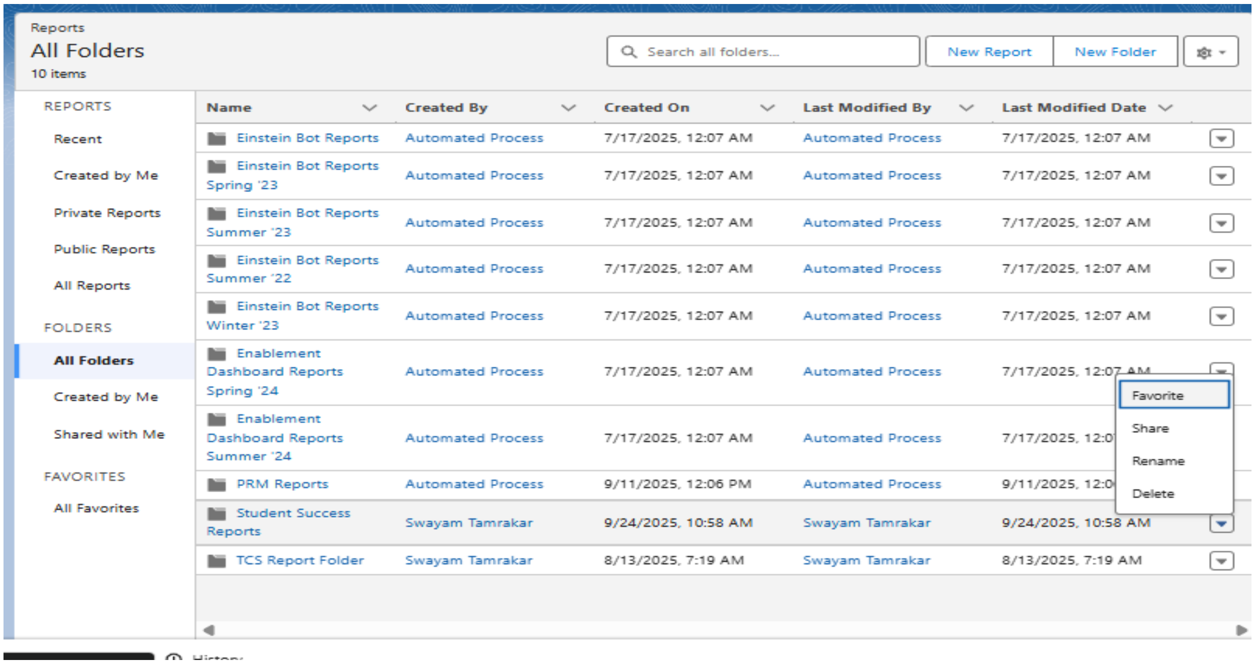
1. Open the Dashboard → click **Edit** → Click the gear (component run settings) or top-right **View Dashboard As**.
2. Choose **The dashboard viewer**Save.



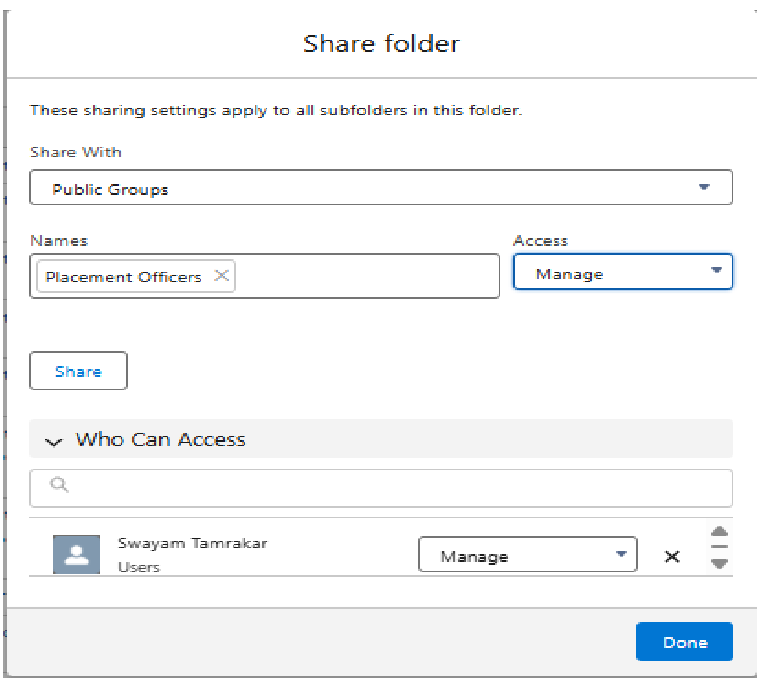
Note: dynamic dashboards are useful so Placement Officers see only their dept's students.

9.7. Sharing & Folder Access

1. In Reports app → locate Student Success Reports folder → Down arrow → **Share**.



2. Add Roles, Users, or Public Groups (e.g., Placement Officers) and set access level (Viewer/Editor). Save.



9.8. Security Review

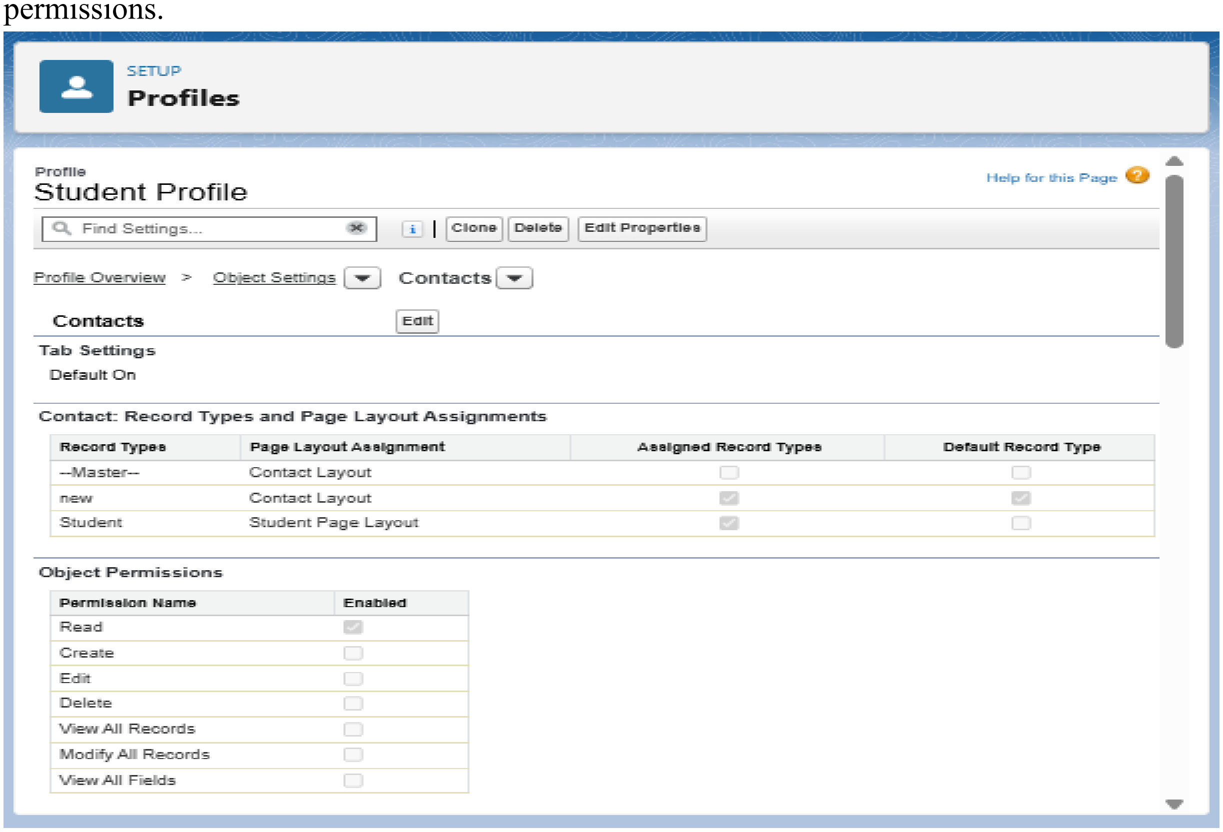
9.81 Field-Level Security (FLS) — hide SSN__c or other PII

1. Setup → **Object Manager** → **Contact** (or object with SSN field).
2. Fields & Relationships → find SSN__c → click field label → **Set Field-Level Security**.
3. Uncheck **Visible** for the Student_Profile (and any other profile that should not see SSN).
4. Save.
5. Also remove SSN__c from the Student Page Layout: Object Manager → Contact → Page Layouts → Edit Student Page Layout → remove SSN__c field → Save.

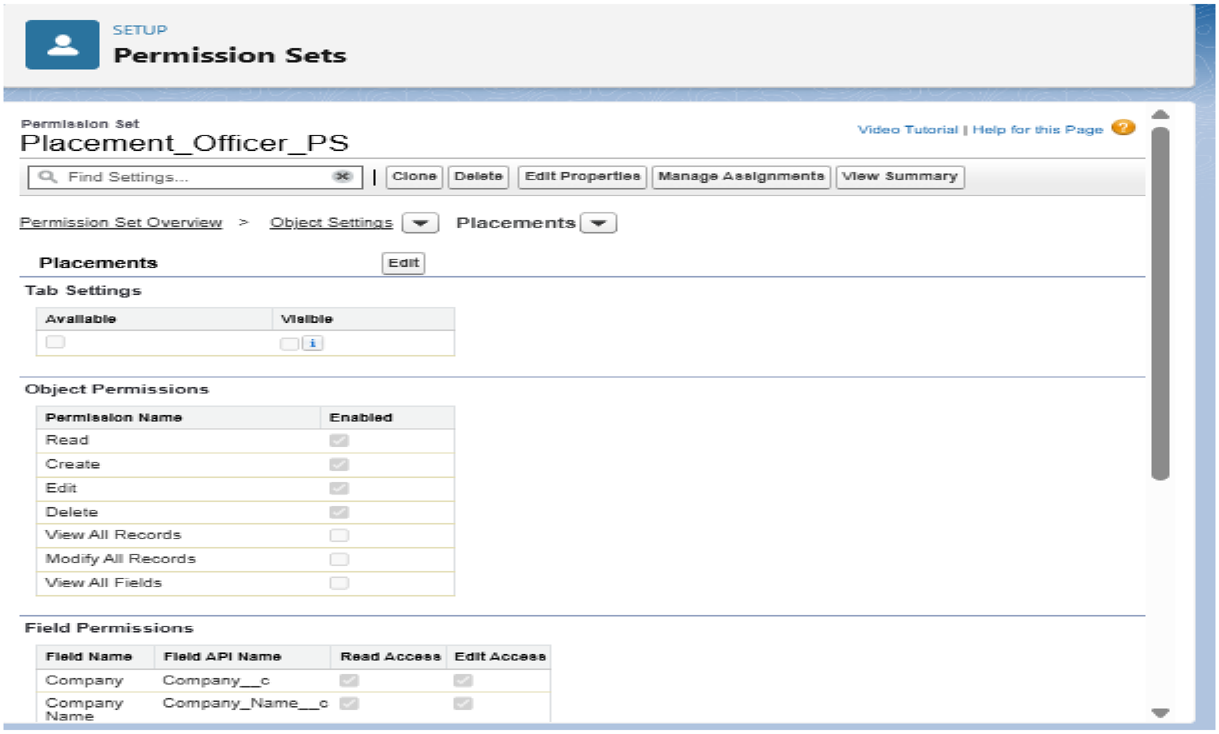
Test: Login as a Student user (or use "Login" as that user) and verify SSN is not visible.

9.82 Profiles & Permission Sets — least privilege

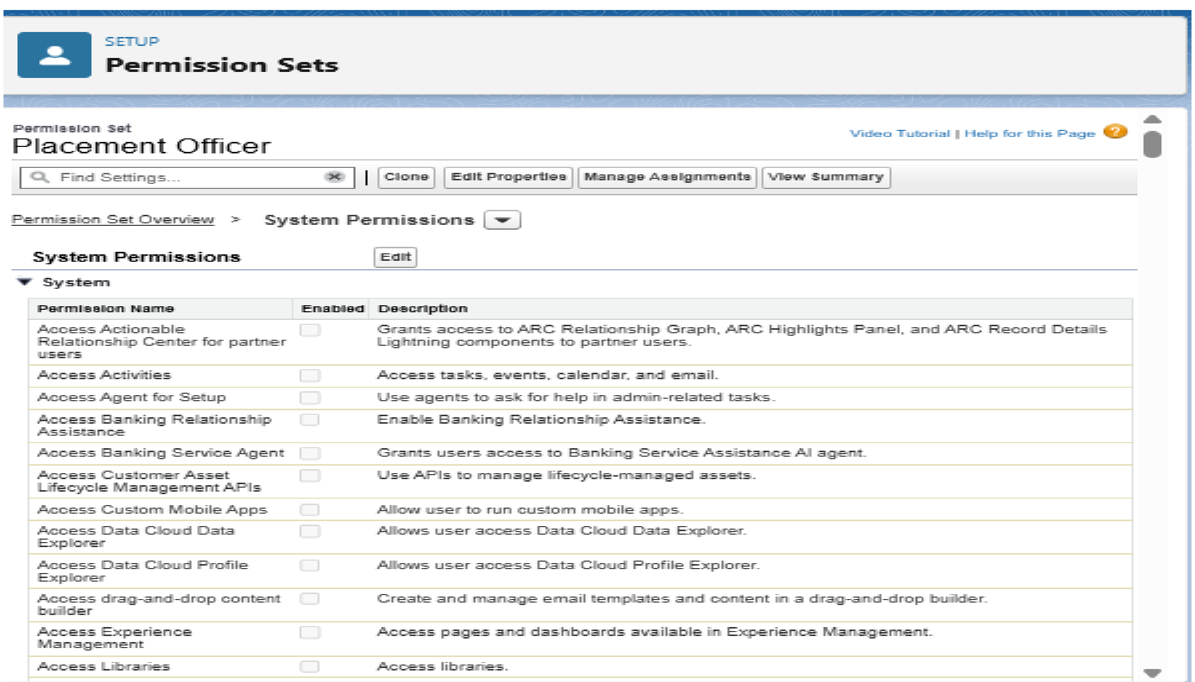
1. Setup → **Profiles** → open Student_Profile.
   - Click **Object Settings** → **Contacts** → Edit → ensure students have **Read** on their own contact only (use Sharing to limit edit). Remove unnecessary object permissions.



2. Setup → **Permission Sets** → create Placement_Officer_PS for extra rights (e.g., Export Reports).



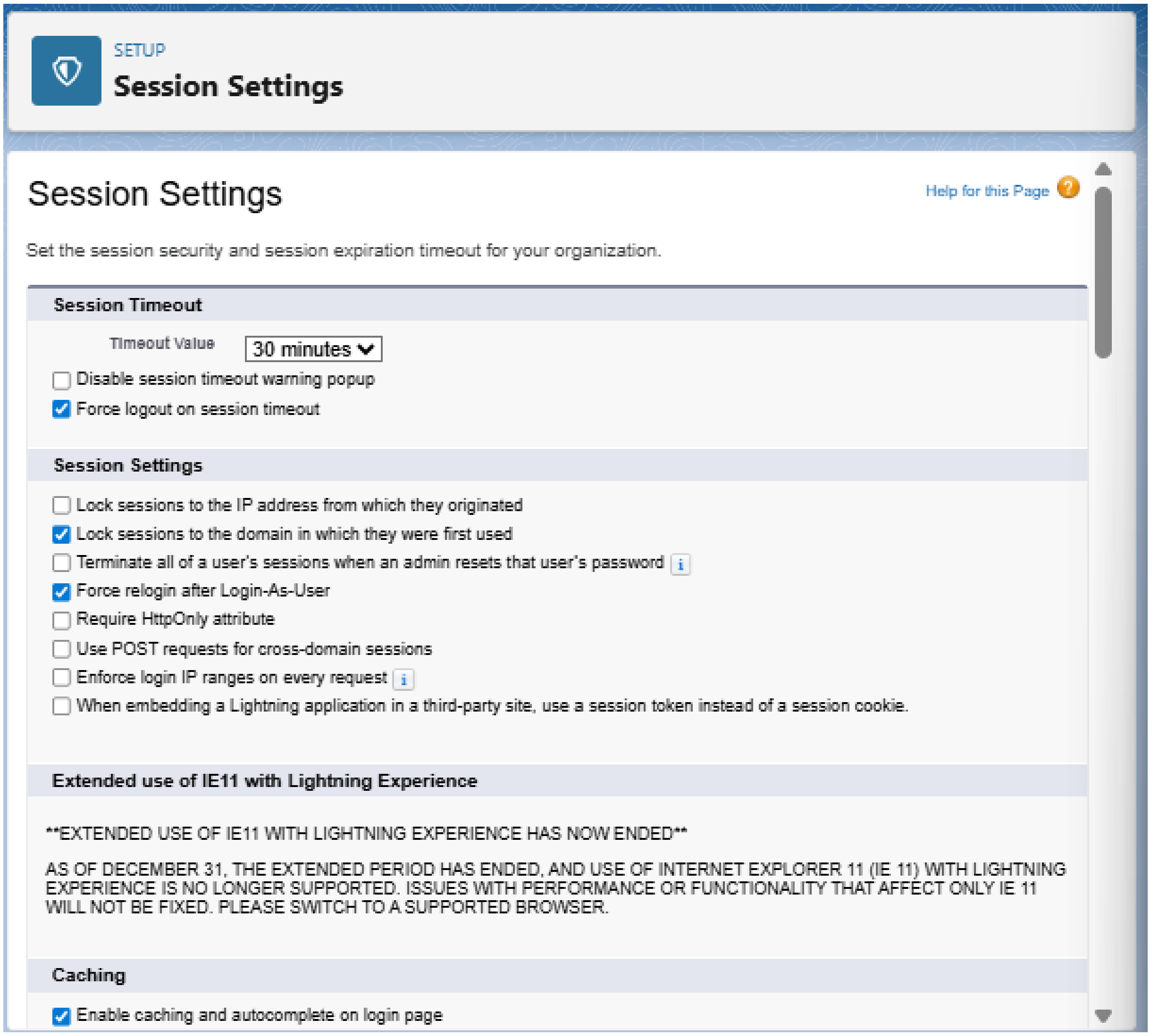
- Assign the permission set to users: Permission Set → Manage Assignments → Add Assignments.



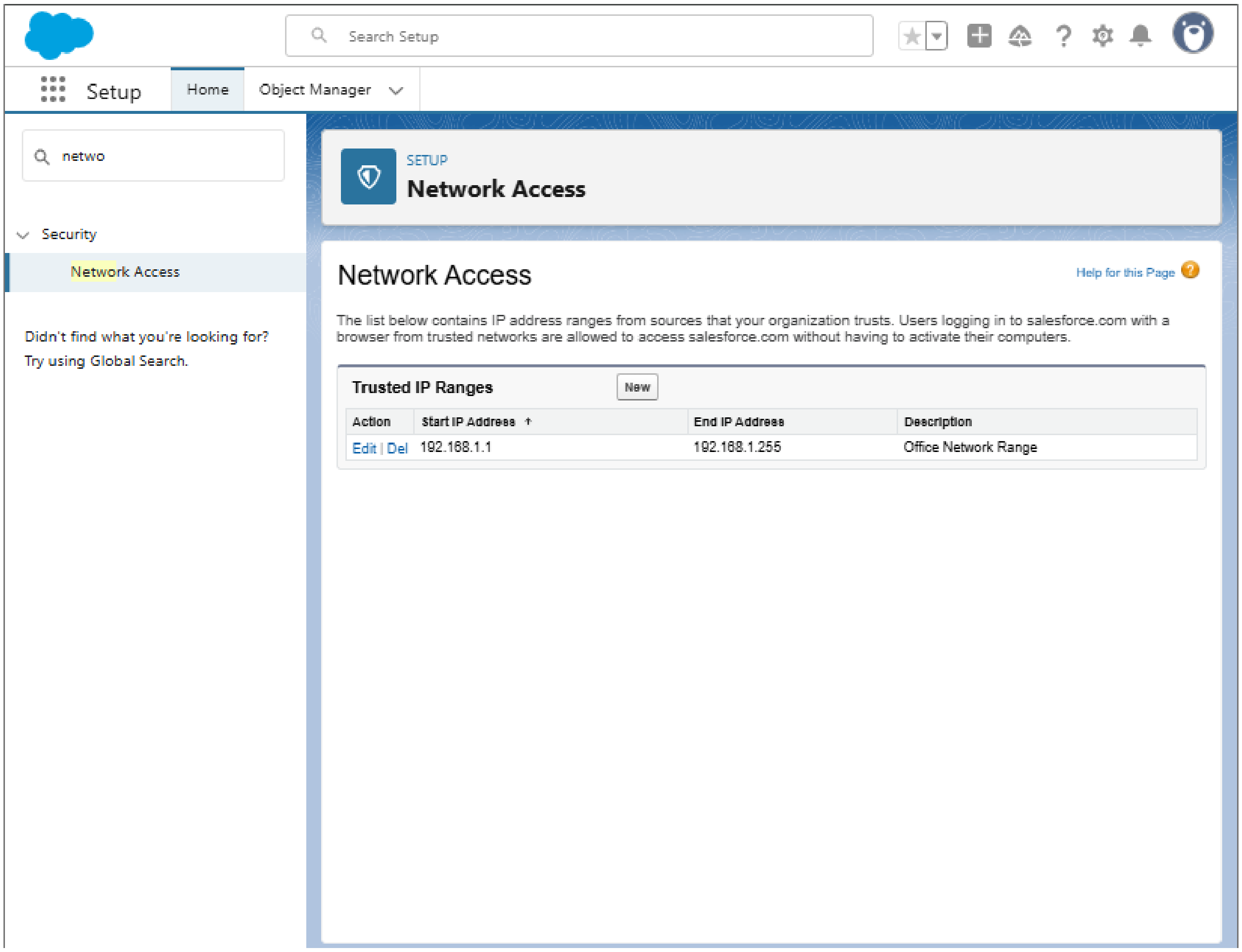
Test: Log in as Placement Officer and Student to validate each sees appropriate data.

9.83 Session Settings & Login IP Ranges

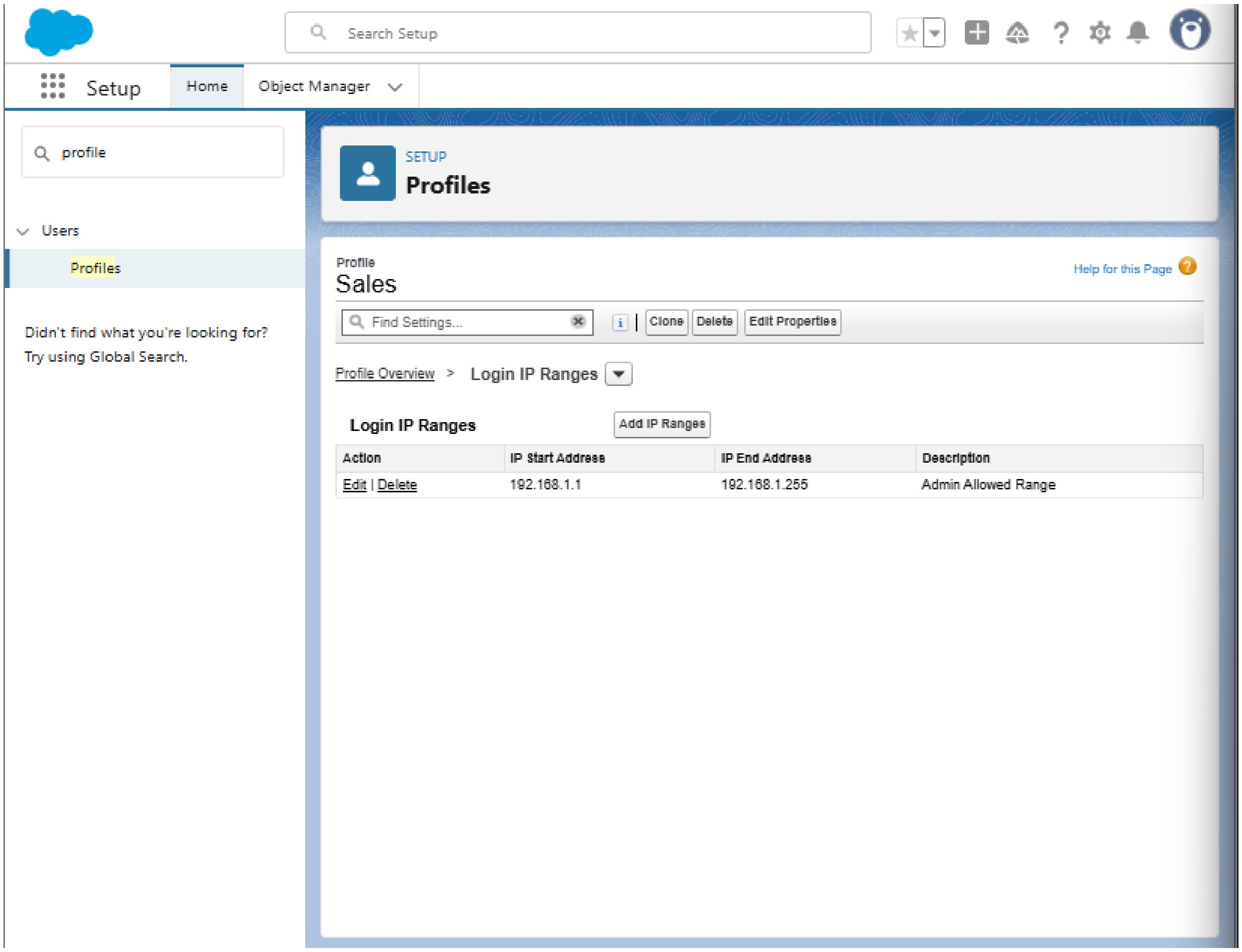
1. Setup → **Session Settings** → set **Session timeout** to reasonable value (e.g., 30 minutes). Save.



2. Setup → **Network Access** → **New** → add **Trusted IP Ranges** for your org (optional) — this prevents logins from outside ranges without verification.

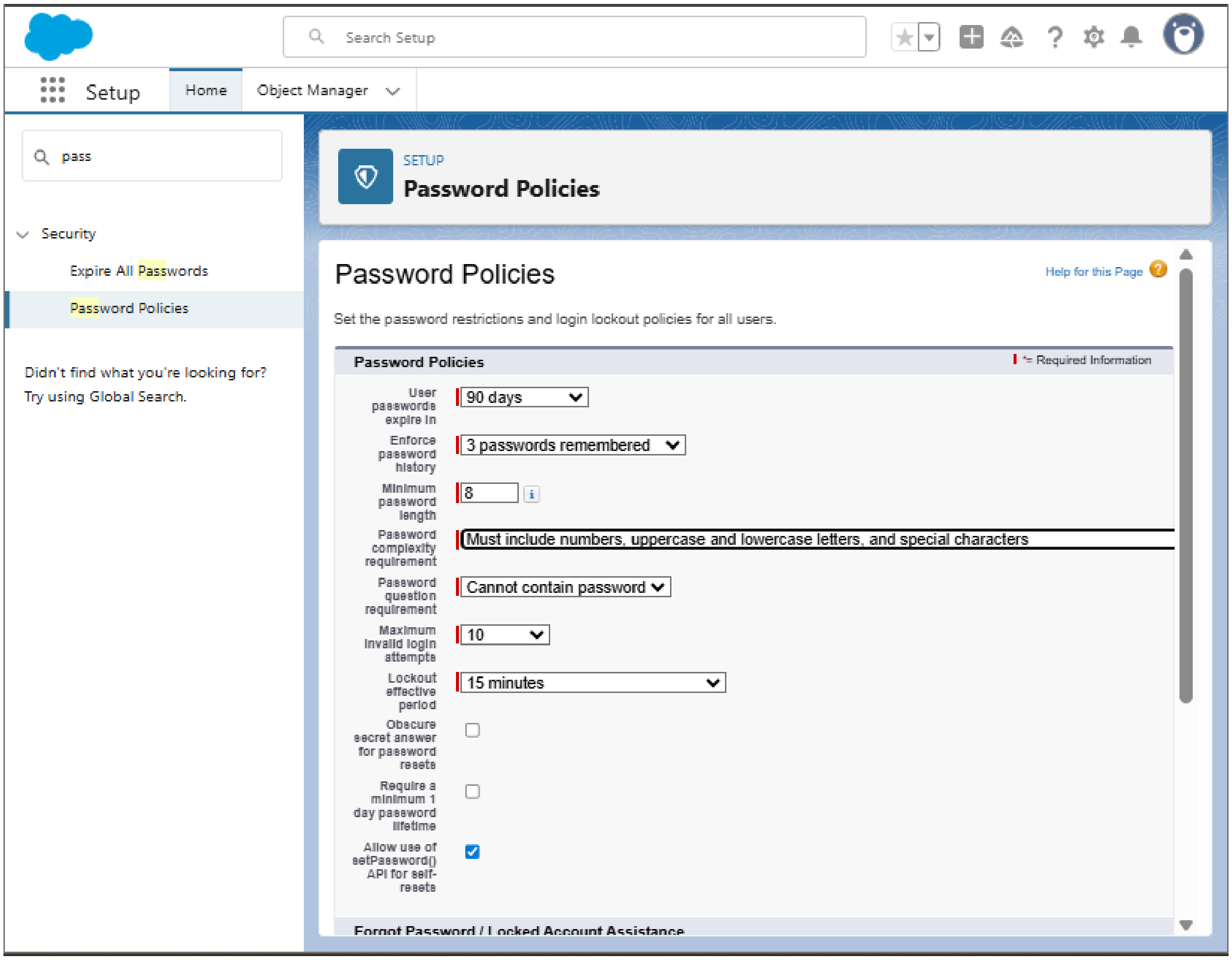


3. For admin profiles: Setup → **Profiles** → select System Administrator → **Login IP Ranges** → Add allowed IPs to restrict admin console access.



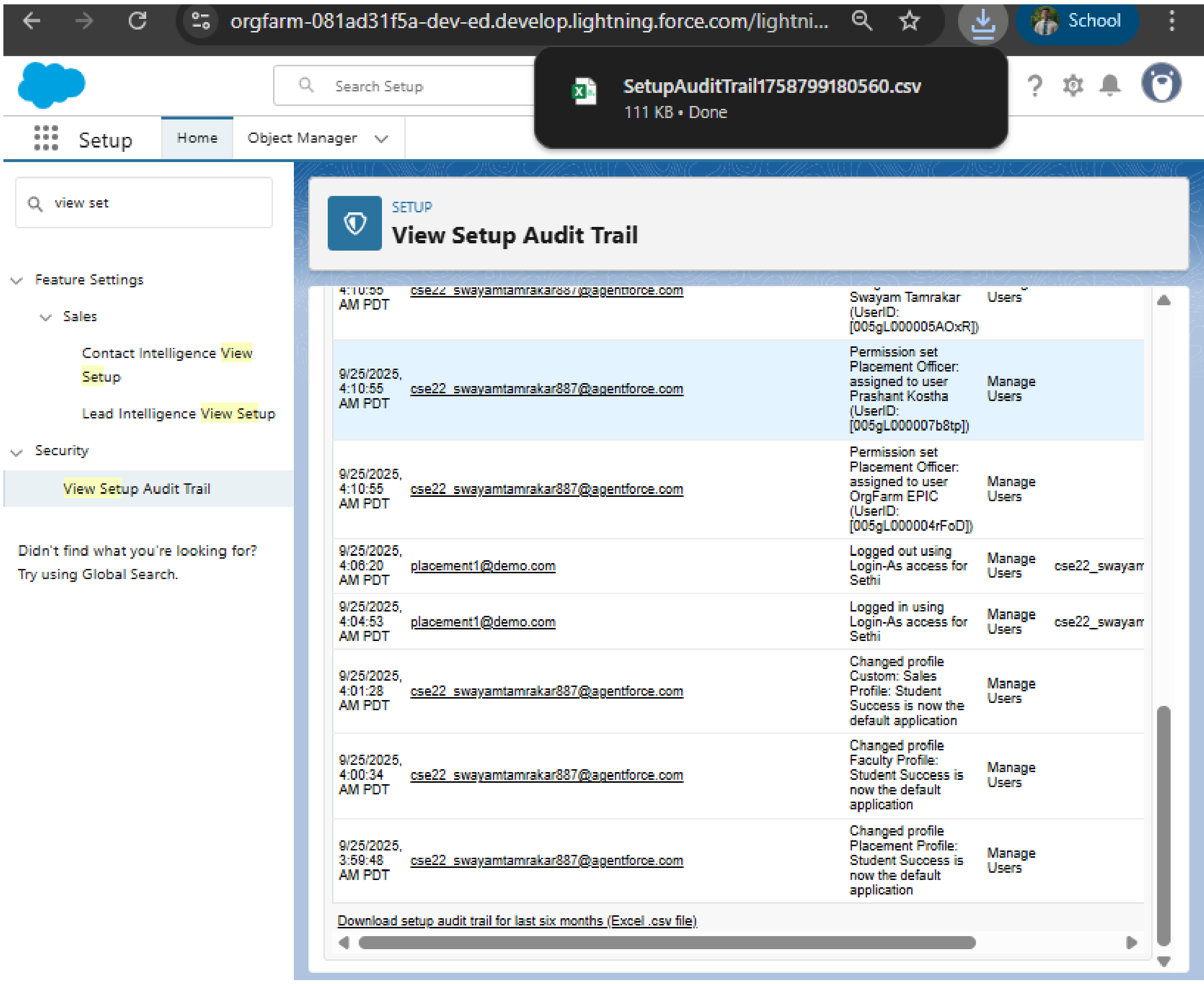
9.84 Password Policies

1. Setup → **Password Policies** (under Security Controls) → set **Password Expiration**, **Complexity** rules.

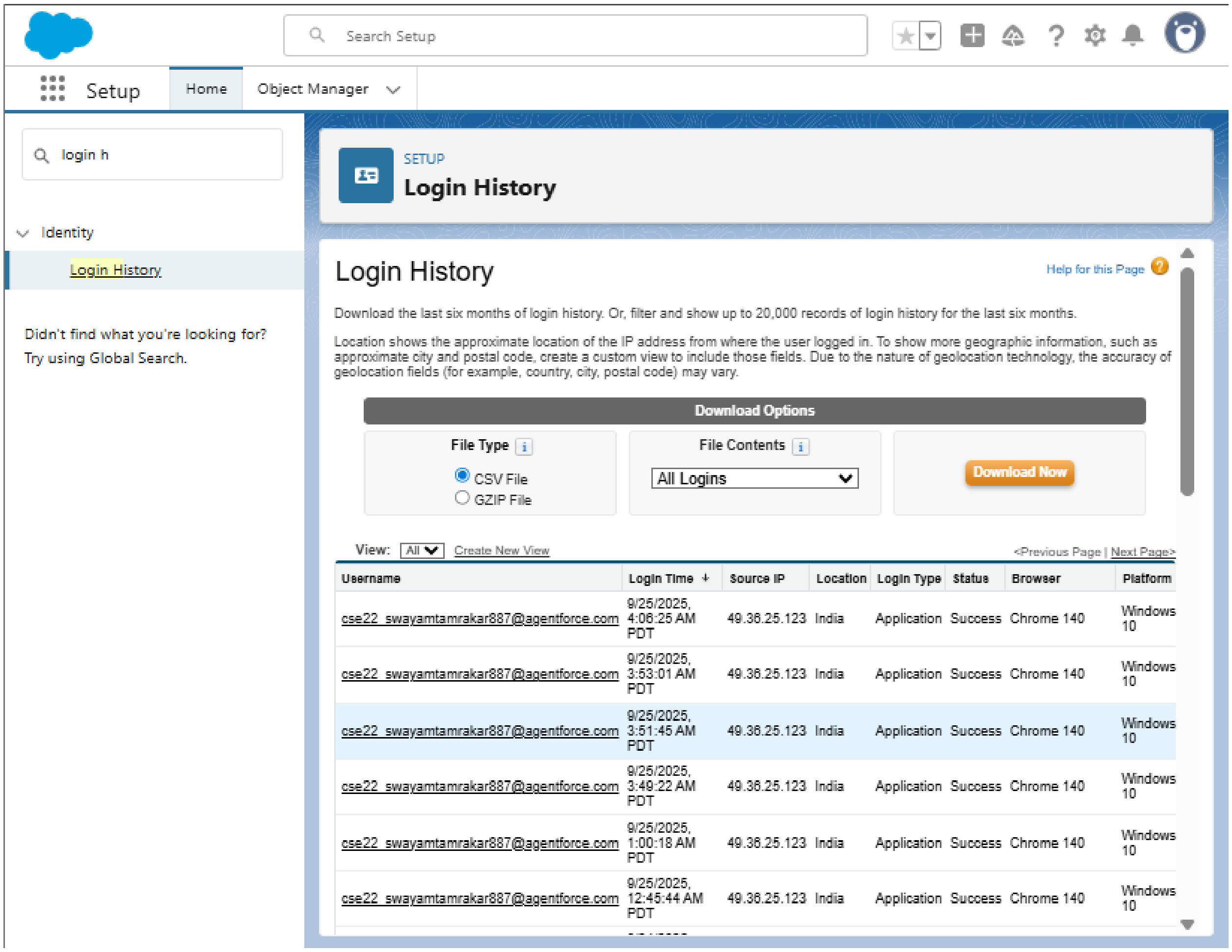


9.85 Audit Trail & Login History

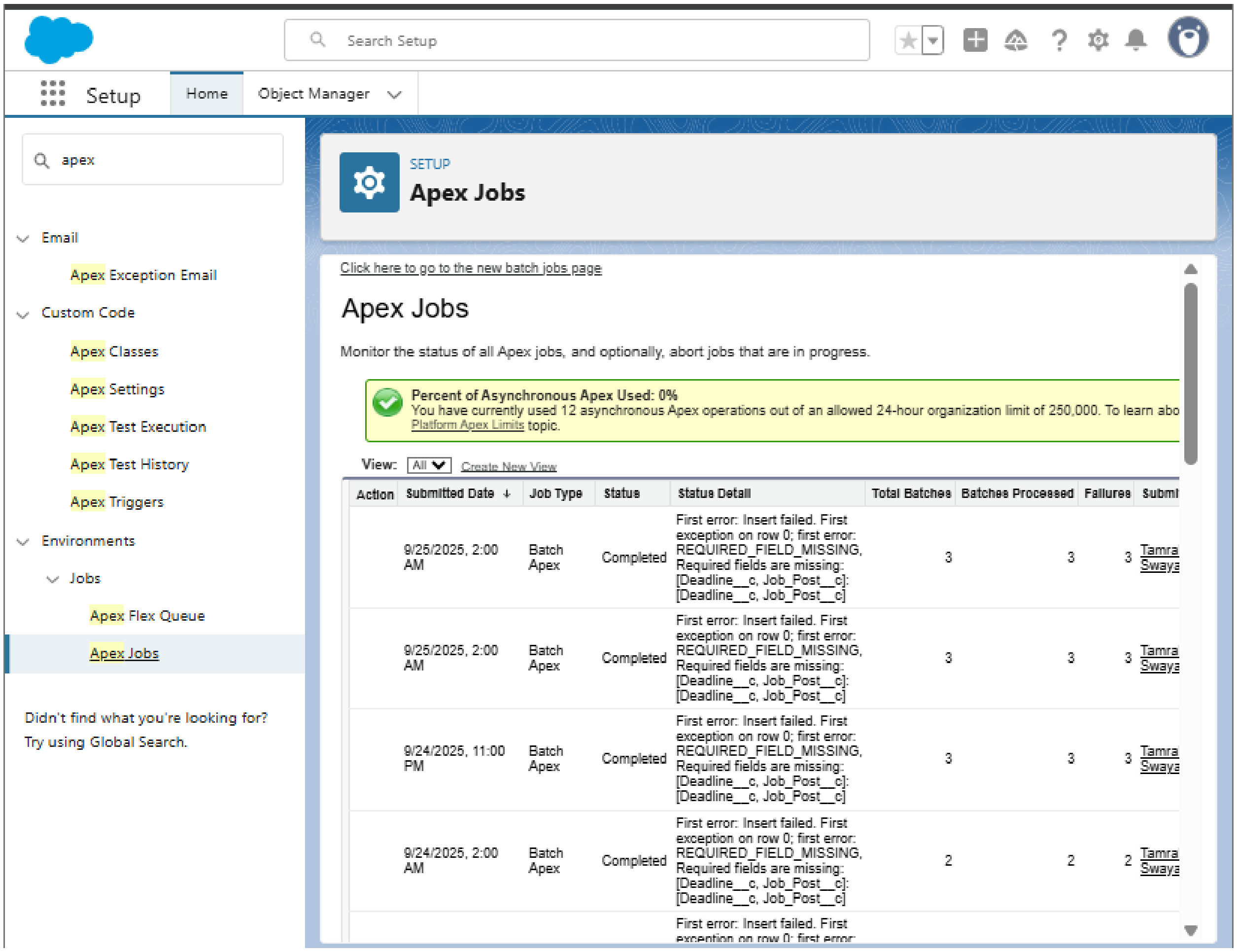
1. Setup → **View Setup Audit Trail** → review recent configuration changes.
   - Click **Download** to export last 180 days.



2. Setup → **Login History** → view user logins, failed logins, and IP addresses.

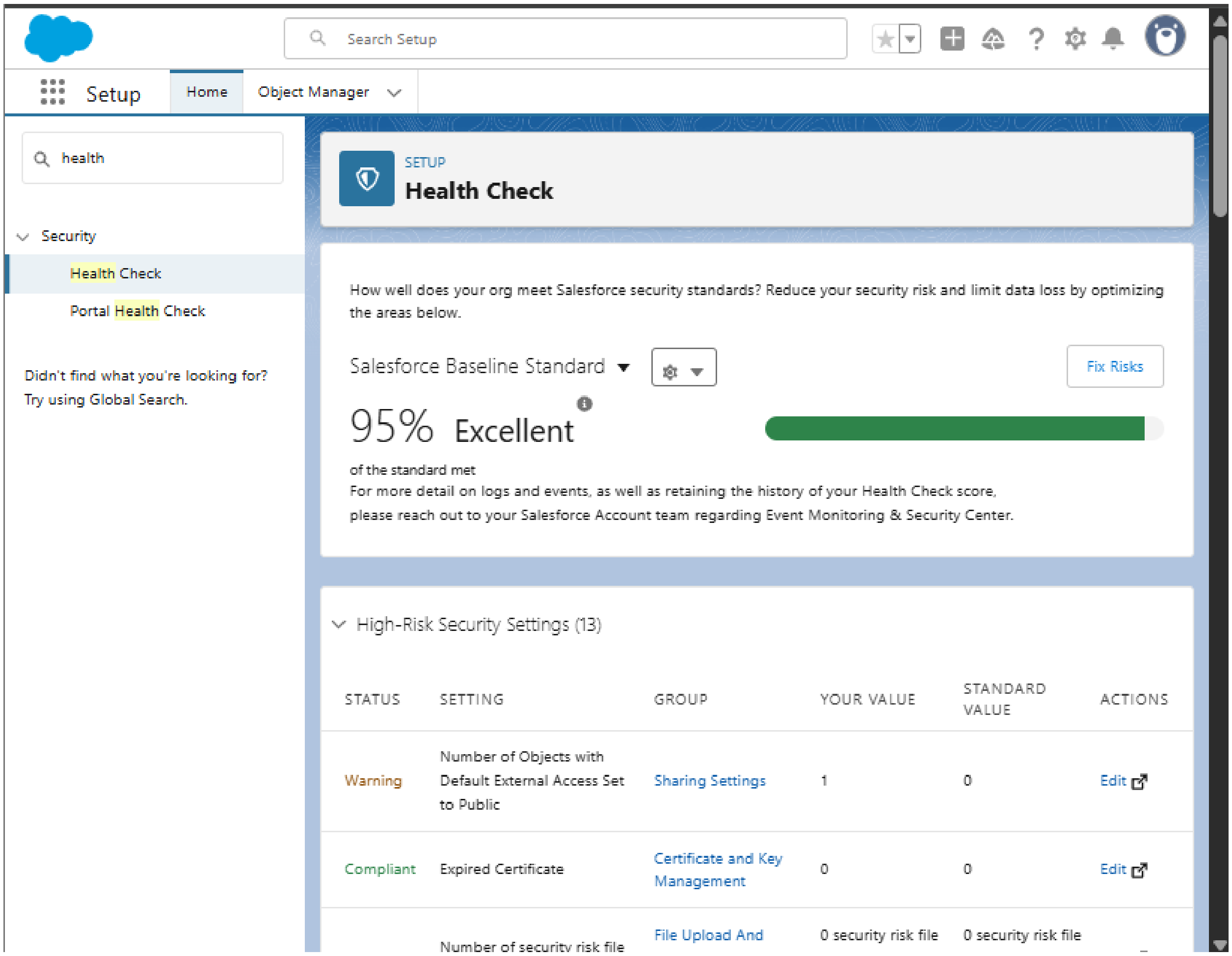


3. Setup → **Apex Jobs**→ check for background jobs running.



9.86 Health Check

1. Setup → **Health Check** → run and review your org's security score.
2. Address *High Risk* and *Medium Risk* settings (click an item → *Fix* suggestions).



Phase 10: Quality Assurance Testing

- ➢ In Each Phases I included test case for every Salesforce feature you implement (e.g., Record Creations in objects, Approval Process, Automatic Task Creation, Flows, Triggers, Validation Rules, etc.).

- ➢ Each test case includes:
- ✓ Input details (what data/action you are testing).
- ✓ Expected Output (what should happen).
- ✓ Actual Output (with results).

- ➢ Screenshots are also available for both input and output.

Demo-Video

Link:- **https://drive.google.com/file/d/10XkP4gLe-Mx-WTmDHyneO_UUNI5biH-a/view?usp=sharing**

Future Enhancements

To further improve the *Student Success & Career Guidance Portal*, the following enhancements can be implemented in future phases:

- **Chatbot Integration**:
  Integrate an AI-powered chatbot (via Salesforce Einstein Bots or external API) to answer student queries instantly, such as internship deadlines, skill recommendations, or company details.

- **Advanced AI Recommendations**:
  Extend the Career Recommendation engine with **Einstein AI / GPT-based suggestions** to provide personalized job and career guidance based on real-time student performance, skills, and job market trends.

- **Student Community Portal**:
  Build a **Salesforce Experience Cloud** portal where students can log in, update skills, view recommended jobs, and track placement status directly.

- **Mobile App Integration**:
  Enable access through Salesforce Mobile App for on-the-go tracking of student readiness and job applications.

- **Alumni Tracking Module**:
  Add alumni profiles and integrate feedback loops, so institutions can track long-term career success and maintain strong industry connections.
- **Third-Party LMS Integration**:
  Integrate with popular Learning Management Systems (Moodle, Canvas, Blackboard) to fetch real-time course completion and skill achievements.

Conclusion

The *Student Success & Career Guidance Portal* built on Salesforce Education Cloud successfully addresses the challenges faced by colleges in tracking student performance, internships, and placements. By providing a **360° student view**, automating processes through **Flows** (such as the Job Sync Flow), and enabling **analytics via dashboards and reports**, the solution streamlines placement operations and supports data-driven decision-making.

The project demonstrates how Salesforce can be customized for the education sector by leveraging **standard objects, custom objects, automation, and security features**. It also ensures scalability, as future enhancements like **chatbot integration, AI-driven recommendations, and community portals** can be added without disrupting the existing system.

In conclusion, this CRM project not only improves institutional efficiency but also enhances student employability by aligning academic progress with career opportunities. It stands as a strong example of how Salesforce can transform higher education management and placement processes.
